BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ DỰA TRÊN NỀN TẢNG BLOCKCHAIN

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Phạm Đức Khải – MSSV: AT190226

Hà Nội, 2026

BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



ĐỒ ÁN TỐT NGHIỆP

NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ DỰA TRÊN NỀN TẢNG BLOCKCHAIN

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Phạm Đức Khải – MSSV: AT190226

Người hướng dẫn:
TS. Trần Trung
Khoa Công nghệ thông tin – Trường Đại học Điện lực

Hà Nội, 2026

MỤC LỤC

MỤC LỤC ..... **i**
DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT ..... **vi**
DANH MỤC HÌNH VẼ ..... **ix**
DANH MỤC BẢNG ..... **x**
LỜI CẢM ƠN ..... **xii**
LỜI CAM ĐOAN ..... **xiii**
LỜI NÓI ĐẦU ..... **xiv**
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ CÔNG NGHỆ BLOCKCHAIN **1**
1.1. Tổng quan về công nghệ Blockchain 1
1.1.1. Khái niệm và mục tiêu hình thành *1*
1.1.2. Quá trình phát triển *1*
1.1.3. Đặc trưng và giới hạn *2*
1.1.4. DLT và cơ sở dữ liệu phân tán *2*
1.2. Cấu trúc dữ liệu và nền tảng mật mã 3
1.2.1. Giao dịch, khối và liên kết băm *3*
1.2.2. Hàm băm mật mã và cây Merkle *4*
1.2.3. Mật mã khóa công khai và chữ ký số *6*
1.2.4. Định danh, dấu thời gian và nonce *7*
1.3. Kiến trúc và nguyên lý hoạt động 7
1.3.1. Mạng ngang hàng và vai trò của nút *7*
1.3.2. Vòng đời giao dịch *8*
1.3.3. Sao chép trạng thái và thực thi xác định *9*
1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống *10*
1.4. Cơ chế đồng thuận 10
1.4.1. Bài toán đồng thuận và mô hình lỗi *10*
1.4.2. Proof of Work và Proof of Stake *11*
1.4.3. Đồng thuận CFT và BFT trong mạng định danh *12*
1.4.4. Tính hoàn tất và đánh đổi thiết kế *13*
1.5. Phân loại Blockchain và điều kiện lựa chọn 15
1.5.1. Các trục phân loại *15*
1.5.2. So sánh các mô hình triển khai *16*
1.5.3. Điều kiện áp dụng và giới hạn của Blockchain *17*
1.6. An toàn, quyền riêng tư và khả năng mở rộng 17

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp 17
1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài ... 19
1.6.3. Quyền riêng tư và dữ liệu cá nhân 19
1.6.4. Khả năng mở rộng và khả năng tương tác 20
1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ 20
1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng 20
1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi 21
1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm 22
1.7.4. Mô hình Blockcerts V3 và quan hệ với Verifiable Credentials 22
1.7.5. Bitcoin OP_RETURN và môi trường regtest 23
1.7.6. Bối cảnh pháp lý và ranh giới áp dụng tại Việt Nam 23
1.7.7. Đối chiếu các phương án và phạm vi đóng góp 23
1.8. Kết luận chương 1 24
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 26
2.1. Phân tích bài toán và phạm vi hệ thống 26
2.1.1. Quy trình nghiệp vụ hiện tại 26
2.1.2. Các bất cập cần giải quyết 27
2.1.3. Quy trình đề xuất 27
2.1.4. Phạm vi, giả định và ràng buộc 28
2.2. Thuật ngữ, tác nhân và quyền hạn 29
2.2.1. Quy ước thuật ngữ 29
2.2.2. Phân tích tác nhân 30
2.2.3. Mô hình use case tổng quát 30
2.3. Yêu cầu chức năng và đặc tả use case 31
2.3.1. Danh sách yêu cầu chức năng 31
2.3.2. UC-01 – Đăng nhập 32
2.3.3. UC-02 – Tạo yêu cầu phát hành 33
2.3.4. UC-03 – Phê duyệt hoặc từ chối 33
2.3.5. UC-04 – Xử lý phát hành bất đồng bộ 34
2.3.6. UC-05 – Xem và tải chứng thư của Student 35
2.3.7. UC-06 – Xác minh công khai 36
2.3.8. UC-07 – Thu hồi chứng thư 36
2.3.9. UC-08 – Quản trị và công bố thông tin hỗ trợ 37
2.4. Yêu cầu phi chức năng 38

2.5. Mô hình đe dọa 40
2.5.1. Phương pháp đánh giá 40
2.5.2. Ma trận tài sản–đe dọa–kiểm soát 40
2.6. Phân tích và lựa chọn kiến trúc 41
2.6.1. Blockchain công khai và Blockchain cấp quyền 41
2.6.2. Bitcoin, Ethereum và Hyperledger Fabric 42
2.6.3. Phân tách cam kết mật mã trên chuỗi và dữ liệu ngoài chuỗi 43
2.6.4. Đồng bộ, bất đồng bộ và phát hành theo lô 44
2.7. Kiến trúc và các luồng xử lý 44
2.7.1. Kiến trúc thành phần và ranh giới tin cậy 44
2.7.2. Trình tự phát hành theo lô 45
2.7.3. Máy trạng thái yêu cầu phát hành 46
2.7.4. Trình tự xác minh công khai 48
2.8. Thiết kế dữ liệu ngoài chuỗi 50
2.8.1. Mô hình dữ liệu đang triển khai 50
2.8.2. Lựa chọn định danh và ánh xạ kết quả 52
2.8.3. Mô hình chuẩn hóa đề xuất cho giai đoạn sau 53
2.8.4. Nguồn dữ liệu chuẩn đối với thu hồi và audit 53
2.9. Thiết kế API và phân quyền 54
2.9.1. Quy ước chung 54
2.9.2. Đặc tả endpoint chính 54
2.9.3. Ma trận phân quyền 55
2.10. Thiết kế ba cổng giao diện 56
2.10.1. Nguyên tắc thiết kế 56
2.10.2. Sơ đồ điều hướng 56
2.10.3. Đặc tả màn hình và trạng thái 57
2.11. An toàn triển khai và vận hành 59
2.11.1. Xác thực, phân quyền và bảo vệ API 59
2.11.2. Ranh giới mạng, secret và dữ liệu nhạy cảm 59
2.11.3. Bảo vệ trạng thái, outbox và phục hồi 60
2.11.4. Audit và giới hạn tính toàn vẹn 60
2.11.5. Giới hạn của môi trường pilot 61
2.12. Tiêu chí chấp nhận và ma trận truy vết 61
2.12.1. Tiêu chí chấp nhận 61

2.12.2. Ma trận FR–use case–thành phần–API–dữ liệu–kiểm thử 62
2.13. Kết luận chương 2 63
CHƯƠNG 3. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG 65
3.1. Môi trường và mô hình triển khai thực nghiệm 65
3.1.1. Môi trường phần cứng và thông số phòng thí nghiệm 65
3.1.2. Kiến trúc triển khai hệ thống 66
3.2. Quy trình cài đặt và triển khai hệ thống trên môi trường LAB 67
3.2.1. Chuẩn bị môi trường hệ điều hành và các gói phụ thuộc 69
3.2.2. Thiết lập cấu trúc thư mục và tải mã nguồn hệ thống 70
3.2.3. Khởi tạo cấu hình runtime và cơ chế bảo vệ bí mật 71
3.2.4. Khởi chạy cơ sở dữ liệu, bộ nhớ đệm và nút Bitcoin regtest 72
3.2.5. Khởi tạo lược đồ cơ sở dữ liệu và biên dịch ứng dụng 73
3.2.6. Cấu hình cặp khóa phát hành và mạng Blockcerts 74
3.2.7. Triển khai dịch vụ backend và ba cổng giao diện Web 75
3.2.8. Khởi tạo tài khoản người dùng và kiểm tra phân quyền RBAC 76
3.2.9. Kiểm tra tính sẵn sàng và các tiêu chí an toàn trước vận hành 77
3.2.10. Thiết lập công cụ đo lường và thu thập dữ liệu thực nghiệm 78
3.2.11. Quản trị vòng đời dịch vụ và bảo trì môi trường hệ thống 78
3.3. Hiện thực backend, dữ liệu và hàng đợi 79
3.3.1. Các mô-đun nghiệp vụ 79
3.3.2. Mô hình dữ liệu và liên kết người học 79
3.3.3. Outbox giao dịch và phục hồi phát hành 80
3.3.4. Audit và dữ liệu xác minh 81
3.4. Hiện thực phát hành, xác minh và thu hồi Blockcerts 81
3.4.1. Phát hành theo lô và gom lô Merkle 81
3.4.2. Xác minh công khai 82
3.4.3. Thu hồi có điểm kiểm soát và đối soát trạng thái 83
3.5. Thực nghiệm quy trình nghiệp vụ trên ba cổng giao diện Web 83
3.5.1. Khởi tạo tài khoản Maker, Checker và Student 84
3.5.2. Maker đăng nhập cổng nghiệp vụ, tra cứu hồ sơ người học và lập phiếu yêu cầu 85

3.5.3. Checker kiểm tra, phê duyệt và Worker phát hành chứng thư *86*
3.5.4. Người học tra cứu và tải chứng thư điện tử *88*
Xác minh văn bằng hợp lệ trên cổng xác minh công khai *89*
3.5.6. Thực nghiệm phát hiện văn bằng bị can thiệp, sửa đổi *90*
3.5.7. Checker thực hiện thu hồi và xác minh lại trạng thái văn bằng *91*
3.6. Kiểm soát an toàn và khả năng phục hồi 93
3.7. Phương pháp kiểm thử và kịch bản thực nghiệm 95
3.7.1. Phân tầng các cấp độ kiểm thử *95*
3.7.2. Phương pháp thu thập và kiểm toán dữ liệu thực nghiệm *95*
Kết quả thực nghiệm Kịch bản 1 – Tính toàn vẹn và kiểm soát Maker–Checker 96
Kết quả thực nghiệm Kịch bản 2 – Hiệu năng phát hành theo lô cây Merkle 97
3.9.1. Kết quả đo các đợt phát hành theo lô *97*
3.9.2. Phân tích giá trị tổng hợp và thông lượng xử lý *98*
3.9.3. Đánh giá mức độ sử dụng tài nguyên hệ thống *100*
3.9.4. Kiểm toán tính toàn vẹn của tệp dữ liệu kết quả *101*
Kết quả thực nghiệm Kịch bản 3 – Quy trình thu hồi, phân quyền RBAC và chuỗi băm nhật ký 102
3.11. Đánh giá kết quả, hạn chế và hướng phát triển 103
3.11.1. Mức độ đáp ứng yêu cầu *103*
3.11.2. Những điểm hạn chế của nguyên mẫu *104*
3.11.3. Hướng phát triển và hoàn thiện *105*
3.12. Kết luận chương 3 106
KẾT LUẬN 107
TÀI LIỆU THAM KHẢO 109
PHỤ LỤC 112
Phụ lục 1. Danh mục API chính và quyền truy cập 112
Phụ lục 2. Khung cấu hình tái lập và bảo vệ bí mật 113
Phụ lục 3. Quy trình tái lập và lệnh đại diện 114
Phụ lục 4. Bộ bằng chứng thực nghiệm chính thức 115
Phụ lục 5. Ma trận truy vết yêu cầu – hiện thực – bằng chứng 116

DANH MỤC KÝ HIỆU VÀ TỪ VIẾT TẮT

Ký hiệu / Từ viết tắt	Ý nghĩa / Từ đầy đủ
$\|\|$	Phép nối hai chuỗi ký tự hoặc chuỗi nhị phân
$H(x)$	Giá trị băm mật mã của dữ liệu đầu vào x
(sk, pk)	Cặp khóa bí mật (sk) và khóa công khai (pk) trong hệ mật khóa công khai
σ	Chữ ký số sinh bởi thuật toán ký $\text{Sign}(sk, m)$
$O(\cdot)$	Ký hiệu độ phức tạp tiệm cận (Big-O)
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
BFT	Byzantine Fault Tolerance
CFT	Crash Fault Tolerance
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
DB	Database
DBA	Database Administrator
DLT	Distributed Ledger Technology
DNS	Domain Name System
DPIA	Data Protection Impact Assessment
DTO	Data Transfer Object
E2E	End-to-End
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
FIPS	Federal Information Processing Standards
HA	High Availability
HSM	Hardware Security Module
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol

Ký hiệu / Từ viết tắt	Ý nghĩa / Từ đầy đủ
JSON	JavaScript Object Notation
JSON-LD	JavaScript Object Notation for Linked Data
JWT	JSON Web Token
KMS	Key Management Service / System
LAN	Local Area Network
LTS	Long-Term Support
MFA	Multi-Factor Authentication
MPC	Multi-Party Computation
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
P2PKH	Pay-to-Public-Key-Hash
PBFT	Practical Byzantine Fault Tolerance
PII	Personally Identifiable Information
PKI	Public Key Infrastructure
PoS	Proof of Stake
PoW	Proof of Work
RAM	Random Access Memory
RBAC	Role-Based Access Control
RPC	Remote Procedure Call
RSA	Rivest–Shamir–Adleman
RSS	Resident Set Size
SD	Standard Deviation (Độ lệch chuẩn)
SHA	Secure Hash Algorithm
SMR	State Machine Replication
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
TPS	Transactions Per Second
URL	Uniform Resource Locator
UTXO	Unspent Transaction Output
UUID	Universally Unique Identifier
VC	Verifiable Credential

Ký hiệu / Từ viết tắt	Ý nghĩa / Từ đầy đủ
W3C	World Wide Web Consortium
WAF	Web Application Firewall
WIF	Wallet Import Format

DANH MỤC HÌNH VẼ

Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp 4
Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch 5
Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát 9
Hình 1.4. Phân loại Blockchain theo quyền tham gia và phạm vi quản trị 15
Hình 1.5. Các lớp bảo vệ trong một hệ thống Blockchain 18
Hình 1.6. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng 21
Hình 2.1. Quy trình quản lý và xác minh văn bằng theo mô hình hiện tại 27
Hình 2.2. Quy trình đề xuất với kiểm soát Maker–Checker và neo Bitcoin 28
Hình 2.3. Tác nhân và nhóm use case trong ranh giới hệ thống 31
Hình 2.4. Kiến trúc thành phần và ranh giới tin cậy 44
Hình 2.5. Trình tự phát hành chứng thư theo lô 46
Hình 2.6. Máy trạng thái yêu cầu phát hành và thu hồi 47
Hình 2.7. Trình tự xác minh công khai và các nhánh kết quả 49
Hình 2.8. Mô hình dữ liệu vật lý hiện tại và liên kết logic 51
Hình 2.9. Sơ đồ điều hướng ba cổng giao diện 57
Hình 3.1. Kiến trúc triển khai của hệ thống trong môi trường thực nghiệm 67
Hình 3.2. Sơ đồ quy trình triển khai hệ thống trên môi trường phòng thí nghiệm . 69
Hình 3.3. Màn hình đăng nhập cổng nghiệp vụ dùng chung cho Maker và Checker 85
Hình 3.4. Maker đối chiếu hồ sơ Student và nhập dữ liệu văn bằng 86
Hình 3.5. Phiếu được tạo và chờ Checker phê duyệt 86
Hình 3.6. Checker kiểm tra phiếu đang chờ phê duyệt 87
Hình 3.7. Danh sách văn bằng sau khi Worker phát hành 88
Hình 3.8. Student xem và tải chứng thư thuộc đúng tài khoản 89
Hình 3.9. Cổng Verify xác nhận chứng thư hợp lệ 90
Hình 3.10. Verifier phát hiện chứng thư đã bị thay đổi 91
Hình 3.11. Checker thu hồi chứng thư và hệ thống cập nhật trạng thái 92
Hình 3.12. Cổng Verify trả trạng thái REVOKED cho chứng thư gốc 93
Hình 3.13. Thời gian phát hành trung bình theo kích thước lô 99
Hình 3.14. Thông lượng và thời gian trung bình trên mỗi chứng thư 100
Hình 3.15. Mức sử dụng tài nguyên theo kích thước lô 101

DANH MỤC BẢNG

Bảng 1.1. So sánh đặc tính kỹ thuật giữa các nhóm cơ chế đồng thuận tiêu biểu .. 13
Bảng 1.2. So sánh khái quát các mô hình triển khai Blockchain 16
Bảng 1.3. Đối chiếu các phương án quản lý và kiểm chứng văn bằng 24
Bảng 2.1. Quy trình nghiệp vụ hiện tại và điểm yếu cần xử lý..... 26
Bảng 2.2. Vai trò và giới hạn của Blockchain theo công đoạn..... 28
Bảng 2.3. Quy ước thuật ngữ trong đồ án..... 29
Bảng 2.4. Mục tiêu, quyền hạn và giới hạn của tác nhân 30
Bảng 2.5. Danh sách yêu cầu chức năng..... 31
Bảng 2.6. Đặc tả UC-01 – Đăng nhập..... 32
Bảng 2.7. Đặc tả UC-02 – Tạo yêu cầu phát hành 33
Bảng 2.8. Đặc tả UC-03 – Phê duyệt hoặc từ chối..... 34
Bảng 2.9. Đặc tả UC-04 – Xử lý phát hành bất đồng bộ 34
Bảng 2.10. Đặc tả UC-05 – Xem và tải chứng thư của Student..... 35
Bảng 2.11. Đặc tả UC-06 – Xác minh công khai 36
Bảng 2.12. Đặc tả UC-07 – Thu hồi chứng thư..... 37
Bảng 2.13. Đặc tả UC-08 – Quản trị và công bố thông tin hỗ trợ 38
Bảng 2.14. Yêu cầu phi chức năng và trạng thái thiết kế 38
Bảng 2.15. Ma trận tài sản–đe dọa–kiểm soát..... 40
Bảng 2.16. So sánh Blockchain công khai và Blockchain cấp quyền..... 42
Bảng 2.17. So sánh nền tảng cho phạm vi đồ án..... 42
Bảng 2.18. Chuyển trạng thái yêu cầu và lô phát hành 47
Bảng 2.19. Các bảng dữ liệu vật lý hiện tại 52
Bảng 2.20. Đặc tả các endpoint chính..... 54
Bảng 2.21. Ma trận phân quyền chức năng..... 55
Bảng 2.22. Đặc tả màn hình và trạng thái giao diện..... 58
Bảng 2.23. Tiêu chí chấp nhận 61
Bảng 2.24. Ma trận truy vết yêu cầu–thiết kế–kiểm thử..... 62
Bảng 3.1. Thành phần phần mềm của môi trường thực nghiệm 65
Bảng 3.2. Nhóm cấu hình runtime và nguyên tắc bảo vệ 72
Bảng 3.3. Điều kiện hoàn thành giai đoạn triển khai 76
Bảng 3.4. Các thực thể dữ liệu trực tiếp tham gia luồng phát hành 80
Bảng 3.5. Kiểm soát an toàn và bằng chứng kiểm thử 93
Bảng 3.6. Kết quả kiểm thử tự động và E2E..... 95
Bảng 3.7. Chỉ số kiểm soát cuối của bộ bằng chứng..... 96

Bảng 3.8. Kết quả Kịch bản 1 96
Bảng 3.9. Chín lần chạy hiệu năng tái kiểm tra 97
Bảng 3.10. Tổng hợp theo kích thước lô 98
Bảng 3.11. Tài nguyên hệ thống theo kích thước lô 100
Bảng 3.12. Kết quả kiểm toán tệp kết quả 102
Bảng 3.13. Kết quả Kịch bản 3 102
Bảng 3.14. Một số kiểm tra E2E bổ sung của phiên bản hiện tại 103
Bảng 3.15. Đối chiếu yêu cầu chính và bằng chứng 103
Bảng PL1.1. API chính của nguyên mẫu 112
Bảng PL2.1. Nhóm cấu hình và mức nhạy cảm 113
Bảng PL4.1. Thành phần bộ bằng chứng và mục đích 115
Bảng PL5.1. Ma trận truy vết yêu cầu và bằng chứng 116

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp này, tôi đã nhận được sự hướng dẫn tận tình của TS. Trần Trung – Giảng viên Khoa Công nghệ thông tin, Trường Đại học Điện lực. Tôi xin trân trọng cảm ơn ThS. Nguyễn Đức Ngân – Giảng viên Khoa An toàn thông tin, Học viện Kỹ thuật Mật mã – cùng cán bộ Hệ Quản lý sinh viên đã hỗ trợ tôi trong suốt quá trình học tập và hoàn thiện đồ án. Tôi cũng xin chân thành cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện tốt nhất để tôi hoàn thành đồ án tốt nghiệp này!

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Phạm Đức Khải

LỜI CAM ĐOAN

Tôi xin cam đoan đây là công trình nghiên cứu thực sự của riêng tôi, dưới sự hướng dẫn của TS. Trần Trung. Mọi tham khảo từ các tài liệu, công trình nghiên cứu liên quan trong nước hay quốc tế đều được trích dẫn rõ ràng trong đồ án. Mọi sao chép không hợp lệ, vi phạm quy chế hay gian lận tôi xin chịu hoàn toàn trách nhiệm và chịu mọi kỷ luật của Học viện Kỹ thuật Mật mã.

Hà Nội, ngày 06 tháng 11 năm 2026

SINH VIÊN THỰC HIỆN ĐỒ ÁN

Phạm Đức Khải

LỜI NÓI ĐẦU

Trong quá trình chuyển đổi số giáo dục, các cơ sở đào tạo đang từng bước chuyển từ văn bằng giấy sang hồ sơ học tập và văn bằng, chứng chỉ số. Cơ sở dữ liệu tập trung vẫn là nền tảng chính để nhà trường lưu trữ hồ sơ và xử lý nghiệp vụ đào tạo. Tuy nhiên, khi người học sử dụng văn bằng sau khi tốt nghiệp, bên thứ ba thường phải xác minh thông tin thông qua cổng tra cứu riêng của từng cơ sở đào tạo hoặc thực hiện đối soát thủ công. Nếu dữ liệu nội bộ bị can thiệp, tệp văn bằng bị chỉnh sửa hoặc cổng tra cứu tạm thời không khả dụng, việc kiểm tra nguồn gốc và trạng thái hiệu lực của văn bằng sẽ gặp nhiều khó khăn. Vì vậy, hệ thống quản lý văn bằng cần kết hợp số hóa hồ sơ với các cơ chế bảo đảm tính toàn vẹn, phân tách thẩm quyền phê duyệt và hỗ trợ kiểm chứng độc lập.

Công nghệ chuỗi khối (Blockchain) kết hợp hàm băm mật mã, chữ ký số, cây Merkle và cơ chế đồng thuận để tạo lập một sổ cái lưu giữ bằng chứng dữ liệu có khả năng phát hiện thay đổi trong lịch sử ghi nhận [1], [2]. Trong lĩnh vực giáo dục, chuẩn mở Blockcerts do MIT Media Lab khởi xướng [3] tổ chức hồ sơ văn bằng theo hướng lưu dữ liệu chi tiết ngoài chuỗi, đồng thời đưa giá trị gốc Merkle đại diện cho từng đợt cấp phát lên mạng Blockchain để hỗ trợ kiểm chứng độc lập [4], [5]. Cách tiếp cận này không thay thế cơ sở dữ liệu đào tạo hoặc quy trình xét duyệt tốt nghiệp của nhà trường. Nó bổ sung một lớp bằng chứng kỹ thuật, giúp phát hiện việc thay đổi dữ liệu sau thời điểm chứng thư được phát hành.

Tại Việt Nam, các quy định về văn bằng số và bảo vệ thông tin người học đang được hoàn thiện. Thông tư số 10/2026/TT-BGDĐT của Bộ Giáo dục và Đào tạo quy định việc quản lý, khai thác và xác minh văn bằng, chứng chỉ số [6]. Bên cạnh đó, Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 và Nghị định số 356/2025/NĐ-CP đặt ra các yêu cầu về xử lý và bảo vệ thông tin cá nhân trên môi trường mạng [7], [8]. Từ yêu cầu thực tiễn và bối cảnh pháp lý đó, đồ án lựa chọn đề tài "Nghiên cứu phát triển hệ thống thông tin hỗ trợ quản lý văn bằng chứng chỉ dựa trên nền tảng Blockchain". Trong phạm vi nghiên cứu, hệ thống được xây dựng dưới dạng nguyên mẫu hỗ trợ lập yêu cầu, phê duyệt, phát hành, thu hồi và xác minh chứng thư theo Blockcerts V3. Dữ liệu cá nhân của người học được quản lý ngoài chuỗi; việc neo bằng chứng mật mã được thực nghiệm trên Bitcoin regtest để kiểm tra quy trình mà không phát sinh chi phí giao dịch thực tế [9].

Đồ án tập trung giải quyết các mục tiêu chính sau:

- Hệ thống hóa cơ sở lý thuyết về Blockchain, mô hình chứng thư Blockcerts V3, cơ chế neo dữ liệu trên Bitcoin và các quy định liên quan đến văn bằng số, bảo vệ dữ liệu cá nhân tại Việt Nam;
- Phân tích, thiết kế kiến trúc hệ thống quản lý văn bằng với quy trình kiểm soát chéo Maker–Checker, cơ chế phân quyền theo vai trò và phạm vi đối tượng, cùng hàng đợi bền vững và checkpoint nhằm hạn chế phát hành trùng khi xảy ra sự cố;
- Xây dựng nguyên mẫu gồm ba cổng giao diện: cổng nghiệp vụ cho Maker và Checker, cổng Holder dành cho người học và cổng Verify công khai; hệ thống hỗ trợ vòng đời từ lập phiếu, phê duyệt, phát hành theo lô, xác minh, thu hồi đến kiểm toán nhật ký;
- Triển khai nguyên mẫu trên môi trường phòng thí nghiệm, thực hiện kiểm thử chức năng, kiểm thử an toàn, kiểm thử khả năng phục hồi và đo hiệu năng phát hành theo lô.

Nội dung chính của đồ án gồm ba chương:

- Chương 1 trình bày cơ sở lý thuyết về Blockchain, nền tảng mật mã, cơ chế đồng thuận và định hướng ứng dụng Blockcerts V3 trong quản lý văn bằng, chứng chỉ;
- Chương 2 phân tích bài toán nghiệp vụ, xây dựng mô hình đe dọa, thiết kế kiến trúc hệ thống, mô tả các luồng phát hành, xác minh và thu hồi, đồng thời trình bày thiết kế dữ liệu ngoài chuỗi và API;
- Chương 3 trình bày quy trình triển khai hệ thống trong môi trường phòng thí nghiệm, các kịch bản kiểm thử, kết quả đo hiệu năng, minh họa vận hành trên ba cổng Web và bộ bằng chứng thực nghiệm.

Do thời gian và phạm vi thử nghiệm còn giới hạn, đồ án vẫn có thể còn thiếu sót. Các kết quả được trình bày trong phạm vi nguyên mẫu và môi trường Bitcoin regtest; vì vậy, không nên suy rộng trực tiếp thành kết luận về khả năng vận hành trên Bitcoin mainnet hoặc trong môi trường sản xuất.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ CÔNG NGHỆ BLOCKCHAIN

1.1. Tổng quan về công nghệ Blockchain

1.1.1. Khái niệm và mục tiêu hình thành

Công nghệ chuỗi khối (Blockchain) là một dạng sổ cái phân tán (Distributed Ledger Technology – DLT), trong đó các giao dịch được tập hợp thành từng khối và liên kết theo trình tự thời gian [1], [2]. Mỗi khối thường chứa giá trị băm của khối liền trước, nhờ đó hình thành chuỗi liên kết mật mã. Thay vì duy trì dữ liệu trên một máy chủ duy nhất, hệ thống sao chép và đồng bộ sổ cái trên nhiều nút theo các quy tắc của giao thức và cơ chế đồng thuận [1], [2]. Khi dữ liệu của một khối bị thay đổi, giá trị băm của khối đó và các liên kết tiếp theo cũng thay đổi; sự sai lệch có thể được phát hiện khi đối chiếu với các bản sao hợp lệ trên mạng [1], [10].

Mục tiêu cốt lõi của Blockchain là cho phép nhiều tổ chức độc lập cùng ghi nhận và kiểm tra một lịch sử dữ liệu chung mà không phải giao toàn bộ quyền kiểm soát cho một trung tâm duy nhất [1]. Trong mô hình truyền thống, mỗi đơn vị thường quản lý một cơ sở dữ liệu riêng; việc đối chiếu giữa các bên vì thế phụ thuộc vào trung gian, mất thời gian và dễ gián đoạn khi hệ thống trung tâm không khả dụng. Blockchain kết hợp mật mã khóa công khai, cây Merkle, mạng ngang hàng và cơ chế đồng thuận để các bên có thể tự kiểm tra tính toàn vẹn của dữ liệu theo cùng một bộ quy tắc [2], [10].

Blockchain không thay thế cơ sở dữ liệu truyền thống và cũng không phù hợp để lưu trữ các tệp dung lượng lớn. Một hệ thống Blockchain vẫn cần quy chế quản trị về thành viên, quyền truy cập, quản lý khóa, phê duyệt giao dịch và nâng cấp phần mềm [1]. Vì vậy, công nghệ này phù hợp hơn với các bài toán có nhiều bên cùng tham gia, cần kiểm chứng chéo dữ liệu nhưng không muốn một tổ chức đơn lẻ toàn quyền sửa đổi lịch sử đã xác nhận [10].

1.1.2. Quá trình phát triển

Blockchain hình thành từ sự kết hợp của nhiều kết quả nghiên cứu trong mật mã học và hệ thống phân tán. Các nghiên cứu về mật mã khóa công khai, cây Merkle [11] và dấu thời gian cho tài liệu điện tử đã cung cấp nền tảng để kiểm tra tính toàn vẹn và thứ tự của dữ liệu. Song song với đó, các thuật toán đồng thuận chịu lỗi được phát triển để giúp các máy chủ thống nhất trạng thái ngay cả khi một số nút bị ngắt kết nối hoặc phát tán thông tin sai lệch [12], [13].

Bước ngoặt quan trọng diễn ra vào năm 2008 khi Satoshi Nakamoto công bố Bitcoin, hệ thống kết hợp chuỗi khối liên kết bằng giá trị băm với Proof of Work để xử lý bài toán chi tiêu kép mà không cần trung gian tập trung [2]. Từ nền tảng ban

đầu dành cho giao dịch tiền mã hóa, Blockchain được mở rộng với hợp đồng thông minh, cho phép mô tả và thực thi tự động một số quy tắc nghiệp vụ trên sổ cái [1], [10]. Bên cạnh mạng công khai, các mạng cấp quyền như Hyperledger Fabric được phát triển cho những môi trường mà thành viên có danh tính và các tổ chức cần phối hợp kiểm toán, đối soát dữ liệu [1], [14].

1.1.3. Đặc trưng và giới hạn

Các hệ thống chuỗi khối thường có bốn đặc trưng kỹ thuật: lưu trữ phân tán, khả năng tự kiểm tra, khả năng truy vết giao dịch và khả năng phát hiện việc thay đổi dữ liệu lịch sử [1]. Giao dịch hợp lệ được xác nhận bằng chữ ký số và liên kết với các khối khác thông qua cơ chế băm, giúp các bên đối chiếu lịch sử theo cùng một quy tắc [2], [10]. Tuy nhiên, những đặc trưng này không đồng nghĩa với việc Blockchain bảo đảm mọi thông tin đầu vào là đúng hoặc hệ thống an toàn trong mọi điều kiện. Cần tránh ba cách hiểu sai phổ biến sau:

Thứ nhất, dữ liệu được ghi lên chuỗi không vì thế mà trở thành đúng trong thực tế. Blockchain chủ yếu bảo vệ tính toàn vẹn của bản ghi kể từ khi bản ghi được chấp nhận. Nếu thông tin đã bị nhập sai hoặc bị làm giả ở quy trình ngoài chuỗi, hệ thống vẫn có thể lưu giữ nguyên vẹn dữ liệu sai đó [1].

Thứ hai, không nên hiểu dữ liệu trên chuỗi là khó bị thay đổi. Khả năng chống thay đổi phụ thuộc vào thuật toán mật mã, cơ chế đồng thuận và ngưỡng chịu lỗi của mạng. Nếu một tác nhân kiểm soát đủ năng lực tính toán hoặc đủ số nút để vượt ngưỡng an toàn, lịch sử giao dịch vẫn có thể bị phân nhánh hoặc bị viết lại trong phạm vi mô hình đe dọa tương ứng [1], [12].

Thứ ba, Blockchain không tự giải quyết vấn đề bảo vệ khóa, quyền riêng tư hay phân quyền nghiệp vụ. Nếu khóa bí mật của đơn vị phát hành bị lộ, kẻ tấn công có thể tạo chữ ký hợp lệ về mặt toán học cho giao dịch giả mạo. Tương tự, việc ghi trực tiếp dữ liệu nhạy cảm lên sổ cái dùng chung có thể làm lộ thông tin cá nhân và gây khó khăn cho việc hạn chế hoặc xóa dữ liệu [1], [10].

1.1.4. DLT và cơ sở dữ liệu phân tán

Sổ cái phân tán là khái niệm rộng, chỉ các kiến trúc cho phép nhiều nút cùng lưu trữ và đồng bộ dữ liệu mà không phụ thuộc hoàn toàn vào một máy chủ trung tâm [1]. Blockchain là một dạng sổ cái phân tán trong đó dữ liệu thường được tổ chức thành các khối nối tiếp và liên kết bằng giá trị băm. Những dạng DLT khác có thể sử dụng cấu trúc liên kết khác, chẳng hạn đồ thị hoặc mạng rẽ nhánh, tùy theo thiết kế của nền tảng [1], [10].

DLT hướng tới những mô hình trong đó nhiều chủ thể cùng duy trì lịch sử dữ liệu theo các quy tắc thống nhất [1], [14]. Với hệ thống liên tổ chức, quyền cập nhật cần được phân bổ để không một tài khoản hoặc tổ chức đơn lẻ nào có thể tự ý sửa lịch sử đã xác nhận [10]. Tuy nhiên, mức độ phân quyền thực tế còn phụ thuộc vào số tổ chức vận hành, chính sách nâng cấp và khả năng các bên thông đồng [1]. Vì vậy, điểm khác biệt giữa Blockchain và một cơ sở dữ liệu phân tán truyền thống không chỉ nằm ở số lượng bản sao, mà còn ở ranh giới tin cậy và cách các bên thống nhất thay đổi trạng thái [1], [10].

1.2. Cấu trúc dữ liệu và nền tảng mật mã

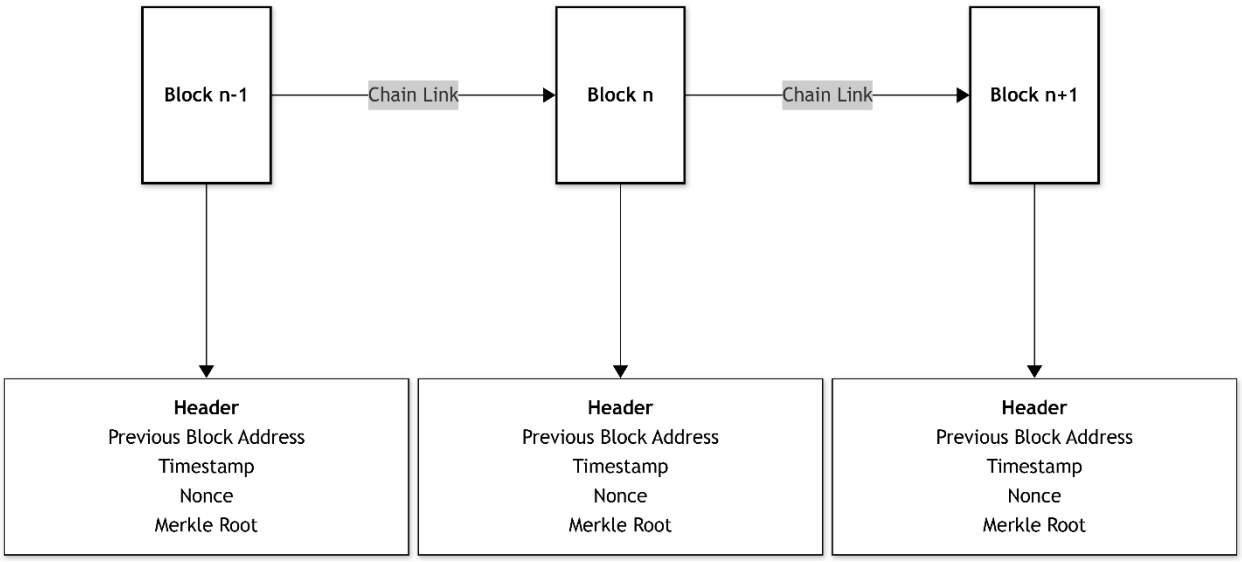
1.2.1. Giao dịch, khối và liên kết băm

Giao dịch là đơn vị xử lý độc lập biểu diễn một yêu cầu thay đổi trạng thái trong hệ thống sổ cái phân tán [1], [10]. Tùy thuộc vào nền tảng cụ thể, mỗi giao dịch thường chứa thông tin định danh hoặc địa chỉ của người gửi, dữ liệu đầu vào, cơ chế chống phát lại giao dịch cũ, các tham số phí và chữ ký số [1], [2]. Khi được phát tán vào mạng lưới, các nút tiếp nhận sẽ tiến hành kiểm tra độc lập tính hợp lệ về mặt cú pháp, tính toàn vẹn của chữ ký số và các điều kiện logic của trạng thái hiện hành trước khi chấp nhận đưa vào hàng đợi xử lý [1], [2].

Sau khi được kiểm tra và sắp xếp, các giao dịch được đóng gói thành khối. Một khối thường gồm phần đầu chứa siêu dữ liệu của giao thức và phần thân chứa danh sách giao dịch [1], [2]. Tùy nền tảng, phần đầu có thể chứa giá trị băm của khối trước, gốc cây Merkle, dấu thời gian và các trường phục vụ đồng thuận hoặc quản trị. Mối liên kết giữa các khối được mô tả bằng biểu thức (1.1):

$$H_i = H(H_{i-1} \parallel \text{MerkleRoot}_i \parallel \text{Timestamp}_i \parallel \text{Metadata}_i) \qquad (1.1)$$

Trong biểu thức (1.1), toán tử $\parallel$ biểu thị phép nối chuỗi ký tự; đây là mô hình minh họa chung còn tập hợp các trường dữ liệu cụ thể do từng giao thức quy định. Giá trị băm H_{i-1} chính là yếu tố tạo ra liên kết kiểm chứng mật mã giữa hai khối liên tiếp: khối thứ i chỉ hợp lệ khi chứa chính xác giá trị băm của khối thứ $i - 1$ [1], [2]. Cấu trúc liên kết mật mã này được minh họa cụ thể trên Hình 1.1.



Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp

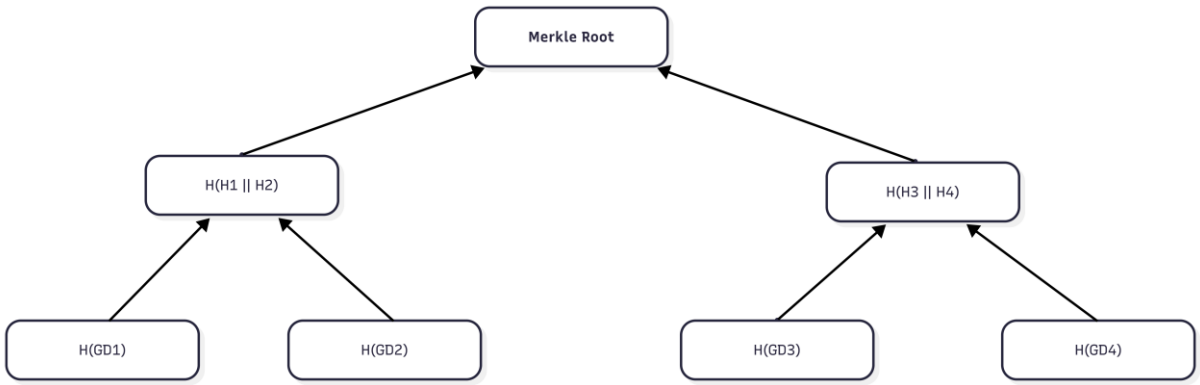
Nếu dữ liệu trong một khối cũ bị thay đổi, giá trị băm của giao dịch, gốc cây Merkle và giá trị băm của khối đó cũng thay đổi [15]. Tham chiếu tại khối kế tiếp khi đó không còn khớp, giúp phát hiện dấu hiệu can thiệp vào lịch sử. Tuy nhiên, để bản ghi thay đổi được mạng chấp nhận, tác nhân tấn công còn phải vượt qua các quy tắc đồng thuận và chính sách quản trị; do đó, an toàn thực tế không chỉ phụ thuộc vào liên kết băm [1], [2].

1.2.2. Hàm băm mật mã và cây Merkle

Hàm băm mật mã đóng vai trò là công cụ toán học nền tảng đảm bảo tính toàn vẹn dữ liệu trong kiến trúc chuỗi khối [1], [15]. Một hàm băm mật mã H thực hiện phép biến đổi một thông điệp đầu vào M có độ dài tùy ý thành một chuỗi nhị phân có kích thước cố định $h = H(M)$, tiêu biểu như chuẩn SHA-256 với không gian đầu ra 256-bit [15]. Về mặt an toàn thông tin, một hàm băm mật mã chất lượng cao phải thỏa mãn ba thuộc tính cốt lõi với mức độ chặt chẽ tăng dần:

- Tính kháng tiền ảnh: Cho trước một giá trị băm đầu ra y, việc tìm ngược lại thông điệp gốc x sao cho $H(x) = y$ là bất khả thi về mặt tính toán [15];
- Tính kháng tiền ảnh thứ hai: Cho trước một thông điệp x_1 cụ thể, việc tìm được một thông điệp đầu vào x_2 khác x_1 ($x_2 \neq x_1$) sao cho $H(x_2) = H(x_1)$ là bất khả thi về mặt tính toán [15];
- Tính kháng va chạm mạnh: Việc tìm kiếm một cặp thông điệp phân biệt bất kỳ (x_1, x_2) thỏa mãn $x_1 \neq x_2$ sao cho $H(x_1) = H(x_2)$ là bất khả thi về mặt tính toán. Bên cạnh đó, hiệu ứng tuyết lở đảm bảo rằng mọi thay đổi dù là nhỏ nhất tại dữ liệu đầu vào (thậm chí chỉ 1 bit) cũng sẽ làm thay đổi hoàn toàn và ngẫu nhiên giá trị băm đầu ra [15].

Để quản lý hiệu quả hàng nghìn giao dịch trong một khối mà không làm cồng kềnh bộ nhớ của các nút mạng, cấu trúc Cây Merkle được áp dụng để gom nhóm dữ liệu [11]. Cây Merkle thường được biểu diễn dưới dạng cây băm nhị phân, trong đó mỗi nút lá ở tầng thấp nhất lưu giá trị băm của một giao dịch đơn lẻ, còn mỗi nút trung gian cao hơn lưu giá trị băm của phép nối chuỗi hai nút con trực tiếp theo biểu thức $H(u,v) = H\big(H(u) \parallel H(v)\big)$ [11]. Quá trình ghép và băm từng cặp này thực hiện đệ quy từ nút lá lên đỉnh để tạo ra một giá trị đại diện duy nhất là Gốc Merkle, giá trị này sau đó được ghi trực tiếp vào phần đầu khối [11]. Trong thực tế, cây nhị phân cân bằng là mô hình minh họa phổ biến nhất, còn quy tắc cụ thể để xử lý trường hợp số lượng nút ở một tầng bị lẻ như nhân đôi nút cuối cùng hay giữ nguyên nút lẻ lên tầng trên sẽ do từng nền tảng quy định riêng. Cấu trúc cây và đường dẫn kiểm chứng được minh họa trên Hình 1.2.



Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch

Cấu trúc cây Merkle cho phép xác minh một giao dịch có thực sự nằm trong tập giao dịch đã cam kết hay không thông qua bằng chứng Merkle mà không cần tải về toàn bộ dữ liệu của khối [10], [11]. Đối với một cây nhị phân cân bằng chứa n phần tử, kích thước của bằng chứng và số lượng phép tính băm cần thực hiện khi xác minh chỉ tăng theo hàm logarit $O(\log n)$ theo số lượng phần tử trong cây [10], [11]. Cụ thể, nút kiểm tra chỉ cần lấy giá trị băm của giao dịch cần xác minh, lần lượt băm ghép cặp với các giá trị băm nhánh khác nằm trên đường dẫn kiểm chứng từ lá lên đỉnh để tính lại gốc Merkle, sau đó đối chiếu kết quả này với gốc Merkle đã được cam kết trong phần đầu khối [2], [11]. Dù vậy, cần phân biệt rõ rằng bằng chứng Merkle chỉ chứng minh sự tồn tại và tính toàn vẹn của bản ghi bên trong tập dữ liệu đã cam kết, chứ không tự động chứng minh chữ ký số hợp lệ, điều kiện chi tiêu hay các quy tắc nghiệp vụ khác của giao dịch; những điều kiện đó vẫn phải được hệ thống kiểm tra riêng theo quy định của giao thức [2].

1.2.3. Mật mã khóa công khai và chữ ký số

Trong hệ thống chuỗi khối, mật mã khóa công khai và chữ ký số được sử dụng để xác nhận nguồn gốc giao dịch và kiểm tra quyền cập nhật trạng thái trên sổ cái [1], [16]. Theo tiêu chuẩn NIST FIPS 186-5 [16], một lược đồ chữ ký số an toàn được mô tả về mặt toán học qua bộ ba thuật toán $(\mathrm{KeyGen}, \mathrm{Sign}, \mathrm{Verify})$. Trong đó, thuật toán sinh khóa $\mathrm{KeyGen}(1^{\lambda}) \rightarrow (sk, pk)$ nhận đầu vào là tham số an toàn λ để tạo cặp khóa bất đối xứng gồm khóa bí mật sk dùng cho việc ký và khóa công khai pk dùng cho việc xác minh chữ ký. Khi cần phát hành giao dịch, thuật toán ký $\mathrm{Sign}(sk, m) \rightarrow \sigma$ kết hợp thông điệp m với khóa bí mật sk để sinh ra chữ ký số σ. Ở chiều ngược lại, thuật toán xác minh $\mathrm{Verify}(pk, m, \sigma) \rightarrow \{\mathrm{Valid}, \mathrm{Invalid}\}$ tiếp nhận khóa công khai pk, thông điệp m cùng chữ ký σ để kiểm tra và trả về kết luận chữ ký có hợp lệ hay không. Mô hình tổng quát của lược đồ được biểu diễn theo công thức (1.2):

$$\sigma \leftarrow \mathrm{Sign}(sk, m); \quad \mathrm{Verify}(pk, m, \sigma) \in \{0, 1\} \tag{1.2}$$

Cần lưu ý rằng không nên hiểu chữ ký số một cách đơn giản hóa là việc đem mã hóa giá trị băm của thông điệp bằng khóa bí mật rồi giải mã lại bằng khóa công khai. Cách hình dung này chỉ xuất hiện trong mô hình minh họa sơ khai của hệ mật RSA và không phản ánh đúng bản chất toán học của các lược đồ chữ ký số hiện đại đang dùng trong chuỗi khối như ECDSA hay EdDSA trên đường cong Elliptic [16]. Ở mức tổng quát, thuật toán Verify thực hiện kiểm tra mối quan hệ toán học trực tiếp giữa bộ ba tham số gồm khóa công khai pk, thông điệp m và chữ ký σ để đưa ra kết quả hợp lệ hoặc không hợp lệ, chứ không thực hiện phép giải mã để khôi phục lại thông điệp ban đầu [16].

Về phương diện an toàn thông tin, chữ ký số đảm bảo ba tính chất quan trọng cho các giao dịch trên sổ cái [1], [16]: (1) Xác thực nguồn gốc: chỉ chủ thể nắm giữ khóa bí mật sk mới có thể tạo ra chữ ký hợp lệ tương ứng với khóa công khai pk đã công bố rộng rãi; (2) Bảo vệ tính toàn vẹn: nếu nội dung thông điệp m bị thay đổi dù chỉ một bit trên đường truyền mạng, phép kiểm tra $\mathrm{Verify}(pk, m', \sigma)$ sẽ lập tức trả về kết quả không hợp lệ; (3) Tính chống chối bỏ về mặt kỹ thuật: ngăn chặn người gửi phủ nhận việc đã phát hành giao dịch sau khi chữ ký hợp lệ đã được ghi nhận thành công lên sổ cái chuỗi khối [1], [16].

Dù vậy, chữ ký số chỉ chứng minh được sự khớp nhau về mặt toán học của bộ dữ liệu (m, σ, pk). Bản thân thuật toán không thể tự nhận biết việc người kích hoạt lệnh ký ngoài đời thực có đúng là chủ sở hữu hợp pháp hay không nếu khóa bí mật

sk đã bị lộ, bị mã độc đánh cắp hoặc bị trích xuất trái phép khỏi thiết bị lưu trữ [1], [16]. Do đó, độ tin cậy thực tế của chữ ký số trong các hệ thống cấp phát văn bằng vẫn phụ thuộc chặt chẽ vào quy trình quản lý vòng đời khóa, cơ chế bảo vệ thiết bị lưu khóa, việc định danh chủ thể gắn với khóa công khai, nhật ký giám sát thao tác ký và quy trình thu hồi khóa kịp thời khi xảy ra sự cố [1], [16].

1.2.4. Định danh, dấu thời gian và nonce

Cơ chế định danh trong chuỗi khối cho phép gắn mỗi giao dịch với một khóa mật mã cụ thể hoặc một thành viên đã được cấp quyền trong mạng lưới [1]. Ở các mạng công khai không cấp quyền, địa chỉ giao dịch thường được tính toán dẫn xuất từ khóa công khai thông qua hàm băm và đóng vai trò như một mã định danh đại diện mà không hiển thị tên thật của người dùng. Ngược lại, ở các mạng cấp quyền, khóa công khai có thể được gắn kết chặt chẽ với danh tính pháp nhân của từng tổ chức thông qua chứng thư số và hạ tầng khóa công khai [14]. Dù hoạt động theo mô hình nào, một chữ ký số hợp lệ trên sổ cái cũng chỉ chứng minh rằng giao dịch được tạo bởi bên đang nắm giữ khóa bí mật tương ứng, còn mối liên hệ giữa khóa đó với cá nhân hay tổ chức ngoài thực tế vẫn phụ thuộc hoàn toàn vào quy trình đăng ký, cấp phát và quản lý danh tính ở ngoài chuỗi.

Dấu thời gian trong khối hỗ trợ sắp xếp sự kiện và áp dụng các quy tắc có yếu tố thời hạn, nhưng không nên mặc nhiên xem đó là chứng cứ thời gian có giá trị pháp lý. Mạng phân tán không có một đồng hồ vật lý chung tuyệt đối; vì vậy mỗi giao thức phải quy định nguồn thời gian, sai lệch cho phép và cách loại bỏ khối có thời gian không hợp lệ [1], [2]. Nếu nghiệp vụ yêu cầu chứng thực thời gian theo pháp luật, hệ thống cần kết hợp dịch vụ đóng dấu thời gian có thẩm quyền ngoài Blockchain.

Nonce là giá trị do giao thức sử dụng để tạo tính duy nhất cho phép tính mật mã hoặc kiểm soát thứ tự giao dịch. Trong Proof of Work, nút khai thác thay đổi nonce trong phần đầu khối để tìm giá trị băm đáp ứng độ khó của mạng [2]. Ở các nền tảng dùng mô hình tài khoản, nonce thường là bộ đếm giao dịch của địa chỉ gửi, giúp phát hiện giao dịch bị gửi lại hoặc sai thứ tự. Cơ chế này hỗ trợ chống phát lại ở tầng giao dịch, nhưng không thay thế biện pháp bảo vệ phiên làm việc và kênh truyền ở tầng ứng dụng [1], [10].

1.3. Kiến trúc và nguyên lý hoạt động

1.3.1. Mạng ngang hàng và vai trò của nút

Chuỗi khối thường sử dụng mạng ngang hàng, trong đó các nút trao đổi giao dịch, khối và thông tin trạng thái mà không bắt buộc đi qua một máy chủ điều phối

trung tâm [1], [2]. Mỗi nút kiểm tra thông điệp theo quy tắc giao thức trước khi chuyển tiếp cho các nút lân cận. Cách tổ chức này giúp duy trì bản sao sổ cái khi một số nút hoặc đường truyền gặp sự cố. Tuy nhiên, mạng ngang hàng không đồng nghĩa với phân quyền tuyệt đối: các nút vẫn có thể cùng đặt trên một nhà cung cấp hạ tầng hoặc chịu sự kiểm soát của một nhóm vận hành nhỏ [1], [10].

Trong mạng lưới, các nút không nhất thiết phải đảm nhận công việc giống hệt nhau mà được phân chia vai trò tùy theo thiết kế của từng nền tảng [1], [14]. Cụ thể, nút đầy đủ lưu trữ toàn bộ lịch sử sổ cái hoặc đầy đủ trạng thái cần thiết để tự kiểm tra độc lập mọi giao dịch và khối dữ liệu mà không cần tin tưởng nút khác [1], [2]. Ngược lại, nút nhẹ chỉ lưu giữ phần đầu của các khối nhằm tiết kiệm dung lượng lưu trữ và băng thông, khi cần kiểm tra một giao dịch cụ thể sẽ dựa vào bằng chứng mật mã trên cây Merkle do nút đầy đủ cung cấp [2], [11]. Bên cạnh nhiệm vụ lưu trữ và kiểm tra, nhóm nút đề xuất hoặc nút xác thực còn trực tiếp tham gia vào quá trình đóng gói khối và chạy thuật toán đồng thuận để chốt trạng thái chung cho mạng lưới [1]. Ở một số nền tảng chuỗi khối cấp quyền như Hyperledger Fabric, kiến trúc hệ thống còn tách bạch hoàn toàn các vai trò thành những nút chuyên trách riêng biệt từ khâu thực thi logic nghiệp vụ, sắp xếp thứ tự giao dịch cho đến kiểm tra cuối cùng và ghi nhận vào sổ cái [1], [14]. Chính vì sự đa dạng này, các cách gọi phổ biến như nút đào, nút xác thực hay nút lưu trữ không phải là một bộ phân loại cố định áp dụng giống nhau cho mọi hệ thống chuỗi khối. Khi phân tích hoặc thiết kế kiến trúc thực tế, cần làm rõ từng loại nút trong hệ thống lưu trữ dữ liệu gì, tự kiểm chứng được những bước nào, thuộc sự quản lý của tổ chức nào và có quyền hạn ra sao đối với việc thay đổi trạng thái sổ cái.

1.3.2. Vòng đời giao dịch

Quá trình một yêu cầu thay đổi dữ liệu đi từ lúc khởi tạo cho đến khi được ghi nhận chính thức trên sổ cái trải qua nhiều bước kiểm soát chặt chẽ, được khái quát trên Hình 1.3. Vòng đời này bắt đầu khi chủ thể khởi tạo lập dữ liệu giao dịch và ký xác nhận bằng khóa bí mật của mình. Khi giao dịch được gửi tới một nút trên mạng lưới, nút tiếp nhận sẽ thực hiện kiểm tra sơ bộ về cấu trúc định dạng, tính hợp lệ của chữ ký số, bộ đếm chống phát lại giao dịch cũ, quyền hạn thao tác của người gửi và sự phù hợp với trạng thái hiện tại của sổ cái [1]. Những giao dịch vượt qua bước kiểm tra này mới được xem là hợp lệ và tiếp tục truyền tới các thành phần chịu trách nhiệm sắp xếp thứ tự hoặc đồng thuận. Ở các mạng công khai, giao dịch hợp lệ thường được lưu tạm vào vùng nhớ chờ xử lý trước khi nút tạo khối lựa chọn để đưa vào khối mới; trong khi đó, các mạng cấp quyền có thể áp dụng quy trình đề xuất,

ký chứng thực chéo giữa các nút và gom lô sắp thứ tự riêng mà không cần qua hoạt động đào khối [2], [14].



Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát

Sau khi đáp ứng điều kiện của giao thức đồng thuận, các giao dịch được đóng gói thành khối, phát tán và kiểm tra độc lập trước khi các nút cập nhật trạng thái. Tầng ứng dụng chỉ nên thông báo hoàn tất khi có bằng chứng giao dịch đã được đưa vào một khối hợp lệ; việc mới tạo mã giao dịch hoặc phát tán yêu cầu chưa chứng minh thao tác đã hoàn thành. Mức độ chắc chắn còn phụ thuộc cơ chế đồng thuận: mạng công khai có thể cần thêm các khối xác nhận, trong khi mạng cấp quyền có thể chốt trạng thái sau khi đạt đủ phiếu theo giao thức [1], [12].

1.3.3. Sao chép trạng thái và thực thi xác định

Về mặt toán học và kiến trúc hệ thống phân tán, chuỗi khối có thể được phân tích như một mô hình sao chép máy trạng thái (State Machine Replication – SMR) [10]. Gọi S là tập trạng thái hiện tại của hệ thống, T là giao dịch đầu vào và δ là hàm chuyển trạng thái quy định cách hệ thống cập nhật dữ liệu khi xử lý giao dịch [10]. Nguyên lý cốt lõi của mô hình này là nếu mọi nút trung thực trong mạng lưới đều bắt đầu từ cùng một trạng thái khởi tạo S_0 và xử lý chính xác cùng một chuỗi các giao dịch theo đúng thứ tự thời gian, thì kết quả thu được ở tất cả các nút phải hoàn toàn đồng nhất, tạo ra trạng thái kế tiếp theo biểu thức $S' = \delta(S, T)$ [10].

Để duy trì nhất quán giữa các bản sao, logic xử lý giao dịch phải có tính xác định: cùng một đầu vào và cùng một trạng thái trước xử lý phải tạo ra cùng một kết quả trên mọi nút [2], [10]. Vì vậy, chương trình không nên phụ thuộc trực tiếp vào đồng hồ cục bộ, nguồn ngẫu nhiên không được thống nhất, dịch vụ Internet bên ngoài hoặc cách làm tròn khác nhau giữa các kiến trúc phần cứng [10].

Trong thực tiễn thiết kế kiến trúc, nhiều nền tảng thường tách biệt rõ ràng giữa nhật ký giao dịch nối tiếp theo thời gian và kho biểu diễn trạng thái hiện hành [10], [14]. Nhật ký lưu trữ toàn bộ lịch sử các giao dịch đã được phê duyệt, đóng vai trò

là nguồn sự thật không thể chối bỏ dùng cho việc kiểm toán, minh bạch hóa và tái dựng lại hệ thống khi cần thiết [1], [10]. Trong khi đó, phần biểu diễn trạng thái hiện tại (chẳng hạn như cơ sở dữ liệu lưu thông tin tài khoản hoặc hồ sơ phát hành) được tối ưu hóa để phục vụ các truy vấn nghiệp vụ thường xuyên của ứng dụng [10], [14]. Mô hình phân tách này giúp hệ thống vừa bảo toàn được lịch sử giao dịch bất biến, vừa đảm bảo hiệu năng xử lý dữ liệu thực tế, tuy nhiên cách tổ chức chi tiết, cơ chế xử lý giao dịch không hợp lệ và phương pháp khôi phục trạng thái khi xảy ra sự cố vẫn có sự khác biệt tùy thuộc vào thiết kế của từng nền tảng [10], [14].

1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống

Hợp đồng thông minh là chương trình được triển khai trên sổ cái và thực thi các quy tắc chuyển trạng thái theo cơ chế thống nhất của nền tảng [1], [10]. Hợp đồng giúp chuẩn hóa điều kiện nghiệp vụ, giảm can thiệp thủ công và tạo dấu vết giao dịch nhất quán. Tuy nhiên, nó không tự bảo đảm mã nguồn đúng hoặc không có lỗ hổng. Lỗi trong phân quyền, kiểm tra dữ liệu, xử lý đồng thời và cơ chế nâng cấp có thể ảnh hưởng đến toàn hệ thống [10], [20]. Vì vậy, mã hợp đồng cần được kiểm thử, rà soát an toàn, quản lý phiên bản và phê duyệt trước khi triển khai.

Môi trường thực thi Blockchain yêu cầu tính xác định, nên hợp đồng không thể tùy ý lấy dữ liệu trực tiếp từ Internet; các nút có thể nhận phản hồi khác nhau và dẫn đến trạng thái không nhất quán [10]. Dữ liệu ngoài chuỗi thường được đưa vào thông qua nguồn có thẩm quyền hoặc dịch vụ trung gian chuyên trách như oracle [1]. Chữ ký của nguồn chỉ chứng minh dữ liệu đến từ đúng khóa và không bị sửa trong quá trình truyền, chứ không bảo đảm nội dung phản ánh đúng thực tế. Do đó, kiến trúc phải xác định nguồn tin cậy, thời hạn hiệu lực, cách xử lý sai lệch và cơ chế dừng an toàn [1], [10].

1.4. Cơ chế đồng thuận

1.4.1. Bài toán đồng thuận và mô hình lỗi

Bài toán đồng thuận trong hệ thống phân tán đặt ra yêu cầu làm thế nào để các nút mạng độc lập cùng thống nhất được một thứ tự giao dịch duy nhất và đi đến quyết định chung về trạng thái của hệ thống, ngay cả trong điều kiện mạng truyền tin có thể chậm trễ, mất mát hoặc xuất hiện các nút gặp sự cố [1], [12]. Khi thiết kế hoặc lựa chọn một giao thức đồng thuận, hai thuộc tính lý thuyết cốt lõi luôn phải được xem xét kỹ lưỡng là tính an toàn và khả năng tiến triển của hệ thống [12], [13]. Trong đó, tính an toàn bảo đảm rằng các nút trung thực tuyệt đối không bao giờ chấp nhận hai quyết định mâu thuẫn nhau trong cùng một ngữ cảnh (không có trạng thái phân nhánh trái phép), còn khả năng tiến triển của hệ thống bảo đảm rằng hệ thống

luôn tiếp tục phát triển và đưa ra quyết định mới khi các giả định về mạng và số lượng nút lỗi vẫn còn được thỏa mãn [12], [13]. Theo định lý cơ bản về hệ thống phân tán, không có một giao thức đồng thuận nào có thể đồng thời bảo đảm tuyệt đối cả hai thuộc tính này trong mọi điều kiện mạng bất đồng bộ; do đó, mỗi thiết kế bắt buộc phải xác định rõ mô hình mạng, loại lỗi giả định và ngưỡng chịu lỗi trước các hành vi tấn công [12], [13].

Các mô hình lỗi trong hệ thống phân tán thường được chia thành hai nhóm chính để đánh giá mức độ rủi ro [12], [13]. Mô hình chịu lỗi dừng giả định rằng các nút mạng lỗi chỉ có thể bị treo máy, sập nguồn hoặc mất kết nối mạng, chứ không bao giờ chủ động tạo ra thông điệp sai lệch để lừa các nút khác [13]. Ngược lại, mô hình chịu lỗi Byzantine mang tính tổng quát và khắt khe hơn rất nhiều, cho phép các nút bị kẻ tấn công chiếm quyền điều khiển có thể gửi các thông điệp mâu thuẫn tới các bên khác nhau, thông đồng với nhau hoặc cố ý vi phạm giao thức để phá hoại hệ thống [12], [13]. Việc lựa chọn giữa cơ chế chịu lỗi dừng hay chịu lỗi Byzantine phụ thuộc hoàn toàn vào ranh giới tin cậy của mô hình triển khai: đối với các hệ thống nội bộ nằm trong cùng một miền quản trị và được bảo vệ nghiêm ngặt bằng hạ tầng tường lửa, giả định lỗi dừng có thể được chấp nhận; tuy nhiên, đối với các mạng liên tổ chức, nơi các bên tham gia có lợi ích độc lập và tồn tại rủi ro về hành vi tùy ý hoặc thao túng, hệ thống bắt buộc phải áp dụng các thuật toán chịu lỗi Byzantine hoặc thiết lập thêm các lớp kiểm soát nghiệp vụ bổ sung [1], [12], [13].

1.4.2. Proof of Work và Proof of Stake

Trong các mạng lưới chuỗi khối không cấp quyền, bất kỳ ai cũng có thể tham gia vào hệ thống với chi phí thấp, dẫn đến rủi ro lớn về tấn công giả mạo danh tính, trong đó một thực thể duy nhất có thể tự tạo ra vô số định danh ảo nhằm thao túng kết quả biểu quyết của mạng lưới [2]. Để giải quyết bài toán này mà không cần dựa vào một danh sách thành viên được phê duyệt từ trước, hai nhóm cơ chế đồng thuận cốt lõi đã được phát triển [2]:

Thứ nhất là cơ chế bằng chứng công việc (Proof of Work). Cơ chế này gắn liền quyền đề xuất khối mới với năng lực tính toán phần cứng của các nút tham gia, buộc bên muốn tạo khối phải giải quyết các bài toán đố mã hóa tiêu tốn một lượng lớn tài nguyên tính toán và điện năng [2].

Thứ hai là cơ chế bằng chứng cổ phần (Proof of Stake). Để khắc phục hạn chế tiêu tốn năng lượng của phương thức trước, cơ chế này gắn quyền tham gia và đề xuất khối với lượng tài sản hoặc cổ phần mà người tham gia cam kết khóa lại trong

hệ thống, kết hợp cùng các quy tắc khuyến khích kinh tế hoặc hình thức xử phạt tự động nếu phát hiện hành vi vi phạm giao thức [2].

Mặc dù đóng vai trò nền tảng trong các mạng lưới tiền mã hóa công khai, cả hai cơ chế đồng thuận trên thường không phù hợp hoặc không phải là lựa chọn tối ưu cho các bài toán quản lý dữ liệu chuyên biệt như hệ thống cấp phát và xác minh văn bằng, chứng chỉ [2]. Trong bối cảnh quản lý văn bằng, quyền tham gia hệ thống và xác thực thông tin đã được kiểm soát trực tiếp thông qua tư cách pháp nhân và chính sách quản trị giữa các cơ quan, tổ chức giáo dục, do đó không cần thiết phải sử dụng tài nguyên tính toán lớn hay các hệ thống kinh tế tài sản phức tạp để chống lại tấn công giả mạo danh tính [1], [14].

1.4.3. Đồng thuận CFT và BFT trong mạng định danh

Trong mạng cấp quyền, danh sách thành viên được xác định trước nên giao thức đồng thuận có thể dựa trên biểu quyết giữa các nút đã biết danh tính, thay vì dựa vào chi phí tính toán hay tài sản ký quỹ như bằng chứng công việc và bằng chứng cổ phần trong mạng mở [12], [13]. Hai họ giao thức tiêu biểu là đồng thuận chịu lỗi dừng (CFT) và đồng thuận chịu lỗi Byzantine (BFT), tương ứng với hai mô hình lỗi đã trình bày ở mục 1.4.1.

Raft là đại diện tiêu biểu của họ CFT. Thuật toán bầu ra một nút lãnh đạo (leader) cho mỗi nhiệm kỳ (term); nút này nhận yêu cầu, sắp xếp thứ tự và sao chép nhật ký giao dịch sang các nút còn lại. Về điều kiện cam kết (log commitment), Raft quy định nút lãnh đạo chỉ có thể trực tiếp xác nhận một mục nhật ký là đã cam kết khi mục nhật ký đó thuộc về chính nhiệm kỳ hiện tại của nó và đã được sao chép thành công tới đa số nút trong cụm (ngưỡng đa số tối thiểu). Đối với các mục nhật ký thuộc về các nhiệm kỳ trước đó, Raft không cho phép lãnh đạo suy ra đã cam kết chỉ từ việc đếm số lượng bản sao trên đa số nút (do rủi ro bị ghi đè khi đổi lãnh đạo theo kịch bản mô tả trong [13]), mà các mục này chỉ được xem là cam kết gián tiếp khi có một mục thuộc nhiệm kỳ hiện tại được cam kết thành công. Khi nút lãnh đạo gặp sự cố, các nút còn lại tổ chức bầu lại, và nút mới chỉ được chọn nếu có nhật ký đủ cập nhật [13]. Tuy nhiên, Raft giả định mọi nút đều tuân thủ giao thức, nên trường hợp một nút bị chiếm quyền và cố ý gửi thông điệp sai lệch nằm ngoài mô hình lỗi mà thuật toán xử lý [13].

PBFT cổ điển thuộc họ BFT. Trong mỗi vòng xử lý, nút chính đề xuất thứ tự cho yêu cầu, sau đó các nút còn lại trao đổi thông điệp với nhau qua ba pha là pre-prepare, prepare và commit; yêu cầu chỉ được thực thi khi đã thu đủ xác nhận từ các nút khác nhau. Nếu nút chính không đạt tiến triển trong thời hạn quy định, các nút

chuyển sang nút chính mới thông qua thủ tục thay đổi nút chính (view change). Nhờ cơ chế này, giao thức vẫn giữ được tính an toàn ngay cả khi một số nút lỗi gửi thông điệp mâu thuẫn; còn khả năng tiến triển chỉ được bảo đảm khi mạng thỏa mãn giả định đồng bộ một phần [12].

$$n \geq 2f + 1 \text{ (CFT)}; \quad n \geq 3f + 1 \text{ (BFT)} \tag{1.3}$$

Trong đó n là tổng số nút và f là số nút lỗi tối đa. Với $n = 3f + 1$, cơ chế BFT thường yêu cầu tối thiểu $2f + 1$ phiếu xác nhận. Hai tập hợp xác nhận như vậy giao nhau tại ít nhất $f + 1$ nút, nên phần giao vẫn chứa ít nhất một nút trung thực khi có tối đa f nút Byzantine [12]. PBFT cổ điển còn có chi phí truyền thông bậc hai $O(n^2)$ ở các pha nhiều nút trao đổi chéo, do đó chi phí tăng nhanh theo quy mô mạng [12].

1.4.4. Tính hoàn tất và đánh đổi thiết kế

Tính hoàn tất (finality) biểu thị mức độ chắc chắn rằng một giao dịch đã được chấp nhận sẽ không bị đảo ngược trong điều kiện an toàn giả định. Trong cơ chế hoàn tất theo xác suất, khả năng tổ chức lại lịch sử giảm dần khi có thêm khối xác nhận nhưng không triệt tiêu hoàn toàn [2]. Trong cơ chế hoàn tất xác định dựa trên ngưỡng phiếu xác nhận, giao dịch được coi là hoàn tất sau khi đạt đủ phiếu theo giao thức; kết luận này vẫn phụ thuộc vào giả định số nút lỗi, an toàn khóa và quy tắc quản trị [12], [13]. Sự khác biệt kỹ thuật giữa các nhóm cơ chế đồng thuận được tổng hợp và đối chiếu cụ thể tại Bảng 1.1.

Bảng 1.1. So sánh đặc tính kỹ thuật giữa các nhóm cơ chế đồng thuận tiêu biểu

Tiêu chí phân tích	Proof of Work (PoW)	Proof of Stake (PoS)	Đồng thuận định danh (CFT / BFT)
Cơ chế chống Sybil & Căn cứ quyền lực	Năng lực tính toán hàm băm (Hashrate / Computation Power).	Tỷ lệ tài sản tiền mã hóa đặt cọc ký quỹ (Staked Capital / Bond).	Danh tính xác thực & Số phiếu xác nhận tối thiểu (Known Identity & Quorum Voting).
Giao thức / Thuật toán đại diện	Nakamoto Consensus (Bitcoin, SHA-256), Scrypt, Ethash cũ.	Casper FFG, LMD-GHOST (Ethereum Proof of Stake), Ouroboros.	Raft (CFT), PBFT, IBFT 2.0, QBFT (Hyperledger Fabric, Quorum).

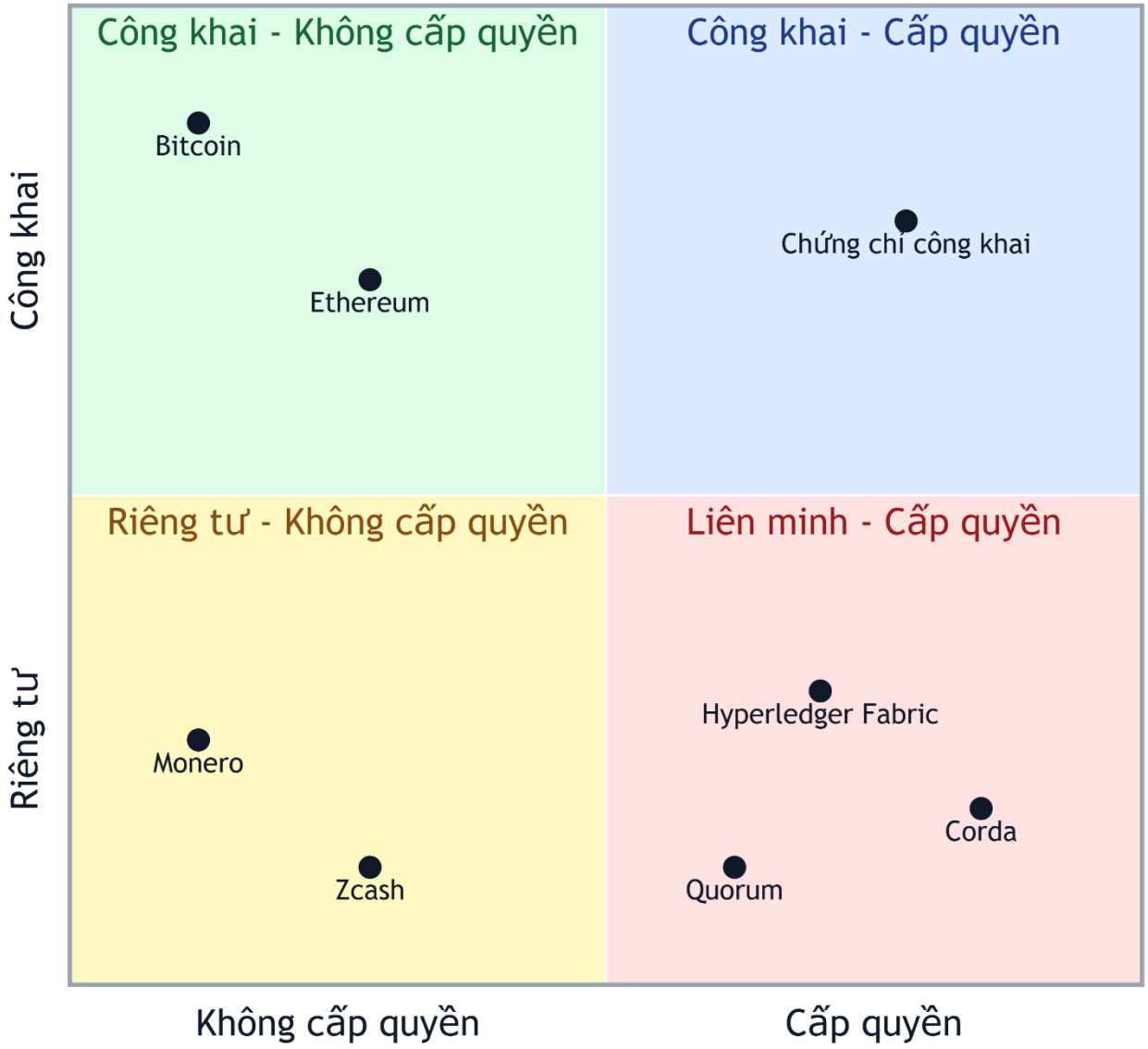
Tiêu chí phân tích	Proof of Work (PoW)	Proof of Stake (PoS)	Đồng thuận định danh (CFT / BFT)
Hiệu năng (TPS) & Mức tiêu thụ năng lượng	Thông lượng lớp cơ sở thấp (Bitcoin ~7 TPS, Ethereum 1.0 trước đây ~15 TPS [2]), độ trễ xác nhận từ vài phút đến hàng giờ; tiêu hao năng lượng tính toán băm rất lớn.	Thông lượng lớp cơ sở ở mức hàng chục đến hàng trăm TPS tùy thông số khối [2]; tiết kiệm trên 99,9% năng lượng tiêu thụ so với PoW (như Ethereum sau The Merge [2]).	Thông lượng cao trong mạng quy mô nhỏ (đạt hàng trăm đến hàng nghìn TPS tùy cấu hình cụm và mạng [12], [13]), độ trễ mili-giây đến vài giây; tiêu hao năng lượng thấp.
Tính hoàn tất của giao dịch (Finality)	Hoàn tất theo xác suất (Probabilistic): Khối có nguy cơ bị rẽ nhánh, độ an toàn tăng theo số khối xác nhận.	Kết hợp xác suất và xác định (Hybrid): Xác suất ở mức khối đơn lẻ, chốt xác định theo từng kỷ nguyên (Epoch).	Hoàn tất xác định tức thì (Deterministic): Không rẽ nhánh, chung quyết ngay khi đạt đủ số phiếu xác nhận tối thiểu ($n \geq 3f + 1$).
Đánh đổi kiến trúc & Thách thức an toàn	Lãng phí năng lượng; rủi ro tấn công 51% sức mạnh băm; khó mở rộng quy mô giao dịch.	Thiết kế kinh tế học mật mã phức tạp; rủi ro phân nhánh không phạt (Nothing-at-Stake); nguy cơ tích tụ tài sản.	Khó mở rộng khi mạng có quá nhiều nút (chi phí truyền thông $O(n^2)$); yêu cầu hạ tầng PKI và ranh giới tin cậy ban đầu.

Lựa chọn đồng thuận là bài toán đánh đổi giữa mức phân tán quyền quyết định, khả năng chịu lỗi, độ trễ, thông lượng và chi phí vận hành [10]. Bộ ba phân quyền – an toàn – khả năng mở rộng có thể dùng như một khung trực giác, không phải định luật buộc mọi hệ thống chỉ được chọn hai thuộc tính. Cơ chế phù hợp phải được đánh giá trên tải thực tế, số tổ chức vận hành, loại hành vi đối phương và yêu cầu phục hồi, thay vì dựa trên tên gọi hoặc tuyên bố hiệu năng của nền tảng.

1.5. Phân loại Blockchain và điều kiện lựa chọn

1.5.1. Các trục phân loại

Để lựa chọn mô hình sổ cái phù hợp với từng bài toán nghiệp vụ cụ thể, các hệ thống chuỗi khối trước hết cần được phân định rõ theo quyền tham gia mạng lưới và phạm vi quản trị như minh họa trên Hình 1.4.



Hình 1.4. Phân loại Blockchain theo quyền tham gia và phạm vi quản trị

Như thể hiện trên Hình 1.4, công nghệ chuỗi khối được phân loại dựa trên hai trục độc lập gồm quyền tham gia và phạm vi quản trị. Trục quyền tham gia phân biệt mạng không cấp quyền (nơi các nút có thể tự do tham gia theo quy tắc mở) với mạng cấp quyền (nơi danh tính và vai trò của từng thành viên được phê duyệt trước) [1]. Trong khi đó, trục phạm vi quản trị phân biệt mạng công khai, mạng riêng do một tổ chức chi phối và mạng liên minh do nhiều tổ chức cùng điều hành. Quyền đọc dữ liệu lại là một quyết định thiết kế riêng biệt: một mạng cấp quyền vẫn có thể cho phép công chúng tra cứu bằng chứng tối thiểu, và ngược lại một mạng công khai vẫn

có thể áp dụng các cơ chế bảo vệ nội dung riêng tư. Do đó, không nên đồng nhất khái niệm "công khai" với "không cấp quyền", hoặc "riêng tư" với "cấp quyền" [1], [14].

1.5.2. So sánh các mô hình triển khai

Trên cơ sở hai trục phân loại về quyền tham gia và phạm vi quản trị, đặc tính kỹ thuật cùng phạm vi ứng dụng của ba mô hình triển khai chuỗi khối tiêu biểu được tổng hợp và đối chiếu tại Bảng 1.2.

Bảng 1.2. So sánh khái quát các mô hình triển khai Blockchain

Tiêu chí	Công khai, không cấp quyền	Riêng, cấp quyền	Liên minh, cấp quyền
Quản trị	Cộng đồng và quy tắc giao thức	Một tổ chức hoặc một miền quản trị	Nhiều tổ chức theo chính sách chung
Thành viên xác thực	Mở theo điều kiện chống Sybil	Danh tính được phê duyệt nội bộ	Danh tính thuộc các tổ chức thành viên
Khả năng kiểm chứng	Rộng, thường công khai	Chủ yếu trong nội bộ	Giữa các thành viên và bên được cấp quyền
Quyền riêng tư	Khó kiểm soát phạm vi phổ biến dữ liệu	Dễ giới hạn hơn theo chính sách nội bộ	Có thể phân vùng theo tổ chức và nghiệp vụ
Hiệu năng	Phụ thuộc mạng mở và cơ chế kinh tế	Có thể tối ưu trong phạm vi nhỏ	Phụ thuộc số tổ chức, ngưỡng xác nhận và hạ tầng
Bài toán phù hợp	Hệ sinh thái mở cần kiểm chứng rộng	Nhật ký và quy trình nội bộ	Phối hợp và kiểm toán liên tổ chức

Phân tích từ Bảng 1.2 chỉ ra rằng không có một mô hình triển khai nào là tối ưu tuyệt đối trong mọi hoàn cảnh [1], [10]. Chuỗi công khai đem lại khả năng kiểm chứng độc lập trên phạm vi toàn cầu nhưng đi kèm chi phí giao dịch biến động, độ trễ cao và khó kiểm soát phạm vi phổ biến dữ liệu nhạy cảm. Mạng riêng tối ưu hóa hiệu năng nhưng làm mất đi ý nghĩa chống kiểm duyệt và làm suy giảm giá trị cốt lõi của sổ cái phân tán do ranh giới tin cậy vẫn phụ thuộc vào một tổ chức quản trị duy nhất. Mạng liên minh dung hòa tốt nhất giữa hiệu năng và kiểm soát chéo, nhưng đòi hỏi chi phí đàm phán chính sách quản trị và hạ tầng vận hành phức tạp giữa các bên tham gia [1], [10], [14].

Hiệu năng thực tế của mỗi mô hình còn phụ thuộc vào thuật toán đồng thuận cụ thể, số lượng nút tham gia, chính sách xác nhận giao dịch, cơ sở hạ tầng mạng và kích thước tải trọng dữ liệu. Do đó, các chỉ số thông lượng (TPS) hoặc độ trễ chỉ có giá trị so sánh khi được đo đạc trên cùng một cấu hình và điều kiện tải xác định [10].

1.5.3. Điều kiện áp dụng và giới hạn của Blockchain

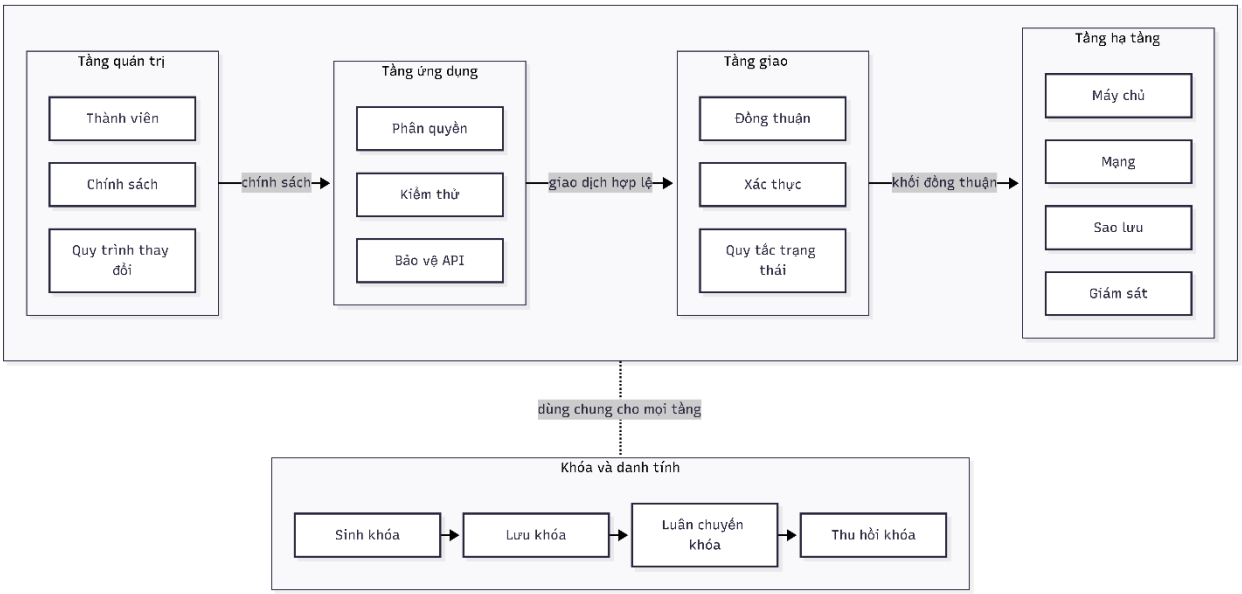
Blockchain phù hợp với các bài toán trong đó nhiều tổ chức độc lập cần cùng ghi nhận, chia sẻ và kiểm chứng dữ liệu nhưng không muốn hoàn toàn phụ thuộc vào một đơn vị quản lý tập trung. Công nghệ này cũng có giá trị khi hệ thống cần bảo đảm khả năng truy vết, đối chiếu và kiểm chứng tính toàn vẹn của lịch sử ghi nhận. Tuy nhiên, việc ứng dụng Blockchain chỉ thực sự hợp lý khi quy trình nghiệp vụ có thể được chuẩn hóa thành các giao dịch, các bên thống nhất được quy tắc xác thực và lợi ích từ việc duy trì sổ cái phân tán tương xứng với chi phí vận hành hệ thống [1], [10].

Ngược lại, Blockchain có thể không phải lựa chọn tối ưu nếu toàn bộ quy trình thuộc quyền quản lý của một tổ chức và cơ sở dữ liệu tập trung đã đáp ứng đầy đủ yêu cầu về độ tin cậy, kiểm soát và kiểm toán. Công nghệ này cũng cần được cân nhắc đối với các hệ thống đòi hỏi sửa đổi hoặc xóa dữ liệu thường xuyên, thời gian phản hồi thấp hoặc thông lượng giao dịch vượt quá khả năng của cơ chế đồng thuận được lựa chọn. Đối với dữ liệu cá nhân và thông tin nhạy cảm, cần tránh ghi trực tiếp dữ liệu gốc lên chuỗi khi không cần thiết; thay vào đó, có thể lưu trữ dữ liệu ngoài chuỗi và chỉ ghi nhận giá trị băm hoặc bằng chứng cần thiết trên chuỗi. Trong những trường hợp này, cơ sở dữ liệu quan hệ kết hợp với chữ ký số và nhật ký kiểm toán có thể là giải pháp đơn giản, dễ quản lý và tiết kiệm chi phí hơn [1], [10].

1.6. An toàn, quyền riêng tư và khả năng mở rộng

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp

Để bảo vệ tính toàn vẹn của sổ cái và tài sản mật mã trước các nguy cơ tấn công từ cả bên ngoài lẫn nội bộ, hệ thống chuỗi khối cần được triển khai theo mô hình phòng thủ nhiều lớp như thể hiện trên Hình 1.5.



Hình 1.5. Các lớp bảo vệ trong một hệ thống Blockchain

Hình 1.5 khái quát kiến trúc bảo vệ đa lớp cần áp dụng trong một hệ thống Blockchain. Bảo đảm an toàn cho hệ thống không thể chỉ đặt niềm tin vào độ an toàn của hàm băm hay giao thức đồng thuận, mà bắt buộc phải triển khai theo nguyên tắc phòng thủ theo chiều sâu (defense-in-depth) qua đầy đủ các lớp: mật mã, mạng ngang hàng, đồng thuận, hợp đồng thông minh, tầng ứng dụng, lưu trữ ngoài chuỗi và chính sách quản trị vận hành [1], [10]. Tài sản quan trọng nhất cần bảo vệ là khóa bí mật của đơn vị phát hành, trạng thái toàn vẹn của sổ cái và nhật ký kiểm toán. Tác nhân đe dọa có thể mở rộng từ kẻ tấn công bên ngoài khai thác lỗ hổng hạ tầng, người dùng nội bộ bị chiếm quyền tài khoản, cho đến nguy cơ quản trị viên lạm quyền hoặc cấu kết nhóm nút đồng thuận.

Mọi biện pháp bảo vệ kỹ thuật phải được xây dựng bám chặt vào mô hình đe dọa đã xác định. Kênh truyền thông phải được mã hóa và xác thực hai chiều (TLS/mTLS); các nút mạng cần được vá lỗi định kỳ, phân đoạn mạng và giám sát toàn diện; chính sách đồng thuận phải giới hạn quyền kiểm soát của bất kỳ một tổ chức đơn lẻ nào; tầng ứng dụng phải kiểm soát chặt chẽ dữ liệu đầu vào, quản lý phiên đăng nhập và thực thi phân quyền theo vai trò. Cần lưu ý rằng việc sao chép sổ cái trên nhiều nút không thay thế kế hoạch ứng phó sự cố: nếu toàn bộ các nút cùng phụ thuộc một hạ tầng đám mây, cùng áp dụng cấu hình sai hoặc cùng bị lộ khóa trong sự cố bảo mật, lỗi có thể lan rộng đồng loạt trên toàn bộ mạng lưới [1], [10].

1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài

Khóa bí mật là yếu tố định danh duy nhất chứng minh quyền ký phát hành giao dịch trên sổ cái; do đó quản trị khóa phải được thực thi nghiêm ngặt trong suốt toàn bộ vòng đời: khởi tạo bằng bộ sinh số ngẫu nhiên an toàn mật mã (CSPRNG), lưu trữ cô lập trong mô-đun phần cứng bảo mật chuyên dụng (HSM) hoặc dịch vụ quản lý khóa KMS, thiết lập hạn mức ký và quy trình luân chuyển khóa định kỳ [16]. Các giải pháp mật mã tiên tiến như chữ ký đa bên (MPC) hoặc mật mã ngưỡng có thể triệt tiêu điểm lỗi đơn lẻ gắn với một cá nhân người ký, nhưng không thể thay thế cho các chính sách kiểm toán và giám sát vận hành hệ thống [16].

Hợp đồng thông minh cần kiểm tra quyền, dữ liệu đầu vào, chuyển trạng thái và các trường hợp đồng thời; phiên bản mới phải có quy trình rà soát, kiểm thử và phê duyệt [10]. Nguồn dữ liệu ngoài phải được xác định thẩm quyền, ký số khi phù hợp và có thời hạn hiệu lực. Dùng nhiều nguồn chỉ hữu ích khi các nguồn đủ độc lập; nếu tất cả cùng lấy dữ liệu từ một hệ thống gốc, biểu quyết không làm dữ liệu đáng tin hơn.

1.6.3. Quyền riêng tư và dữ liệu cá nhân

Bản chất sao chép trạng thái phân tán và tính chống sửa đổi của chuỗi khối tạo ra sự xung đột kỹ thuật trực tiếp với các nguyên tắc pháp lý về bảo vệ quyền riêng tư, đặc biệt là nguyên tắc tối thiểu hóa dữ liệu (Data Minimization) và quyền yêu cầu xóa dữ liệu (Right to be Forgotten) [1], [17]. Việc ghi nhận trực tiếp các trường dữ liệu định danh nhạy cảm như họ tên, ngày sinh, số căn cước công dân, kết quả học tập hoặc lưu trữ toàn bộ tệp văn bằng lên chuỗi khối sẽ làm nhân bản dữ liệu cá nhân tới mọi nút mạng, khiến cho việc chỉnh sửa, giới hạn phạm vi xử lý hoặc tiêu hủy thông tin theo yêu cầu của chủ thể dữ liệu trở nên bất khả thi về mặt kỹ thuật. Ngay cả giải pháp mã hóa dữ liệu cũng không bảo đảm an toàn bền vững, bởi các bản mã tồn tại vĩnh viễn trên sổ cái sẽ đối mặt với nguy cơ bị giải mã hồi tố trong tương lai khi năng lực tính toán phát triển hoặc khóa bí mật bị lộ [1], [17].

Tại Việt Nam, khung pháp lý về bảo vệ quyền riêng tư đã được cụ thể hóa thông qua Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 (có hiệu lực từ ngày 01/01/2026) và Nghị định 356/2025/NĐ-CP quy định chi tiết thi hành Luật [7], [8]. Nhằm bảo đảm tính tuân thủ pháp lý nghiêm ngặt, đồ án phân định rõ ràng giữa hai loại giao dịch trên chuỗi: đối với giao dịch phát hành thông thường, hệ thống chỉ neo duy nhất cam kết mật mã tối thiểu là giá trị băm Merkle Root 32-byte của toàn lô; đối với giao dịch thu hồi trong nguyên mẫu, tải trọng ghi nhận dấu vết REVOKE:<certUid> qua OP_RETURN. Toàn bộ hồ sơ người học, thông tin tài

khoản và dữ liệu văn bằng đầy đủ được bảo vệ ngoài chuỗi trong cơ sở dữ liệu PostgreSQL. Cần lưu ý rằng giá trị băm hay mã certUid (UUID) tuy không chứa trực tiếp thông tin định danh cá nhân (họ tên, căn cước công dân), nhưng bản chất là định danh giả danh (pseudonym) và vẫn tiềm ẩn nguy cơ liên kết ngược (linkability) tới một chứng thư cụ thể nếu có dữ liệu đối chiếu ngoài chuỗi. Do đó, hệ thống không xem dữ liệu trên chuỗi là ẩn danh tuyệt đối mà luôn kiểm soát chặt chẽ phạm vi công bố nhật ký và bằng chứng kiểm thử.

1.6.4. Khả năng mở rộng và khả năng tương tác

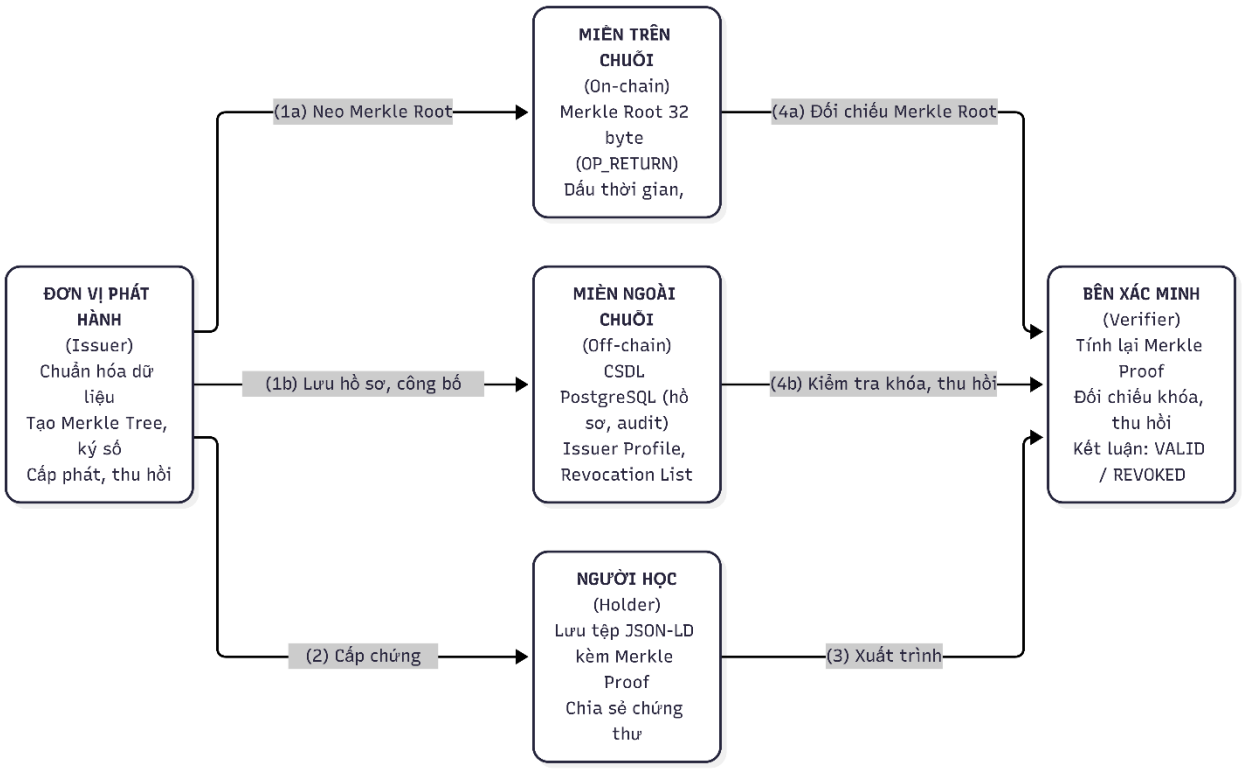
Khi nhiều nút cùng lưu và kiểm tra giao dịch, thông lượng, độ trễ, băng thông và dung lượng trở thành các giới hạn cần đo [10]. Có thể cải thiện bằng cách xử lý theo lô, tối ưu chính sách xác nhận, phân vùng dữ liệu hoặc chuyển một phần xử lý ra ngoài chuỗi; mỗi cách tạo thêm giả định tin cậy và độ phức tạp. Các giải pháp lớp 2 chỉ nên được xem xét khi thật sự kế thừa an toàn từ chuỗi cơ sở, không nên đồng nhất mọi sidechain hoặc cầu nối với lớp 2.

Khả năng tương tác không chỉ là truyền thông điệp giữa hai mạng mà còn là thống nhất ý nghĩa dữ liệu, định danh tổ chức, phiên bản lược đồ và trạng thái nghiệp vụ. Cầu nối hoặc dịch vụ chuyển tiếp bổ sung điểm tin cậy mới và từng là mục tiêu của nhiều sự cố. Các chuẩn dữ liệu chứng thực như W3C Verifiable Credentials giúp thống nhất cách biểu diễn vai trò phát hành, chủ thể và bên xác minh, nhưng không tự giải quyết quản trị khóa, quyền truy cập hay tranh chấp pháp lý [17].

1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ

1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng

Công tác quản lý và phát hành văn bằng, chứng chỉ truyền thống đang đối mặt với nhiều thách thức lớn: nguy cơ làm giả văn bằng tinh vi, quy trình xác minh thủ công tốn kém thời gian và sự phụ thuộc hoàn toàn vào cổng tra cứu riêng của từng cơ sở đào tạo. Mô hình W3C Verifiable Credentials xác định ba vai trò cốt lõi trong chu trình cấp phát và kiểm chứng văn bằng số gồm: cơ sở đào tạo có thẩm quyền phát hành, người học nắm giữ hoặc chia sẻ chứng thư và tổ chức bên ngoài thực hiện xác minh [17]. Quy trình truyền thống thường dựa vào bản giấy, bản sao chứng thực hoặc yêu cầu xác nhận thủ công; khi dữ liệu nằm ở nhiều hệ thống, việc đối soát có thể chậm và khó phát hiện tài liệu đã bị chỉnh sửa. Một sổ bằng chứng dùng chung có thể hỗ trợ bên xác minh kiểm tra nguồn phát hành, tính toàn vẹn và trạng thái hiện thời mà giảm số bước liên hệ thủ công. Mô hình khái niệm của bài toán được trình bày trên Hình 1.6.



Hình 1.6. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng

Yêu cầu kiểm chứng không chỉ đơn thuần là so sánh mã băm. Một quy trình xác minh đầy đủ phải bao gồm: xác minh tính toàn vẹn của dữ liệu văn bằng (không bị chỉnh sửa dù chỉ một ký tự), xác minh chữ ký số của đơn vị phát hành hợp lệ tại thời điểm ký, kiểm tra tình trạng hiệu lực trên sổ cái (văn bằng còn hiệu lực hay đã thu hồi) và xác nhận tư cách pháp lý của tổ chức cấp bằng [1], [17]. Cần lưu ý rằng kết quả xác minh không tìm thấy bằng chứng trên chuỗi không đồng nghĩa với việc văn bằng đó là giả mạo, bởi dữ liệu các khóa học cũ có thể chưa được số hóa hoặc nằm ngoài phạm vi triển khai của hệ thống.

1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi

Ở mức khái niệm, sổ cái chỉ nên giữ lượng dữ liệu tối thiểu phục vụ kiểm chứng, chẳng hạn cam kết mật mã, tham chiếu đơn vị phát hành và trạng thái hiệu lực. Hồ sơ đầy đủ cùng dữ liệu cá nhân được quản lý ngoài chuỗi trong hệ thống có thẩm quyền, với mã hóa, phân quyền, nhật ký truy cập và sao lưu. Hai miền được liên kết bằng quy tắc tạo bằng chứng xác định để tài liệu bị thay đổi tạo kết quả kiểm tra khác [1], [17].

Cách phân tách này giảm việc sao chép dữ liệu nhạy cảm nhưng không làm rủi ro riêng tư biến mất. Bằng chứng trên chuỗi vẫn có thể bị liên kết hoặc dò đoán; kho ngoài chuỗi vẫn có thể bị mất, sửa hoặc ngừng phục vụ. Vì vậy, Chương 2 cần

xác định mô hình dữ liệu, quy tắc chuẩn hóa, quyền truy cập và cơ chế xử lý khi dữ liệu nguồn sai mà không giả định Blockchain tự giải quyết các vấn đề đó.

1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm

Giá trị kỳ vọng của hướng tiếp cận là rút ngắn thời gian xác minh, tăng khả năng phát hiện tài liệu bị sửa và tạo dấu vết trạng thái có thể kiểm chứng. Công nghệ có thể hỗ trợ ghi nhận phát hành hoặc thay đổi hiệu lực, nhưng không tự quyết định người học đủ điều kiện tốt nghiệp, không chứng minh người xuất trình là chủ văn bằng và không thay thế thẩm quyền cấp văn bằng của cơ sở giáo dục.

Cơ sở đào tạo vẫn chịu trách nhiệm về dữ liệu đầu vào, quyết định công nhận tốt nghiệp và thẩm quyền phát hành; người vận hành chịu trách nhiệm bảo vệ khóa và hạ tầng; bên xác minh phải diễn giải kết quả đúng phạm vi. Thông tư 10/2026/TT-BGDĐT đã bổ sung khái niệm và cơ chế quản lý văn bằng, chứng chỉ số, đồng thời xác định vai trò của cơ sở dữ liệu văn bằng, chứng chỉ trong hoạt động cấp và xác minh [6]. Vì vậy, chứng thư Blockcerts trong đồ án được định vị là tệp kết quả kỹ thuật của nguyên mẫu hỗ trợ phát hành và kiểm chứng, không mặc nhiên thay thế văn bằng, chứng chỉ số có giá trị pháp lý theo quy định hiện hành.

1.7.4. Mô hình Blockcerts V3 và quan hệ với Verifiable Credentials

Blockcerts V3 tổ chức chứng thư dưới dạng JSON-LD, kế thừa kiến trúc chứng thư số trên blockchain của MIT Media Lab [3], gồm định danh chứng thư, thông tin chủ thể, issuer và bằng chứng mật mã; cú pháp JSON-LD được quy định trong [18], còn cấu trúc Blockcerts được đối chiếu với schema/context của dự án [4]. Báo cáo dùng W3C Verifiable Credentials Data Model v2.0 để mô tả ba vai trò issuer–holder–verifier [17]. Tuy nhiên, toolchain thực nghiệm phát hành credential với context https://www.w3.org/2018/credentials/v1; do đó tệp kết quả được đối chiếu thêm với VC Data Model v1.1 [19]. Phân biệt này tránh diễn giải rằng nguyên mẫu đã chuyển hoàn toàn sang context VC v2.

Trong quá trình phát hành, nội dung cần bảo vệ được chuẩn hóa theo quy tắc của suite, tính target hash và đưa vào cây Merkle. Merkle root đại diện cho toàn bộ lô; mỗi chứng thư có một đường dẫn riêng cùng thông tin anchor. Nhiều chứng thư vì thế có thể dùng chung một transaction nhưng vẫn có bằng chứng để phát hiện thay đổi nội dung [4], [5]. Chữ ký giao dịch Bitcoin chứng minh quyền sử dụng khóa phát hành, còn việc gắn khóa đó với cơ sở đào tạo có thẩm quyền phụ thuộc Issuer Profile và quy trình quản trị danh tính.

1.7.5. Bitcoin OP_RETURN và môi trường regtest

Mã lệnh OP_RETURN của Bitcoin cho phép tạo một đầu ra giao dịch không thể chi tiêu (provably unspendable output), đồng thời mang theo tải trọng dữ liệu tùy biến tối đa 80 bytes. Do giá trị băm Merkle Root SHA-256 chỉ có kích thước 32 bytes, cơ chế này hoàn toàn đáp ứng nhu cầu neo giữ cam kết mật mã của một lô chứng thư lên mạng Bitcoin với chi phí giao dịch tối thiểu. Nguyên mẫu chỉ neo Merkle Root của lô phát hành và dấu vết thu hồi tối thiểu thông qua OP_RETURN, tuyệt đối không ghi bất kỳ thông tin định danh cá nhân nào lên sổ cái công khai, phù hợp với nguyên tắc tối thiểu hóa dữ liệu được ghi trên chuỗi [1], [7], [8].

Môi trường Bitcoin regtest (Regression Testing) được lựa chọn vì cho phép nút quản trị chủ động tạo khối tức thì theo yêu cầu mà không tiêu tốn tài nguyên tính toán, không phát sinh chi phí tiền tệ thực tế và cho phép thiết lập kịch bản kiểm thử xác định để tái lập ổn định [9]. Cần nhấn mạnh rằng môi trường regtest chỉ phục vụ mục tiêu chứng minh tính đúng đắn của logic phát hành, kiểm chứng và phục hồi; kết quả thực nghiệm không phản ánh các yếu tố phí giao dịch biến động, độ trễ lan truyền hàng đợi mempool hay nguy cơ phân nhánh chuỗi trên mạng Bitcoin mainnet.

1.7.6. Bối cảnh pháp lý và ranh giới áp dụng tại Việt Nam

Thông tư 10/2026/TT-BGDĐT của Bộ Giáo dục và Đào tạo (có hiệu lực từ ngày 15/04/2026) chính thức bổ sung các quy định về văn bằng, chứng chỉ số, xây dựng cơ sở dữ liệu văn bằng quốc gia và quy trình khai thác, xác minh thông tin văn bằng số [6]. Khuôn khổ pháp lý này đặt nền tảng quan trọng cho việc triển khai hệ thống văn bằng số tại Việt Nam. Tuy nhiên, một chứng thư kỹ thuật do nguyên mẫu thử nghiệm phát hành không tự động trở thành văn bằng số hợp pháp; giá trị pháp lý của văn bằng vẫn phụ thuộc vào chủ thể có thẩm quyền, quy trình công nhận tốt nghiệp, dữ liệu được cập nhật vào hệ thống cơ sở dữ liệu theo quy định và các điều kiện pháp lý liên quan của Bộ Giáo dục và Đào tạo.

Do hệ thống xử lý họ tên, ngày sinh, email, mã sinh viên và thông tin văn bằng, việc triển khai thực tế còn phải tuân thủ Luật Bảo vệ dữ liệu cá nhân số 91/2025/QH15 và các quy định hướng dẫn thi hành [7], [8]. Trong phạm vi đồ án, dữ liệu benchmark là dữ liệu giả lập; bí mật, credential runtime và dữ liệu cá nhân không cần thiết không được công bố trong repository hoặc bộ bằng chứng.

1.7.7. Đối chiếu các phương án và phạm vi đóng góp

Bảng 1.3 đối chiếu ba cách tiếp cận tiêu biểu cho bài toán quản lý và kiểm chứng văn bằng. Phương án cơ sở dữ liệu tập trung kết hợp chữ ký số hoàn toàn đáp ứng nhu cầu quản lý nội bộ và kiểm tra tính toàn vẹn của tệp dữ liệu. Chuẩn

Blockcerts bổ sung cơ chế đóng gói bằng chứng Merkle cùng khả năng neo cam kết lịch sử lên mạng chuỗi khối để bên thứ ba có thể kiểm chứng độc lập bên ngoài cơ sở dữ liệu của đơn vị cấp. Tuy nhiên, giá trị thực tế của lớp bằng chứng này vẫn phụ thuộc chặt chẽ vào ranh giới tin cậy của môi trường neo, khả năng truy cập hồ sơ định danh đơn vị phát hành (Issuer Profile) và chính sách quản lý danh sách thu hồi [1], [4], [17].

Bảng 1.3. Đối chiếu các phương án quản lý và kiểm chứng văn bằng

Tiêu chí	CSDL và chữ ký số	Chứng thư theo chuẩn VC	Blockcerts neo Bitcoin
Bằng chứng toàn vẹn	Chữ ký trên dữ liệu; kiểm tra khóa	Phụ thuộc cơ chế bảo vệ của credential	Target hash, Merkle proof và anchor
Lịch sử cam kết	Phụ thuộc kho log và miền quản trị	Chuẩn VC không tự quy định sổ cái	Có cam kết trên chuỗi; regtest vẫn do lab kiểm soát
Thu hồi	CSDL hoặc danh sách trạng thái	Cơ chế trạng thái theo hệ triển khai	Nguyên mẫu dùng Revocation List và dấu vết riêng
Chi phí vận hành	Không phát sinh phí neo Blockchain	Phụ thuộc cơ chế bảo vệ được chọn	Thêm node/RPC, phí và confirmation nếu dùng mainnet
Phạm vi đồ án	Phương án đối chiếu về thiết kế	Khung dữ liệu để đối chiếu	Nguyên mẫu được hiện thực và đo trên regtest

Đồ án kế thừa Blockcerts và các chuẩn có sẵn; đóng góp tập trung vào tích hợp Maker–Checker, phân quyền đối tượng, outbox, checkpoint/phục hồi và chuỗi bằng chứng tái lập. Báo cáo chưa có thí nghiệm đối chứng giữa ba phương án trên, nên không khẳng định Blockchain nhanh hơn hoặc an toàn hơn trong mọi tình huống. Việc kiểm tra trên regtest chứng minh logic cam kết và xác minh, chưa chứng minh mức phân tán niềm tin của Bitcoin mainnet.

1.8. Kết luận chương 1

Chương 1 đã hệ thống hóa toàn diện cơ sở lý thuyết về công nghệ chuỗi khối, phân tích chính xác các giới hạn kỹ thuật về tính đúng đắn dữ liệu đầu vào, bảo mật khóa mật mã, quyền riêng tư và khả năng mở rộng. Đối với bài toán văn bằng, chứng

chỉ số tại Việt Nam, chương đã làm rõ vai trò của chuẩn Blockcerts V3, cơ chế bằng chứng Merkle, cơ chế neo dữ liệu qua đầu ra OP_RETURN của Bitcoin, mối quan hệ giữa tệp chứng thư Blockcerts và đặc tả W3C Verifiable Credentials (phân biệt rõ context VC v1.1 và v2.0) và ranh giới pháp lý theo Thông tư 10/2026/TT-BGDĐT, Luật số 91/2025/QH15 và Nghị định 356/2025/NĐ-CP [6], [7], [8], [17], [19].

Các nguyên tắc kiến trúc đúc kết từ Chương 1 là tiền đề trực tiếp để chuyển hóa thành bản thiết kế hệ thống trong Chương 2: lưu giữ toàn bộ dữ liệu cá nhân và dữ liệu nghiệp vụ ngoài chuỗi tại PostgreSQL, chỉ neo cam kết mật mã tối thiểu lên Bitcoin, kết hợp quy trình kiểm soát chéo Maker–Checker, Transactional Outbox, Checkpoint/Idempotency và chuỗi băm nhật ký kiểm toán (Audit Hash Chain) ở lớp ứng dụng. Chương 2 sẽ chuyển các nguyên tắc này thành yêu cầu chức năng có định lượng, mô hình đe dọa chi tiết, kiến trúc thành phần, máy trạng thái phát hành và thiết kế cơ sở dữ liệu bám sát chặt mã nguồn được công bố.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích bài toán và phạm vi hệ thống

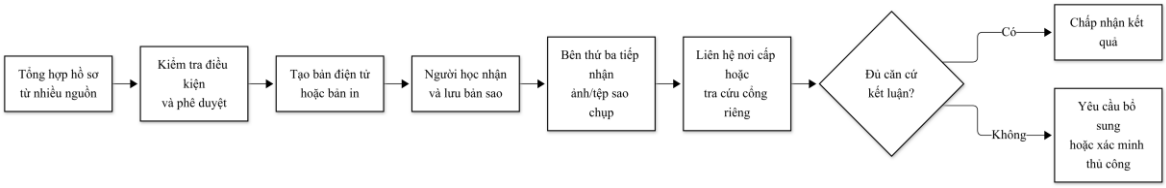
2.1.1. Quy trình nghiệp vụ hiện tại

Trong quy trình quản lý và phát hành văn bằng truyền thống, dữ liệu tốt nghiệp thường được tổng hợp thủ công hoặc bán tự động từ các phân hệ đào tạo, bảng tính và hồ sơ giấy. Cán bộ phụ trách đối chiếu điều kiện tốt nghiệp, lập hồ sơ phê duyệt, sau đó in phôi bằng hoặc xuất tệp điện tử để bàn giao cho người học. Khi cần xác minh, bên thứ ba thường phải sử dụng bản sao chứng thực, ảnh chụp văn bằng, gửi yêu cầu trực tiếp đến cơ sở đào tạo hoặc tra cứu trên cổng thông tin riêng nếu có. Quy trình này có độ trễ cao, phụ thuộc vào tính sẵn sàng của hệ thống tập trung và dễ phát sinh sai lệch khi hồ sơ được chuyển qua nhiều khâu.

Chi tiết các công đoạn trong quy trình nghiệp vụ hiện tại cùng những điểm yếu cần khắc phục được tổng hợp tại Bảng 2.1, đồng thời luồng xử lý và xác minh văn bằng truyền thống được khái quát hóa trên Hình 2.1.

Bảng 2.1. Quy trình nghiệp vụ hiện tại và điểm yếu cần xử lý

Công đoạn	Bên tham gia	Dữ liệu vào	Dữ liệu ra	Điểm yếu
Tổng hợp hồ sơ	Cán bộ nghiệp vụ	Thông tin người học, chương trình, kết quả	Danh sách đề nghị cấp	Dễ sai khi nhập lại hoặc ghép nhiều nguồn
Kiểm tra và phê duyệt	Cán bộ kiểm tra, người có thẩm quyền	Danh sách đề nghị và hồ sơ đối chiếu	Danh sách được phê duyệt	Khó chứng minh tách biệt người lập và người duyệt nếu thiếu nhật ký
Phát hành	Đơn vị cấp	Dữ liệu đã duyệt	Văn bằng hoặc bản điện tử	Tệp có thể bị sao chép, sửa hoặc phát hành ngoài quy trình
Bàn giao và lưu giữ	Đơn vị cấp, người học	Văn bằng đã phát hành	Bản do người học giữ	Nguy cơ thất lạc, hỏng hoặc khó đối chiếu phiên bản
Xác minh	Người học, bên thứ ba, đơn vị cấp	Bản sao hoặc dữ liệu tra cứu	Kết luận xác minh	Phụ thuộc nơi cấp; thời gian phản hồi và tiêu chí không đồng nhất
Thu hồi, đính chính	Đơn vị cấp, bên xác minh	Quyết định nghiệp vụ	Trạng thái mới	Trạng thái phân tán, khó đồng bộ tới mọi bên



Hình 2.1. Quy trình quản lý và xác minh văn bằng theo mô hình hiện tại

Sơ đồ trên Hình 2.1 cho thấy hạn chế lớn nhất của quy trình truyền thống không chỉ nằm ở bước nhập liệu mà còn ở sự đứt gãy giữa tệp văn bằng người học nắm giữ với trạng thái quản lý nội bộ của cơ sở đào tạo. Khi người học gửi bản sao văn bằng dạng tệp ảnh hoặc PDF cho đơn vị tuyển dụng, tệp tin này không kèm theo bằng chứng mật mã tự thân để kiểm chứng độc lập, buộc bên xác minh phải tra cứu qua cổng thông tin riêng của nhà trường hoặc gửi công văn đối soát thủ công.

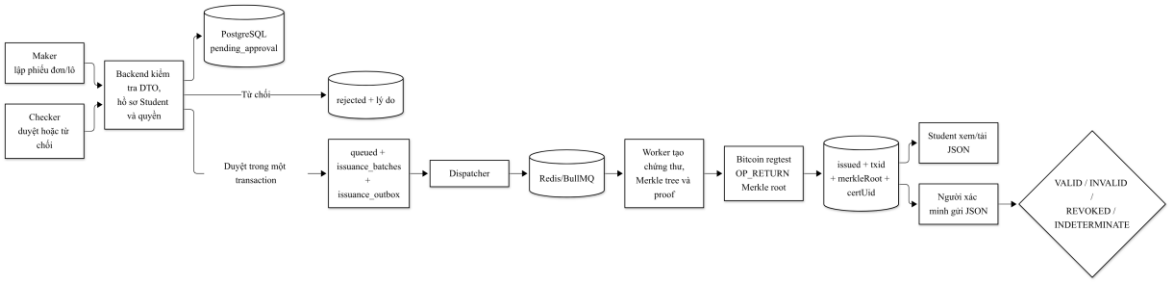
2.1.2. Các bất cập cần giải quyết

Từ các vấn đề nêu trên, quy trình hiện tại có bốn nhóm hạn chế. Thứ nhất, dữ liệu tập trung có thể bị sửa nhầm hoặc bị can thiệp khi thiếu cơ chế kiểm tra độc lập và nhật ký có khả năng phát hiện thay đổi. Thứ hai, bên xác minh phụ thuộc vào phản hồi của cơ sở đào tạo và không thể tự kiểm tra bằng chứng hoặc trạng thái thu hồi. Thứ ba, nếu một tài khoản vừa nhập liệu vừa phê duyệt, nguy cơ lạm quyền và phát hành sai thẩm quyền sẽ tăng. Thứ tư, chu trình phát hành qua hàng đợi, công cụ ký và mạng Blockchain có thể gặp lỗi giữa chừng; nếu không có checkpoint và cơ chế idempotency, việc thử lại có thể tạo bản ghi hoặc giao dịch trùng.

Blockchain chỉ hỗ trợ bảo vệ tính toàn vẹn của bằng chứng sau khi giao dịch được ghi nhận trên sổ cái; công nghệ này không tự xác nhận dữ liệu đầu vào là đúng và không thay thế thẩm quyền pháp lý của cơ sở đào tạo [1], [10]. Vì vậy, kiến trúc đề xuất phải kết hợp kiểm soát chéo Maker–Checker, định danh người học, lưu dữ liệu nghiệp vụ ngoài chuỗi, Audit Hash Chain, quản lý khóa và cơ chế neo cam kết mật mã lên Bitcoin.

2.1.3. Quy trình đề xuất

Luồng nghiệp vụ đề xuất tách rõ hai giai đoạn lập yêu cầu và phê duyệt phát hành. Cán bộ lập phiếu (Maker) tra cứu hồ sơ người học từ cơ sở dữ liệu của nhà trường và tạo phiếu ở trạng thái chờ duyệt. Cán bộ phê duyệt (Checker) kiểm tra chéo thông tin rồi quyết định phê duyệt hoặc từ chối; nếu được phê duyệt, tác vụ phát hành được ghi vào bảng hàng đợi trung gian (outbox) và chuyển cho tiến trình nền (Worker) xử lý bất đồng bộ. Toàn bộ luồng phối hợp từ khâu lập phiếu, duyệt phát hành đến xác minh công khai được minh họa trên Hình 2.2.



Hình 2.2. Quy trình đề xuất với kiểm soát Maker–Checker và neo Bitcoin

Hình 2.2 thể hiện rõ ranh giới giữa dữ liệu ngoài chuỗi và bằng chứng trên chuỗi trong quy trình đề xuất. Cơ sở dữ liệu PostgreSQL quản lý thông tin tài khoản, trạng thái phiếu yêu cầu, hàng đợi trung gian, lô phát hành và nhật ký kiểm toán; trong khi đó, mạng Bitcoin regtest chỉ lưu giữ cam kết mật mã tối thiểu thông qua trường OP_RETURN, còn tệp chứng thư JSON-LD kèm bằng chứng Merkle (Merkle proof) được cung cấp cho người học để phục vụ kiểm chứng độc lập.

2.1.4. Phạm vi, giả định và ràng buộc

Đồ án tập trung vào xác thực và phân quyền tài khoản; quản lý vòng đời yêu cầu phát hành; phát hành chứng thư đơn lẻ và theo lô từ 1 đến 500 bản ghi; cổng Holder; cổng Verify; thu hồi chứng thư; Audit Hash Chain và thử nghiệm trên môi trường pilot phục vụ nghiên cứu và kiểm thử [9]. Đồ án không bao gồm việc xét điều kiện tốt nghiệp đầu vào, không tuyên bố chứng thư của nguyên mẫu có giá trị pháp lý tương đương văn bằng chính thức và không đại diện cho kiến trúc vận hành trên Bitcoin mainnet.

Chứng thư Blockcerts trong nguyên mẫu là kết quả kỹ thuật dùng để minh họa kiến trúc, không đồng nhất với văn bằng hoặc chứng chỉ số có giá trị pháp lý theo Thông tư 10/2026/TT-BGDĐT [6]. Dữ liệu thử nghiệm là dữ liệu giả lập; bí mật và thông tin cá nhân nhạy cảm được loại khỏi bộ bằng chứng công khai trên Blockchain, phù hợp với các yêu cầu bảo vệ dữ liệu cá nhân [7], [8].

Vai trò hỗ trợ và giới hạn kỹ thuật của Blockchain qua từng công đoạn nghiệp vụ được phân định cụ thể trong Bảng 2.2.

Bảng 2.2. Vai trò và giới hạn của Blockchain theo công đoạn

Công đoạn	Dữ liệu ngoài chuỗi	Dữ liệu trên Bitcoin	Blockchain hỗ trợ	Blockchain không bảo đảm
Lập và duyệt phiếu	Hồ sơ Student, dữ liệu văn bằng, actor, trạng thái	Không ghi	Không áp dụng trực tiếp	Tính đúng của dữ liệu đầu vào và thẩm quyền nghiệp vụ

Công đoạn	Dữ liệu ngoài chuỗi	Dữ liệu trên Bitcoin	Blockchain hỗ trợ	Blockchain không bảo đảm
Phát hành	JSON chứng thư, certUid, txid, trạng thái	Merkle root trong OP_RETURN	Phát hiện thay đổi sau phát hành; bằng chứng thời điểm và neo	Danh tính ngoài đời thực hoặc giá trị pháp lý tự động
Xác minh	Chứng thư, issuer profile, log kỹ thuật	Giao dịch, OP_RETURN, confirmations	Đối chiếu proof với root và giao dịch	Tính đúng của thông tin đã được đơn vị cấp chấp nhận
Thu hồi	Lý do, actor, checkpoint, Revocation List	REVOKE:<certUid> trong OP_RETURN	Dấu vết thu hồi tối thiểu của nguyên mẫu	Chuẩn thu hồi chung của Blockcerts hoặc khả năng tái dựng DB hoàn chỉnh
Audit	Chuỗi hash audit trong PostgreSQL	Chưa neo head hash	Phần neo chứng thư/thu hồi độc lập với DB	Audit DB bất biến tuyệt đối

2.2. Thuật ngữ, tác nhân và quyền hạn

2.2.1. Quy ước thuật ngữ

Để thống nhất cách diễn đạt trong toàn bộ báo cáo, các thuật ngữ cốt lõi của hệ thống được định nghĩa tại Bảng 2.3.

Bảng 2.3. Quy ước thuật ngữ trong đồ án

Thuật ngữ	Ý nghĩa sử dụng thống nhất
Maker	Cán bộ lập yêu cầu phát hành; không được duyệt, từ chối, retry hoặc thu hồi.
Checker	Cán bộ kiểm tra và phê duyệt; quản lý tài khoản, retry và thu hồi. Không có role Admin riêng.
Student/Holder	Người học có tài khoản; chỉ xem và tải chứng thư liên kết với userId của chính mình.
Người xác minh	Tác nhân công khai gửi JSON chứng thư để nhận kết luận; không cần đăng nhập.
requestBatchId	Định danh nhóm phiếu được Maker tạo trong cùng một lần requestBatch.
issuanceBatchId	Định danh một lần phát hành cụ thể do Checker duyệt; là khóa idempotency của worker.
certUid	Định danh duy nhất của chứng thư sau phát hành.

Thuật ngữ	Ý nghĩa sử dụng thống nhất
Anchor	Giao dịch Bitcoin chứa Merkle root của một lô trong OP_RETURN.
Checkpoint	Raw transaction, txid và trạng thái bền vững được lưu trước hoặc quanh tác động blockchain để retry không tạo giao dịch khác.
Revocation List	Danh sách công khai được xây từ trạng thái thu hồi trong PostgreSQL của nguyên mẫu.

2.2.2. Phân tích tác nhân

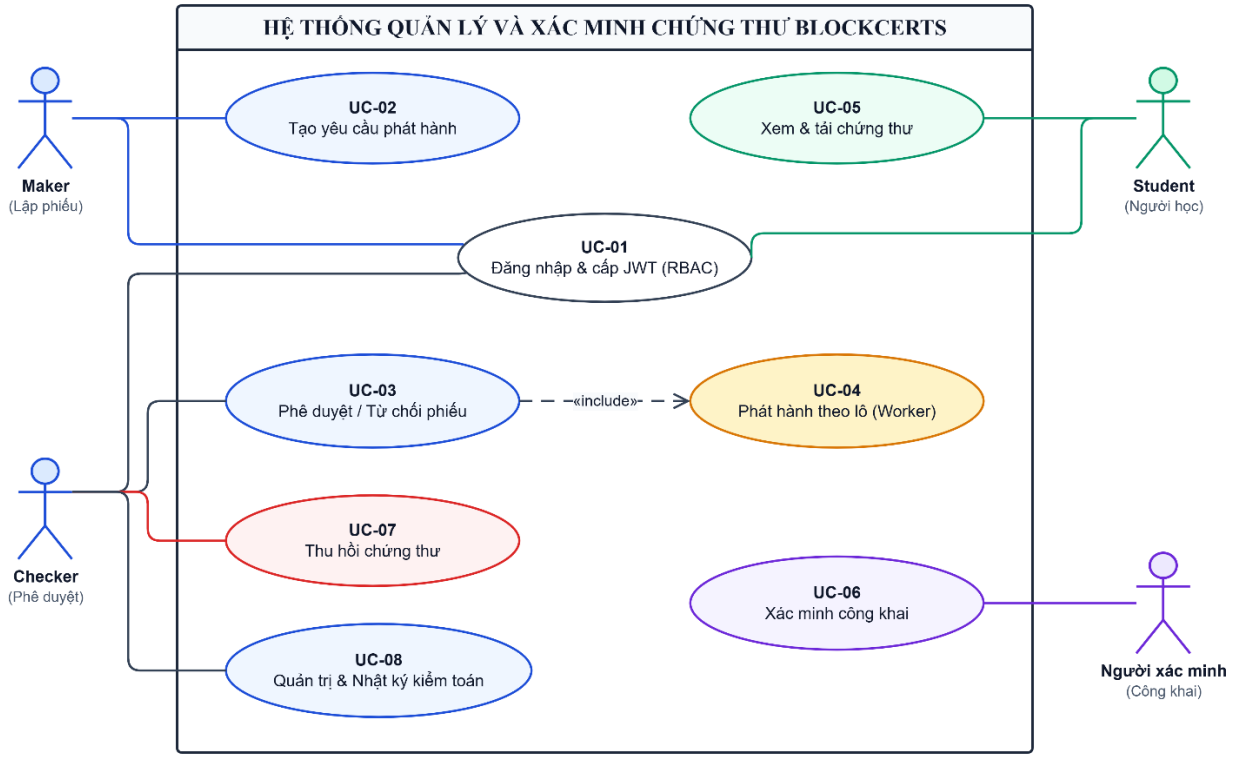
Mục tiêu sử dụng, phạm vi quyền hạn và các giới hạn bắt buộc của từng tác nhân trong hệ thống được phân tích tại Bảng 2.4.

Bảng 2.4. Mục tiêu, quyền hạn và giới hạn của tác nhân

Tác nhân	Mục tiêu	Quyền chính	Giới hạn bắt buộc
Maker	Chuẩn bị dữ liệu phát hành chính xác và theo dõi tiến độ	Tra cứu Student; tạo phiếu đơn/lô; xem phiếu và audit thuộc phạm vi mình	Không duyệt, từ chối, retry, quản lý tài khoản hoặc thu hồi
Checker	Kiểm soát chéo và quản trị vòng đời chứng thư	Duyệt/từ chối; xem toàn bộ; retry; quản lý tài khoản; thu hồi; xem audit	Không thay thế dữ liệu đầu vào bằng trường actor từ client; quyết định dựa trên JWT và trạng thái DB
Student	Nhận và lưu chứng thư của mình	Đăng nhập Holder; xem/tải chứng thư issued hoặc revoked của chính userId	Không xem chứng thư của Student khác; không vào cổng nghiệp vụ
Người xác minh	Tự kiểm tra chứng thư do Holder cung cấp	Gửi JSON; nhận VALID, INVALID, REVOKED hoặc INDETERMINATE	Không được truy cập dữ liệu quản trị; kết quả phụ thuộc bằng chứng và trạng thái sẵn có
Worker/Dispatcher	Thực thi phát hành bền vững	Đọc outbox, enqueue, tạo chứng thư, anchor, checkpoint và đối soát	Không phải tác nhân người dùng; chỉ xử lý lô đã được Checker duyệt

2.2.3. Mô hình use case tổng quát

Trên cơ sở bốn tác nhân đã xác định, phạm vi chức năng của hệ thống được cụ thể hóa thành tám ca sử dụng chính (từ UC-01 đến UC-08) và được mô hình hóa trong sơ đồ tổng quan trên Hình 2.3.



Hình 2.3. Tác nhân và nhóm use case trong ranh giới hệ thống

Sơ đồ ca sử dụng trên Hình 2.3 mô tả ranh giới hệ thống và tương tác của bốn nhóm tác nhân: Maker, Checker, Student và Người xác minh công khai. Các chức năng xác thực, lập phiếu, phê duyệt, tra cứu, thu hồi và xác minh nằm trong ranh giới ứng dụng. Bộ điều phối (Dispatcher) và tiến trình Worker là các dịch vụ nội bộ xử lý bất đồng bộ ở tầng nền nên không được biểu diễn như tác nhân người dùng.

2.3. Yêu cầu chức năng và đặc tả use case

2.3.1. Danh sách yêu cầu chức năng

Như thể hiện trên Hình 2.3, các ca sử dụng được phân bổ trực tiếp theo ranh giới quyền hạn của từng tác nhân: Maker thực hiện UC-01 và UC-02; Checker thực hiện UC-01, UC-03 (bao hàm UC-04 xử lý nền), UC-07 và UC-08; Student thực hiện UC-01 và UC-05; còn Người xác minh công khai thực hiện UC-06 không cần xác thực. Trên cơ sở đó, danh mục chi tiết các yêu cầu chức năng của hệ thống (từ FR-01 đến FR-14) được tổng hợp tại Bảng 2.5.

Bảng 2.5. Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tác nhân
FR-01	Đăng nhập bằng username/password và nhận JWT chứa sub, username, role	Maker, Checker, Student
FR-02	Tạo, liệt kê và cập nhật hồ sơ Student; không trả passwordHash	Checker

Mã	Yêu cầu	Tác nhân
FR-03	Tra cứu hồ sơ Student theo studentCode để lập phiếu	Maker
FR-04	Tạo một yêu cầu phát hành ở trạng thái pending_approval	Maker
FR-05	Tạo nhóm 1–500 yêu cầu và nhận requestBatchId	Maker
FR-06	Duyệt một/tập phiếu hoặc từ chối phiếu; hỗ trợ duyệt từng phần	Checker
FR-07	Phát hành bất đồng bộ, checkpoint, retry và reconciliation theo issuanceBatchId	Hệ thống, Checker
FR-08	Liệt kê/đọc chi tiết theo phạm vi Maker hoặc toàn bộ đối với Checker	Maker, Checker
FR-09	Xem và tải chứng thư thuộc Student hiện tại	Student
FR-10	Xác minh JSON công khai với bốn nhóm kết luận	Người xác minh
FR-11	Thu hồi idempotent, đối soát và tra cứu trạng thái thu hồi	Checker, công khai
FR-12	Công bố Issuer Profile và Revocation List	Công khai
FR-13	Ghi audit hash chain; đọc theo phạm vi và kiểm tra toàn vẹn	Hệ thống, Maker, Checker
FR-14	Cung cấp liveness tối thiểu; readiness được kiểm tra bằng script triển khai	Công khai có giới hạn

2.3.2. UC-01 – Đăng nhập

Kịch bản xác thực người dùng và cấp phát mã thông báo JWT trong ca sử dụng UC-01 được đặc tả chi tiết tại Bảng 2.6.

Bảng 2.6. Đặc tả UC-01 – Đăng nhập

Thuộc tính	Nội dung
Mục tiêu	Xác thực tài khoản và cấp JWT để backend áp dụng RBAC.
Tác nhân	Maker, Checker hoặc Student.
Tiền điều kiện	Tài khoản tồn tại; JWT_SECRET được cấu hình đủ mạnh; dịch vụ đang hoạt động.
Kích hoạt	Tác nhân gửi username và password tới POST /api/auth/login.
Luồng chính	1) Backend kiểm tra DTO. 2) Tìm user theo username. 3) bcrypt so sánh mật khẩu với passwordHash. 4) Ký JWT có sub=user.id, username và role, kèm thời hạn. 5) Ghi audit login. 6) Trả token, role và username.

Thuộc tính	Nội dung
Luồng thay thế	Khi token hết hạn, người dùng đăng nhập lại để nhận token mới. Giao diện điều hướng theo role nhưng không thay thế kiểm tra quyền ở backend.
Ngoại lệ	Sai username hoặc password: trả 401 với thông báo thống nhất. JWT thiếu/sai/hết hạn tại route bảo vệ: từ chối. Thiếu JWT_SECRET hợp lệ: ứng dụng không khởi tạo strategy.
Hậu điều kiện	Client có JWT hợp lệ; passwordHash không xuất hiện trong phản hồi; sự kiện đăng nhập được cố gắng ghi audit.

2.3.3. UC-02 – Tạo yêu cầu phát hành

Quy trình Maker tra cứu hồ sơ người học và khởi tạo yêu cầu phát hành đơn lẻ hoặc theo lô (UC-02) được đặc tả trong Bảng 2.7.

Bảng 2.7. Đặc tả UC-02 – Tạo yêu cầu phát hành

Thuộc tính	Nội dung
Mục tiêu	Tạo phiếu đơn hoặc nhóm phiếu từ dữ liệu Student tin cậy trong DB.
Tác nhân	Maker.
Tiền điều kiện	Maker có JWT hợp lệ; Student tồn tại, có recipientName, dateOfBirth, email, cohort; số hiệu văn bằng chưa tồn tại.
Kích hoạt	Maker gửi POST /api/issue/request hoặc POST /api/issue/batch/request.
Luồng chính	1) Backend lấy actor từ JWT. 2) Chuẩn hóa studentCode và nạp Student. 3) Kiểm tra DTO, địa chỉ P2PKH regtest, ngày cấp, năm tốt nghiệp, danh mục và số hiệu văn bằng. 4) Chụp dữ liệu Student vào phiếu, liên kết studentId. 5) Lưu pending_approval, requestedBy; với lô sinh requestBatchId chung. 6) Ghi audit request và trả ID.
Luồng thay thế	Lô nhận từ 1 đến 500 phần tử. Maker có thể tra cứu trước GET /api/issue/students/:studentCode. Các phiếu cùng requestBatchId có thể được Checker duyệt từng phần về sau.
Ngoại lệ	Student không tồn tại hoặc hồ sơ thiếu: 404/400. Số hiệu trùng trong payload hoặc DB: 409. Dữ liệu sai định dạng hoặc trường ngoài DTO: 400. Maker không đúng role: 403.
Hậu điều kiện	Phiếu ở pending_approval; chưa thuộc issuanceBatchId; chưa có certUid, txid hoặc Merkle root.

2.3.4. UC-03 – Phê duyệt hoặc từ chối

Cơ chế kiểm soát chéo của Checker khi phê duyệt hoặc từ chối phiếu phát hành (UC-03) được đặc tả tại Bảng 2.8.

Bảng 2.8. Đặc tả UC-03 – Phê duyệt hoặc từ chối

Thuộc tính	Nội dung
Mục tiêu	Áp dụng kiểm soát chéo trước khi đưa phiếu vào quy trình phát hành.
Tác nhân	Checker.
Tiền điều kiện	Checker có JWT hợp lệ; phiếu tồn tại và đang pending_approval.
Kích hoạt	Checker chọn duyệt một phiếu, duyệt tập ID hoặc từ chối một phiếu.
Luồng chính – duyệt	1) Khóa pessimistic các phiếu. 2) Kiểm tra tất cả còn pending_approval. 3) Sinh issuanceBatchId riêng cho lần duyệt và jobId ổn định. 4) Trong một transaction, chuyển phiếu sang queued, lưu approvedBy, tạo issuance_batches và issuance_outbox. 5) Commit. 6) Ghi audit và thử dispatch.
Luồng chính – từ chối	1) Cập nhật có điều kiện id + pending_approval. 2) Lưu rejectedBy, lý do và trạng thái rejected. 3) Ghi audit.
Luồng thay thế	Checker có thể duyệt một phần các phiếu cùng requestBatchId; mỗi lần duyệt tạo issuanceBatchId khác nhau. Enqueue lỗi không hủy transaction: outbox vẫn pending để dispatcher thử lại.
Ngoại lệ	ID thiếu, trạng thái đã đổi hoặc cạnh tranh approve/reject: từ chối thao tác. Actor trong body không được dùng; actor luôn lấy từ JWT.
Hậu điều kiện	Duyệt: có lô/outbox bền vững và phiếu queued. Từ chối: phiếu rejected kèm lý do; không tạo job.

2.3.5. UC-04 – Xử lý phát hành bất đồng bộ

Luồng xử lý phát hành bất đồng bộ, gom cây Merkle và neo bằng chứng lên mạng Bitcoin regtest (UC-04) được đặc tả tại Bảng 2.9.

Bảng 2.9. Đặc tả UC-04 – Xử lý phát hành bất đồng bộ

Thuộc tính	Nội dung
Mục tiêu	Tạo Blockcerts V3 cho một lần duyệt, neo đúng một Merkle root và có thể phục hồi an toàn.
Tác nhân	Dispatcher, BullMQ và issuance worker; Checker chỉ kích hoạt retry thủ công khi cần.
Tiền điều kiện	issuance_batches và issuance_outbox đã commit; phiếu queued; Blockcerts toolchain, Redis và Bitcoin regtest sẵn sàng.

Thuộc tính	Nội dung
Kích hoạt	Dispatcher phát hiện outbox pending hoặc lời gọi dispatch sau duyệt thành công.
Luồng chính	1) Enqueue bằng jobId ổn định. 2) Worker lấy advisory lock theo batch. 3) Chuyển batch/phiếu sang processing. 4) Tạo unsigned certificate theo chunk, gắn nonce ổn định. 5) Gom output để cert-issuer tạo một Merkle tree và raw transaction. 6) Lưu checkpoint trước/bao quanh broadcast. 7) Broadcast, tạo block, kiểm tra txid/root. 8) Ánh xạ output bằng nonce, lưu issued, certUid, txid, merkleRoot và completed.
Luồng thay thế	Outbox pending được retry với backoff. Job trùng bị bỏ qua bởi lock/jobId. Nếu đã có checkpoint, worker đối soát hoặc phát lại đúng raw transaction cũ, không tạo anchor mới. Checker có thể gọi retry batch ở trạng thái cho phép.
Ngoại lệ	Lỗi trước checkpoint có thể đưa lô/phiếu sang failed. Lỗi sau checkpoint hoặc phản hồi RPC mơ hồ đưa lô sang reconciliation_required để tiếp tục từ txid cố định. Checkpoint không khớp giao dịch/OP_RETURN bị từ chối.
Hậu điều kiện	Thành công: mọi phiếu trong issuanceBatchId có chứng thư và cùng anchor; lỗi: trạng thái và lỗi đủ để retry/đối soát mà không nhân đôi giao dịch.

2.3.6. UC-05 – Xem và tải chứng thư của Student

Quy trình sinh viên đăng nhập cổng Holder để tra cứu và tải tệp chứng thư cá nhân (UC-05) được mô tả trong Bảng 2.10.

Bảng 2.10. Đặc tả UC-05 – Xem và tải chứng thư của Student

Thuộc tính	Nội dung
Mục tiêu	Cho Holder nhận đúng chứng thư thuộc tài khoản của mình.
Tác nhân	Student.
Tiền điều kiện	Student đăng nhập; JWT chứa sub là userId; có chứng thư issued hoặc revoked liên kết studentId.
Kích hoạt	Cổng Holder gọi GET /api/issue/holder/certificates.
Luồng chính	1) Backend lấy userId từ JWT. 2) Truy vấn issued_certificates theo studentId và trạng thái issued/revoked. 3) Đọc tệp JSON theo certUid từ kho runtime nếu tồn tại. 4) Trả dữ liệu nghiệp vụ, trạng thái, txid, root và certificate. 5) Giao diện cho xem/tải JSON.
Luồng thay thế	Chứng thư revoked vẫn xuất hiện để giữ lịch sử và hiển thị trạng thái. Nếu chưa có chứng thư, trả danh sách rỗng.

Thuộc tính	Nội dung
Ngoại lệ	JWT sai/hết hạn hoặc role không phải Student: từ chối. Tệp JSON thiếu/hỏng: trường certificate là null; dữ liệu DB vẫn được trả để chỉ ra trạng thái, không tự tạo tệp giả.
Hậu điều kiện	Student chỉ nhận chứng thư gắn với userId của mình; không dùng họ tên làm khóa quyền.

2.3.7. UC-06 – Xác minh công khai

Quy trình xác minh công khai tệp chứng thư JSON với bốn nhóm kết luận đầu ra (UC-06) được đặc tả chi tiết tại Bảng 2.11.

Bảng 2.11. Đặc tả UC-06 – Xác minh công khai

Thuộc tính	Nội dung
Mục tiêu	Đánh giá cấu trúc, proof, anchor và trạng thái thu hồi của JSON chứng thư.
Tác nhân	Người xác minh.
Tiền điều kiện	Có JSON chứng thư; backend và các phụ thuộc cần thiết có thể truy cập. Không yêu cầu tài khoản.
Kích hoạt	Gửi POST /api/verify với toàn bộ certificate.
Luồng chính	1) Kiểm tra kích thước/DTO và credentialSubject.id. 2) Ghi tệp tạm quyền 0600. 3) Chạy verifier service có timeout. 4) Kiểm tra cấu trúc, proof và issuer; anchor adapter đọc giao dịch, OP_RETURN, input và confirmations. 5) Nếu lớp mật mã hợp lệ, backend tra certUid để áp trạng thái thu hồi. 6) Lưu verification log đã tối thiểu hóa và audit. 7) Trả kết luận chuẩn hóa.
Luồng thay thế	VALID khi bằng chứng hợp lệ và chưa thu hồi; INVALID khi nội dung/proof/txid không khớp; REVOKED khi bằng chứng hợp lệ nhưng đã thu hồi; INDETERMINATE khi thiếu bằng chứng hoặc lỗi phụ thuộc.
Ngoại lệ	JSON sai hoặc proof hỏng: INVALID. RPC/mạng/container không đủ dữ kiện: INDETERMINATE, không đánh đồng với giả mạo. Tài nguyên ngoài bị safe-fetch chặn nếu dùng URL/giao thức/IP/redirect nguy hiểm [28].
Hậu điều kiện	Người dùng nhận kết quả và chi tiết kỹ thuật phù hợp; log không lưu PII không cần thiết từ credentialSubject.

2.3.8. UC-07 – Thu hồi chứng thư

Việc thu hồi chứng thư trong nguyên mẫu kết hợp hai nguồn dữ liệu: trạng thái hiệu lực trong cơ sở dữ liệu PostgreSQL (cung cấp qua điểm cuối Revocation List công khai) và giao dịch OP_RETURN mang tải trọng REVOKE:<certUid> trên

mạng Bitcoin regtest. Trước khi phát sóng giao dịch lên mạng chuỗi khối, dịch vụ thu hồi ghi nhận điểm kiểm soát (checkpoint) chứa dữ liệu giao dịch thô đã ký (revocationRawTransaction) và mã giao dịch dự kiến (revocationIntentTxid) vào bảng chứng thư. Trường hợp lời gọi RPC tới nút Bitcoin bị quá thời gian chờ hoặc rơi vào trạng thái chưa xác định, lần thử lại tiếp theo sẽ đối soát và sử dụng lại đúng giao dịch đã lưu ở điểm kiểm soát nhằm ngăn chặn việc phát sinh hai giao dịch thu hồi trùng lặp cho cùng một chứng thư. Đặc tả chi tiết cho ca sử dụng thu hồi chứng thư (UC-07) được trình bày tại Bảng 2.12.

Bảng 2.12. Đặc tả UC-07 – Thu hồi chứng thư

Thuộc tính	Nội dung
Mục tiêu	Chuyển chứng thư đã phát hành sang revoked bằng quy trình có checkpoint và idempotency.
Tác nhân	Checker; reconciler là tác nhân nội bộ.
Tiền điều kiện	Checker có JWT; chứng thư ở issued hoặc đang trong trạng thái thu hồi có thể tiếp tục; Bitcoin wallet có UTXO phù hợp.
Kích hoạt	Checker gửi POST /api/revoke/:id kèm lý do.
Luồng chính	1) Lấy advisory lock toàn cục để tuần tự hóa UTXO. 2) Chuyển issued thành revocation_pending. 3) Tạo và ký đúng một raw transaction có OP_RETURN REVOKE:<certUid>. 4) Tính txid, lưu raw/intent txid/thời điểm trước broadcast. 5) Broadcast hoặc tra lại txid cố định. 6) Tạo block nếu cần và yêu cầu ít nhất một confirmation. 7) Lưu revoked, revocationTxid, actor, lý do, thời gian; ghi audit.
Luồng thay thế	Nếu đã revoked, trả kết quả cũ. Nếu tiến trình gián đoạn, reconciler gọi lại với checkpoint hiện có; chỉ phát lại raw transaction cũ. Endpoint /api/revoke/check và Revocation List cho phép tra cứu công khai.
Ngoại lệ	Không đúng role hoặc chứng thư chưa issued: từ chối. Không có UTXO, RPC lỗi hoặc checkpoint chưa hoàn tất: giữ revocation_pending/revocation_reconciliation_required và lưu lỗi để tiếp tục.
Hậu điều kiện	Thành công: Verify trả REVOKED và Revocation List phản ánh trạng thái DB; không tạo giao dịch thu hồi thay thế khi retry.

2.3.9. UC-08 – Quản trị và công bố thông tin hỗ trợ

Các thao tác quản trị tài khoản, kiểm tra chuỗi băm nhật ký và công bố thông tin hỗ trợ xác minh (UC-08) được trình bày tại Bảng 2.13.

Bảng 2.13. Đặc tả UC-08 – Quản trị và công bố thông tin hỗ trợ

Thuộc tính	Nội dung
Mục tiêu	Quản lý tài khoản, hồ sơ Student, audit và các tài nguyên công khai hỗ trợ xác minh.
Tác nhân	Checker; Maker đọc audit theo phạm vi; người dùng công khai đọc issuer/revocation/health.
Tiền điều kiện	Route quản trị yêu cầu JWT đúng role; tài nguyên công khai không yêu cầu đăng nhập.
Kích hoạt	Checker thao tác người dùng; Maker/Checker mở audit; verifier truy cập profile/list; hệ thống giám sát gọi health.
Luồng chính	1) Checker tạo/list user hoặc cập nhật các trường hồ sơ Student cho phép. 2) Password được bcrypt; phản hồi loại passwordHash. 3) Maker đọc audit liên quan actor/phiếu của mình, Checker đọc toàn bộ; integrity endpoint kiểm chuỗi hash. 4) Issuer Profile, Revocation List và /health trả dữ liệu công khai tối thiểu.
Luồng thay thế	Tạo tài khoản có thể qua /api/auth/register hoặc /api/admin/users, đều chỉ Checker. Revocation List được dựng từ các dòng revoked trong DB.
Ngoại lệ	Trùng username/studentCode: 409. Payload ngoài DTO hoặc hồ sơ không hợp lệ: 400. Sai role: 403. /health chỉ là liveness, không được diễn giải như readiness đầy đủ.
Hậu điều kiện	Dữ liệu quản trị thay đổi có audit; tài nguyên công khai không lộ secret hoặc passwordHash.

2.4. Yêu cầu phi chức năng

Các yêu cầu phi chức năng về an toàn, toàn vẹn, hiệu năng, khả dụng và quyền riêng tư cùng giới hạn thiết kế được tổng hợp tại Bảng 2.14.

Bảng 2.14. Yêu cầu phi chức năng và trạng thái thiết kế

Mã	Nhóm	Yêu cầu/biện pháp	Giới hạn còn lại
NFR-01	An toàn	bcrypt; JWT có hạn; RBAC backend; ValidationPipe; Helmet; throttling; CORS allowlist	Chưa có MFA, HSM/KMS hoặc kiểm thử xâm nhập độc lập
NFR-02	Toàn vẹn	Proof, Merkle root, anchor; checkpoint; ràng buộc DB; hash chain audit	Không phát hiện dữ liệu đầu vào sai nhưng đã được duyệt hợp lệ
NFR-03	Truy vết	requestedBy, approvedBy, revokedBy, txid, audit previousHash/entryHash	DBA có thể viết lại cả chuỗi nếu head hash không neo độc lập

Mã	Nhóm	Yêu cầu/biện pháp	Giới hạn còn lại
NFR-04	Hiệu năng	Tối đa 500 chứng thư/lô; unsigned chunk 10; tối đa 16 chunk song song; một anchor/lô	Các ngưỡng là cấu hình/giới hạn đã đánh giá trong môi trường thử nghiệm
NFR-05	Khả dụng	HTTP tách tác vụ dài; outbox, BullMQ, retry, checkpoint và reconciler	Chưa có HA đa vùng; phụ thuộc một cụm PostgreSQL/Redis/Bitcoin pilot
NFR-06	Vận hành	Compose, migration, script readiness và health tối thiểu	/health không kiểm tra toàn bộ dependency
NFR-07	Riêng tư	Không ghi PII trực tiếp lên Bitcoin; log xác minh tối thiểu; retention cấu hình	Cần chính sách lưu giữ/xóa, DPIA và giám sát truy cập khi triển khai thật
NFR-08	Triển khai thực nghiệm	Cố định commit, source manifest, manifest theo lô, checksum và bộ bằng chứng	Kết quả regtest không suy rộng thành mainnet/production
NFR-09	Dễ dùng	Ba cổng tách biệt; thông báo bốn trạng thái xác minh; tải JSON trực tiếp	Cần đánh giá khả dụng với người dùng thực tế
NFR-10	Khả năng bảo trì	NestJS module, TypeORM migration, DTO, test backend và verifier	Toolchain Blockcerts/container cần quản lý phiên bản và chuỗi cung ứng

2.5. Mô hình đe dọa

2.5.1. Phương pháp đánh giá

Mô hình đe dọa được xây dựng nhằm xác định rõ các nhóm tài sản trọng yếu, đối tượng tấn công, bề mặt tiếp xúc và cơ chế kiểm soát tương ứng trong phạm vi nguyên mẫu. Các tài sản cần bảo vệ bao gồm khóa riêng phát hành định dạng WIF, khóa bí mật ký JWT, thông tin xác thực kết nối Bitcoin RPC và cơ sở dữ liệu, tài khoản người dùng, hồ sơ cá nhân của người học, tệp chứng thư JSON-LD, trạng thái phát hành hoặc thu hồi cùng chuỗi băm nhật ký kiểm toán. Các tác nhân đe dọa gồm đối tượng bên ngoài gửi tệp chứng thư giả mạo hoặc tấn công gây gián đoạn dịch vụ, người dùng nội bộ vượt quyền, kẻ tấn công đánh cắp khóa mật mã và quản trị viên cơ sở dữ liệu can thiệp trực tiếp vào bản ghi. Bề mặt tấn công trải rộng trên các điểm cuối API công khai, giao diện đăng nhập, máy chủ proxy ngược Nginx, hàng đợi BullMQ, giao diện RPC của nút Bitcoin và các thư mục lưu trữ tệp tạm trong quá trình vận hành (runtime).

Kiến trúc an toàn của hệ thống áp dụng nguyên tắc từ chối theo mặc định (Deny by Default), trong đó mọi yêu cầu gửi tới các API nghiệp vụ đều phải trải qua bước xác thực phiên đăng nhập và kiểm tra quyền hạn theo vai trò cũng như phạm vi đối tượng. Việc ẩn hoặc làm mờ các nút thao tác trên giao diện người dùng (frontend) chỉ nhằm hỗ trợ trải nghiệm điều hướng, tuyệt đối không được coi là biện pháp kiểm soát truy cập. Mọi quyết định cấp phép hay từ chối thao tác đều bắt buộc thực thi tập trung tại tầng dịch vụ phía máy chủ (backend) theo đúng khuyến nghị an toàn của OWASP [20].

2.5.2. Ma trận tài sản–đe dọa–kiểm soát

Trên cơ sở các tài sản và bề mặt tấn công đã nhận diện, ma trận ánh xạ giữa mối đe dọa, hậu quả và biện pháp kiểm soát được trình bày tại Bảng 2.15.

Bảng 2.15. Ma trận tài sản–đe dọa–kiểm soát

Đe dọa	Tài sản/Hậu quả	Kiểm soát hiện hành	Giới hạn còn lại
Maker tự duyệt hoặc đọc phiếu Maker khác	Quy trình phê duyệt, dữ liệu nghiệp vụ	Checker-only approve/reject; lọc requestedBy ở list/detail/audit	Tài khoản Checker bị chiếm vẫn có quyền rộng
Student đọc chứng thư người khác	PII và chứng thư Holder	Liên kết studentId; truy vấn theo sub JWT	Quản trị tài khoản/thiết bị đầu cuối vẫn cần bảo vệ

Đe dọa	Tài sản/Hậu quả	Kiểm soát hiện hành	Giới hạn còn lại
Sửa nội dung, proof hoặc txid	Tính xác thực chứng thư	Hash, Merkle proof, anchor adapter, confirmations	Không chứng minh đầu vào nghiệp vụ vốn đúng
Chiếm mật khẩu/JWT	Danh tính và quyền truy cập	bcrypt, JWT có hạn, RolesGuard, throttling	Chưa có MFA, token revocation hoặc phát hiện bất thường nâng cao
Lộ WIF hoặc RPC secret	Khả năng phát hành/thu hồi giả	Secret ngoài Git, quyền tệp, RPC loopback	Chưa dùng HSM/KMS hoặc quy trình luân chuyển khóa hoàn chỉnh
Lỗi giữa DB và Redis	Mất job dù phiếu đã queued	Transactional outbox, dispatcher retry, jobId ổn định	Redis/DB vẫn cần backup và giám sát
Lỗi sau broadcast	Tạo anchor/thu hồi trùng khi retry	Lưu raw transaction và intent txid; reconciliation dùng giao dịch cũ	Cần cảnh báo và quy trình vận hành khi reconciliation kéo dài
SSRF qua tài nguyên JSON-LD/issuer	Truy cập mạng nội bộ/metadata	Safe-fetch kiểm URL, DNS/IP, redirect, kích thước và timeout [28]	Phụ thuộc đúng cấu hình egress và cập nhật thư viện
Sửa/xóa audit DB	Mất khả năng truy trách nhiệm	Hash chain SHA-256 và endpoint integrity	DBA có thể viết lại toàn chuỗi hoặc xóa đuôi nếu head không neo độc lập
DoS Verify/API	Cạn CPU, RAM, I/O	Body limit, throttling, timeout, Nginx	Chưa thử tải tấn công dài hạn hoặc autoscaling
Rò rỉ dữ liệu runtime/bằng chứng	Lộ PII hoặc secret	Tệp tạm 0600; log xác minh tối thiểu; loại secret khỏi Git/bộ công bố	Cần chính sách retention, xóa và kiểm toán truy cập chính thức

2.6. Phân tích và lựa chọn kiến trúc

2.6.1. Blockchain công khai và Blockchain cấp quyền

Sự khác biệt giữa mô hình Blockchain công khai và Blockchain cấp quyền đối với bài toán kiểm chứng văn bằng được đối chiếu tại Bảng 2.16.

Bảng 2.16. So sánh Blockchain công khai và Blockchain cấp quyền

Tiêu chí	Blockchain công khai	Blockchain cấp quyền	Lựa chọn trong đồ án
Khả năng kiểm tra	Bất kỳ bên nào có thể đọc bằng chứng	Phụ thuộc quyền truy cập mạng/consortium	Ưu tiên mô hình công khai để verifier độc lập về nguyên tắc
Quản trị	Không do một cơ sở đơn lẻ kiểm soát	Quản trị bởi nhóm thành viên đã biết	Nguyên mẫu sử dụng Bitcoin regtest trong môi trường LAB để kiểm chứng thực nghiệm, không tuyên bố mức phi tập trung mainnet
Riêng tư	Dữ liệu ghi lên chuỗi công khai	Có thể hạn chế người đọc	Chỉ ghi cam kết mật mã tối thiểu; PII để ngoài chuỗi
Chi phí/hiệu năng	Phụ thuộc mạng và phí	Có thể tối ưu theo consortium	Regtest loại chi phí thật để tập trung kiểm chứng thiết kế
Phù hợp liên tổ chức	Cao nếu dùng chuẩn mở và mạng công khai	Cần thỏa thuận thành viên	Blockcerts–Bitcoin phù hợp mục tiêu nghiên cứu về kiểm chứng độc lập

2.6.2. Bitcoin, Ethereum và Hyperledger Fabric

Việc đối chiếu giữa ba nền tảng tiêu biểu gồm Bitcoin (kết hợp Blockcerts), Ethereum và Hyperledger Fabric trong phạm vi đồ án được trình bày tại Bảng 2.17.

Bảng 2.17. So sánh nền tảng cho phạm vi đồ án

Tiêu chí	Bitcoin + Blockcerts	Ethereum	Hyperledger Fabric
Mô hình	UTXO; ghi Merkle root bằng OP_RETURN	Tài khoản và hợp đồng thông minh	Sổ cái cấp quyền, chaincode
Mức độ phức tạp	Không cần hợp đồng; dùng toolchain Blockcerts	Cần thiết kế, triển khai và kiểm toán contract	Cần tổ chức, channel, MSP và vận hành consortium
Khả năng theo chuẩn chứng thư	Blockcerts có hệ sinh thái và proof/receipt phù hợp [4], [5]	Có thể xây riêng bằng VC/contract	Có thể xây ứng dụng riêng trong mạng cấp quyền
Phù hợp mục tiêu	Phù hợp bài toán neo cam kết tối thiểu và xác minh	Mạnh hơn nhu cầu, tăng bề mặt tấn công	Phù hợp khi nhiều tổ chức đã có cơ chế quản trị chung

Tiêu chí	Bitcoin + Blockcerts	Ethereum	Hyperledger Fabric
Quyết định	Được chọn; dùng Bitcoin regtest trong nguyên mẫu	Không chọn	Không chọn

2.6.3. Phân tách cam kết mật mã trên chuỗi và dữ liệu ngoài chuỗi

Hệ thống phân biệt rõ ràng dữ liệu trên chuỗi giữa hai luồng nghiệp vụ: luồng phát hành chỉ neo giá trị băm Merkle Root (32 bytes) của cả lô lên Bitcoin; luồng thu hồi trong nguyên mẫu ghi chuỗi REVOKE:<certUid> trong đầu ra OP_RETURN của giao dịch riêng biệt. Toàn bộ hồ sơ người học, nội dung văn bằng, JSON chứng thư, trạng thái nghiệp vụ và audit được lưu ngoài chuỗi trong PostgreSQL. Sự phân tách này hạn chế tối đa việc đưa dữ liệu cá nhân lên sổ cái công khai, nhưng việc đưa certUid lên chuỗi vẫn tạo ra liên kết logic với chứng thư, đồng thời đặt ra yêu cầu bảo mật cao đối với PostgreSQL, khóa ký, tệp chứng thư và các bản sao lưu.

Cần lưu ý rằng cơ chế ghi chuỗi REVOKE:<certUid> vào đầu ra OP_RETURN là phần mở rộng riêng của nguyên mẫu nghiên cứu nhằm lưu vết thu hồi trên sổ cái, không phải là quy chuẩn thu hồi mặc định trong đặc tả Blockcerts. Trong thiết kế hiện hành, danh sách thu hồi công khai (Revocation List) vẫn lấy nguồn dữ liệu chuẩn từ cơ sở dữ liệu PostgreSQL; để có thể tái tạo hoàn toàn trạng thái thu hồi trực tiếp từ mạng Blockchain mà không phụ thuộc cơ sở dữ liệu nội bộ, hệ thống trong tương lai cần bổ sung bộ lập chỉ mục giao dịch (indexer) cùng các quy tắc xác thực nguồn phát sóng được chuẩn hóa.

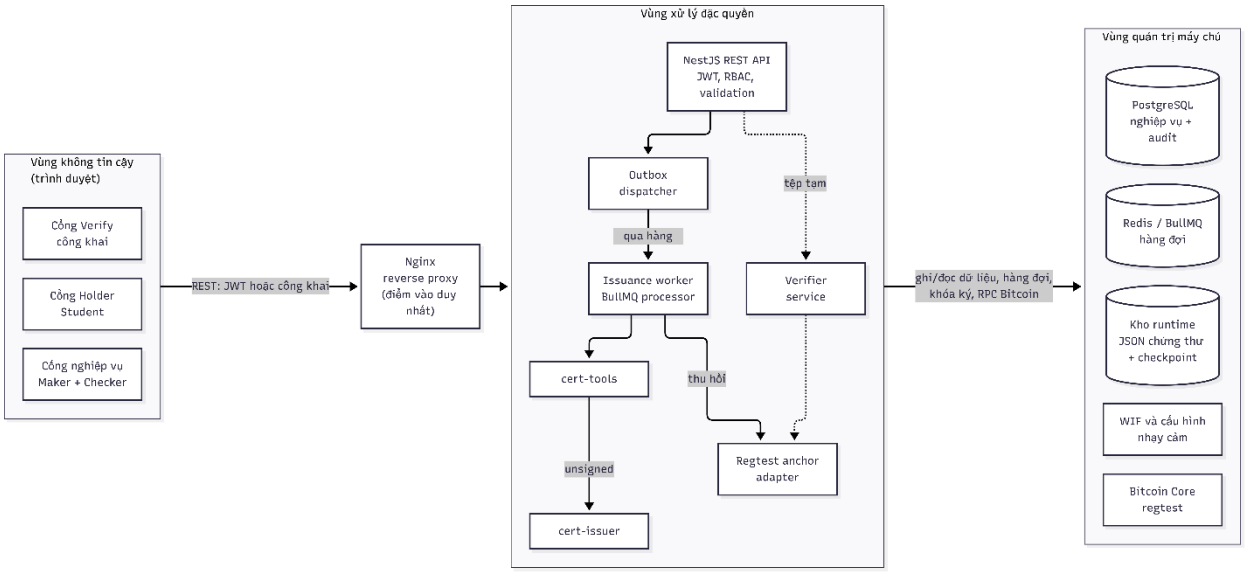
2.6.4. Đồng bộ, bất đồng bộ và phát hành theo lô

Về mô hình xử lý, mỗi yêu cầu HTTP từ trình duyệt chỉ thực hiện phần kiểm tra nghiệp vụ ngắn gọn trong cơ sở dữ liệu và trả về trạng thái tiếp nhận ngay lập tức; toàn bộ quá trình khởi tạo chứng thư và neo dữ liệu được tách sang xử lý bất đồng bộ qua hàng đợi, bởi thời gian thực thi của các công cụ cert-tools, cert-issuer và mạng Bitcoin thường vượt quá chu kỳ sống tiêu chuẩn của một kết nối HTTP. Trong phạm vi thử nghiệm của đồ án, mỗi lô phát hành hỗ trợ tối đa 500 chứng thư. Các tệp chứng thư chưa ký (unsigned certificates) được khởi tạo theo từng cụm (chunk) gồm 10 hồ sơ, với tối đa 16 cụm xử lý song song trên máy chủ thực nghiệm; toàn bộ đầu ra sau đó được gom chung để tính toán một gốc Merkle duy nhất và phát sóng trong một giao dịch Bitcoin. Trường hợp quy mô phát hành thực tế vượt quá ngưỡng này, hệ thống cần chia nhỏ thành nhiều tác vụ neo độc lập thay vì dồn vào một lô vượt quá giới hạn đã được kiểm thử.

2.7. Kiến trúc và các luồng xử lý

2.7.1. Kiến trúc thành phần và ranh giới tin cậy

Để bảo đảm nguyên tắc phân tách trách nhiệm và kiểm soát truy cập chặt chẽ giữa các tầng xử lý, hệ thống được chia thành các khối thành phần đặt trong ba vùng ranh giới tin cậy khác nhau như minh họa trên Hình 2.4.



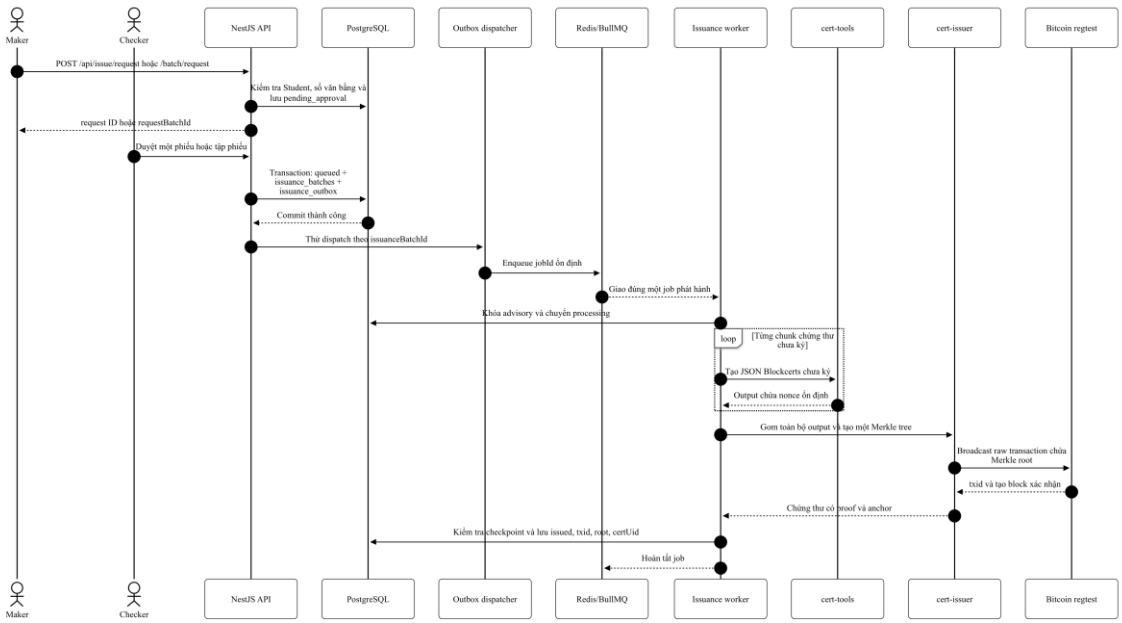
Hình 2.4. Kiến trúc thành phần và ranh giới tin cậy

Như thể hiện trên Hình 2.4, kiến trúc tổng thể bao gồm ba ứng dụng giao diện React, máy chủ proxy ngược Nginx (điểm tiếp nhận lưu lượng duy nhất), dịch vụ backend NestJS, cơ sở dữ liệu PostgreSQL, hàng đợi Redis/BullMQ, tiến trình Worker, bộ công cụ Blockcerts (cert-tools, cert-issuer), dịch vụ xác minh (Verifier Service), bộ chuyển đổi neo (Regtest Anchor Adapter) và nút mạng Bitcoin Core regtest. Trong mô hình này, trình duyệt người dùng được xếp vào vùng không tin cậy; tầng ứng dụng backend và Worker nằm trong vùng xử lý đặc quyền; còn cơ sở dữ liệu, hàng đợi, khóa riêng WIF và các tệp cấu hình nhạy cảm được cô lập trong vùng quản trị máy chủ. Sự phân tách ranh giới này bảo đảm mọi quyết định xác thực và phân quyền đều được cưỡng chế tại tầng backend thay vì phụ thuộc vào trạng thái hiển thị phía giao diện.

Trong việc lựa chọn ngăn xếp công nghệ: NestJS cung cấp cấu trúc module hóa, cơ chế bảo vệ Guard và đường ống kiểm tra dữ liệu ValidationPipe theo tiêu chuẩn hướng đối tượng [21]; PostgreSQL bảo đảm các thuộc tính ACID của giao dịch cùng hệ thống ràng buộc khóa ngoại chặt chẽ [22]; TypeORM đóng vai trò quản lý thực thể dữ liệu và phiên bản dịch chuyển sơ đồ cơ sở dữ liệu (migration) [23]; trong khi BullMQ tận dụng cấu trúc bộ nhớ của Redis để phân tách nhất quán các tác vụ xử lý tốn kém tài nguyên ra khỏi chu trình phản hồi của yêu cầu HTTP [24]. Tuy nhiên, các khung công nghệ trên không tự động đem lại sự an toàn hoàn toàn; mức độ an ninh thực tế phụ thuộc trực tiếp vào việc thiết lập chính sách cấu hình, chất lượng kiểm thử tự động, công tác quản lý tham số bí mật và kỷ luật thực thi quy trình vận hành.

2.7.2. Trình tự phát hành theo lô

Quá trình phát hành chứng thư theo lô đòi hỏi sự phối hợp tuần tự giữa thao tác phê duyệt đồng bộ qua REST API và chuỗi tác vụ ký số, neo dữ liệu bất đồng bộ ở tầng nền. Chi tiết trình tự tương tác giữa các thành phần trong luồng phát hành theo lô được mô tả trên Hình 2.5.



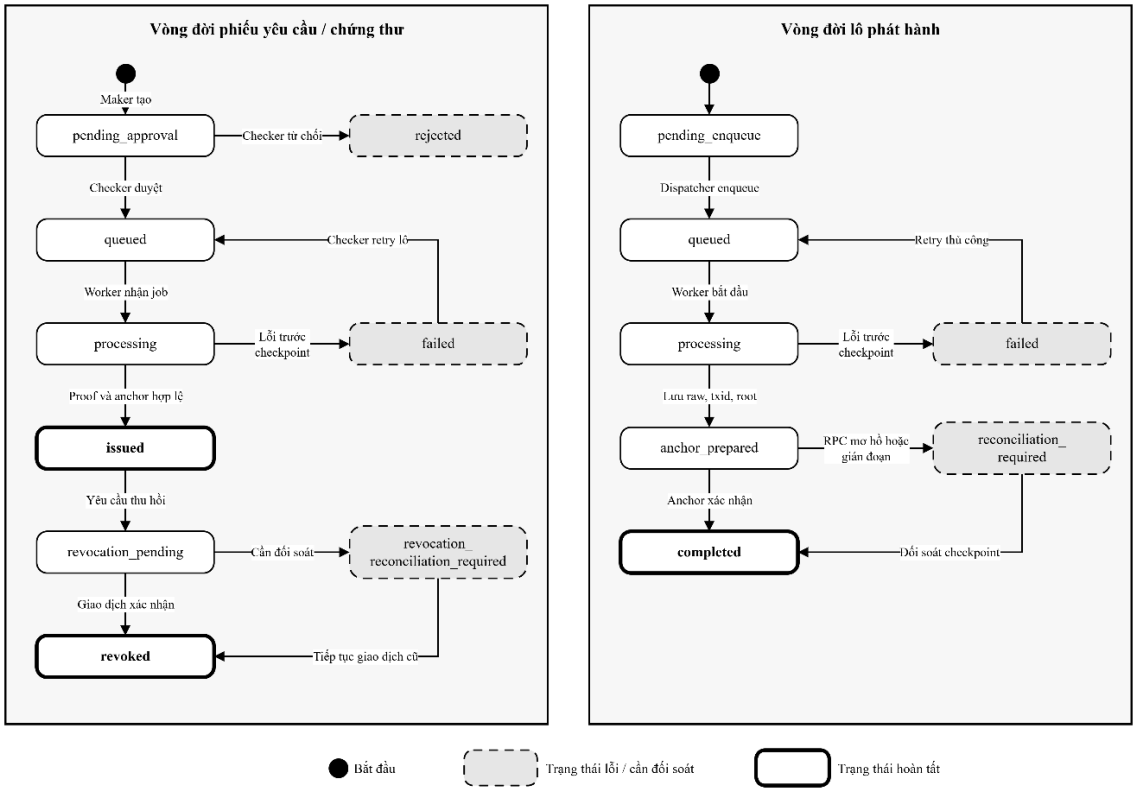
Hình 2.5. Trình tự phát hành chứng thư theo lô

Theo biểu đồ tuần tự trên Hình 2.5, luồng phát hành được bảo vệ bởi bốn điểm kiểm soát trạng thái then chốt: cơ chế Transactional Outbox bảo đảm không thất thoát sự kiện sau khi phê duyệt, mã định danh đợt phát hành (issuanceBatchId) duy nhất, khóa tư vấn (advisory lock) ở cấp cơ sở dữ liệu nhằm ngăn chặn xung đột giữa các tiến trình Worker, và điểm kiểm soát (checkpoint) lưu giữ giao dịch thô trước khi phát sóng lên mạng Bitcoin.

Đặc biệt, thiết kế tách bạch rõ hai mã định danh lô ở hai giai đoạn nghiệp vụ khác nhau: requestBatchId dùng để gom nhóm các phiếu do Maker khởi tạo trong cùng một lần gửi yêu cầu, trong khi issuanceBatchId đại diện cho một đợt phát hành cụ thể được sinh ra tại thời điểm Checker phê duyệt. Nhờ đó, một nhóm yêu cầu (requestBatchId) có thể được chia thành nhiều đợt phát hành (issuanceBatchId) khác nhau khi Checker thực hiện phê duyệt từng phần. Toàn bộ chứng thư thuộc cùng một issuanceBatchId sẽ chia sẻ chung một giá trị gốc Merkle (merkleRoot) và một mã giao dịch neo (txid), nhưng mỗi chứng thư vẫn sở hữu mã định danh certUid và đường dẫn kiểm chứng riêng biệt; vì lý do này, trường txid không được phép đặt ràng buộc duy nhất (unique) trên bảng chứng thư.

2.7.3. Máy trạng thái yêu cầu phát hành

Nhằm kiểm soát chặt chẽ tính nhất quán của dữ liệu qua các bước duyệt, phát hành, phục hồi lỗi và thu hồi, vòng đời của từng phiếu yêu cầu và vòng đời của lô phát hành được mô hình hóa thành hai máy trạng thái liên kết với nhau trên Hình 2.6.



Hình 2.6. Máy trạng thái yêu cầu phát hành và thu hồi

Sơ đồ trên Hình 2.6 phân tách rõ hai vòng đời song hành: vòng đời nghiệp vụ của từng phiếu yêu cầu (từ khi khởi tạo đến khi phát hành và thu hồi) và vòng đời điểm kiểm soát của lô phát hành tại tiến trình Worker. Cần lưu ý rằng định nghĩa kiểu dữ liệu và các tệp di trú cơ sở dữ liệu (migration) vẫn giữ lại giá trị trạng thái trung gian approved để bảo đảm tính tương thích ngược với cấu trúc lược đồ ban đầu; tuy nhiên, trong luồng thực thi thực tế, ngay khi giao dịch phê duyệt của Checker và bản ghi hàng đợi outbox được xác nhận thành công, trạng thái của phiếu sẽ chuyển trực tiếp từ pending_approval sang queued.

Các quy tắc chuyển trạng thái hợp lệ của phiếu yêu cầu và lô phát hành được đặc tả chi tiết tại Bảng 2.18.

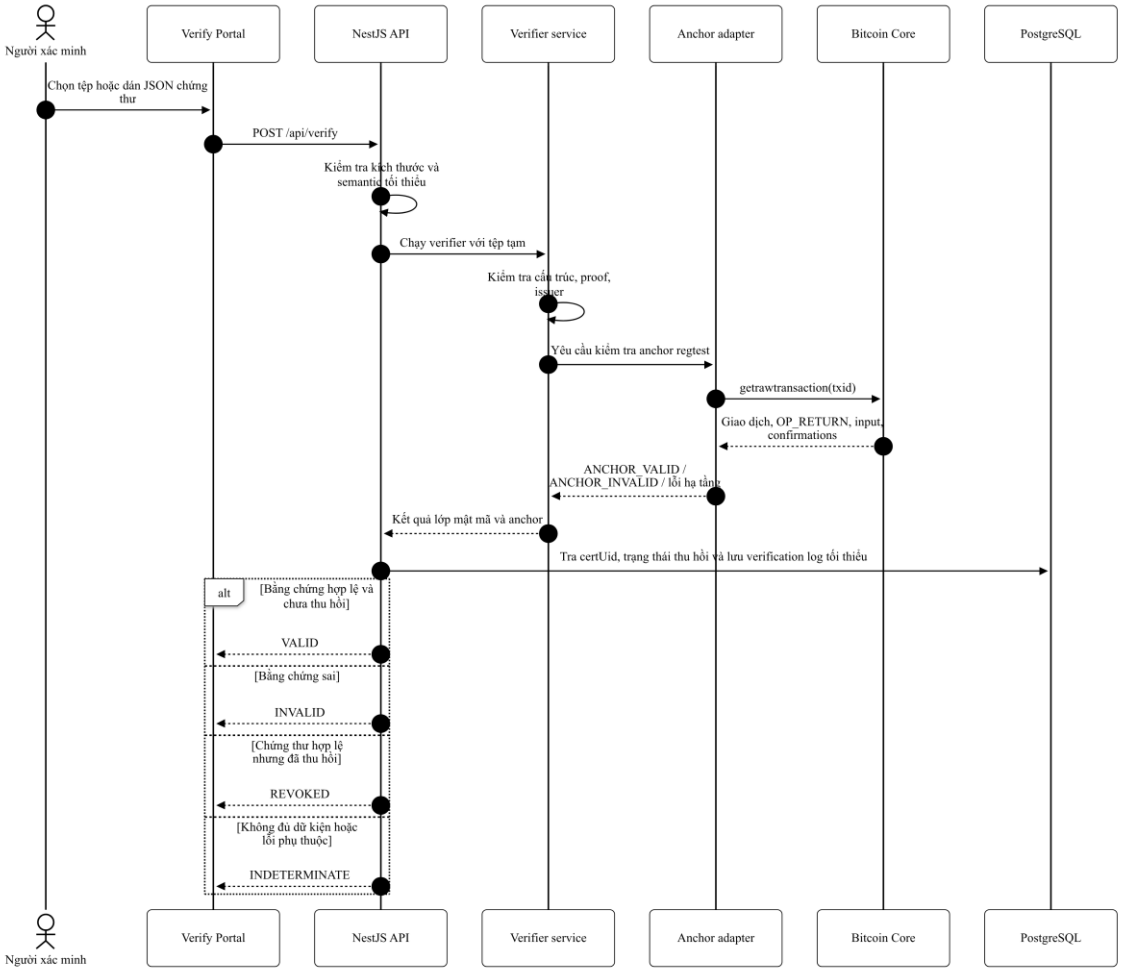
Bảng 2.18. Chuyển trạng thái yêu cầu và lô phát hành

Đối tượng	Trạng thái nguồn	Sự kiện	Trạng thái đích	Điều kiện/ghi chú
Phiếu	—	Maker tạo	pending_approval	Chưa có issuanceBatchId
Phiếu	pending_approval	Checker từ chối	rejected	Cập nhật có điều kiện, lưu lý do
Phiếu	pending_approval	Checker duyệt	queued	Cùng transaction với batch và outbox

Đối tượng	Trạng thái nguồn	Sự kiện	Trạng thái đích	Điều kiện/ghi chú
Phiếu	queued	Worker nhận job	processing	Có advisory lock theo batch
Phiếu	processing	Proof/anchor hoàn tất	issued	Có certUid, txid, merkleRoot
Phiếu	processing	Lỗi trước checkpoint	failed	Cho phép retry thủ công theo lô
Phiếu	issued	Bắt đầu thu hồi	revocation_pending	Checkpoint giao dịch thu hồi sẽ được lưu
Phiếu	revocation_pending	Cần đối soát	revocation_reconciliation_required	Đã có hoặc cần tiếp tục checkpoint
Phiếu	revocation_pending/reconciliation	Giao dịch xác nhận	revoked	Lưu revocationTxid và thời gian
Lô	pending_enqueue	Dispatcher enqueue	queued	Outbox chuyển dispatched
Lô	queued	Worker bắt đầu	processing	Tăng attemptCount
Lô	processing	Lưu raw/txid/root	anchor_prepared	Checkpoint bền vững
Lô	anchor_prepared	RPC mơ hồ/gián đoạn	reconciliation_required	Dùng lại raw transaction cũ
Lô	processing/reconciliation	Cập nhật mọi phiếu	completed	Không tạo anchor thứ hai
Lô	processing	Lỗi kết thúc không checkpoint	failed	Checker có thể retry

2.7.4. Trình tự xác minh công khai

Khi bên thứ ba gửi tệp chứng thư JSON-LD lên cổng tra cứu công khai, hệ thống thực hiện quy trình kiểm tra kết hợp giữa xác minh mật mã trên chuỗi và đối chiếu trạng thái thu hồi ngoài chuỗi. Trình tự xử lý và các nhánh kết luận đầu ra của luồng xác minh công khai được biểu diễn trên Hình 2.7.



Hình 2.7. Trình tự xác minh công khai và các nhánh kết quả

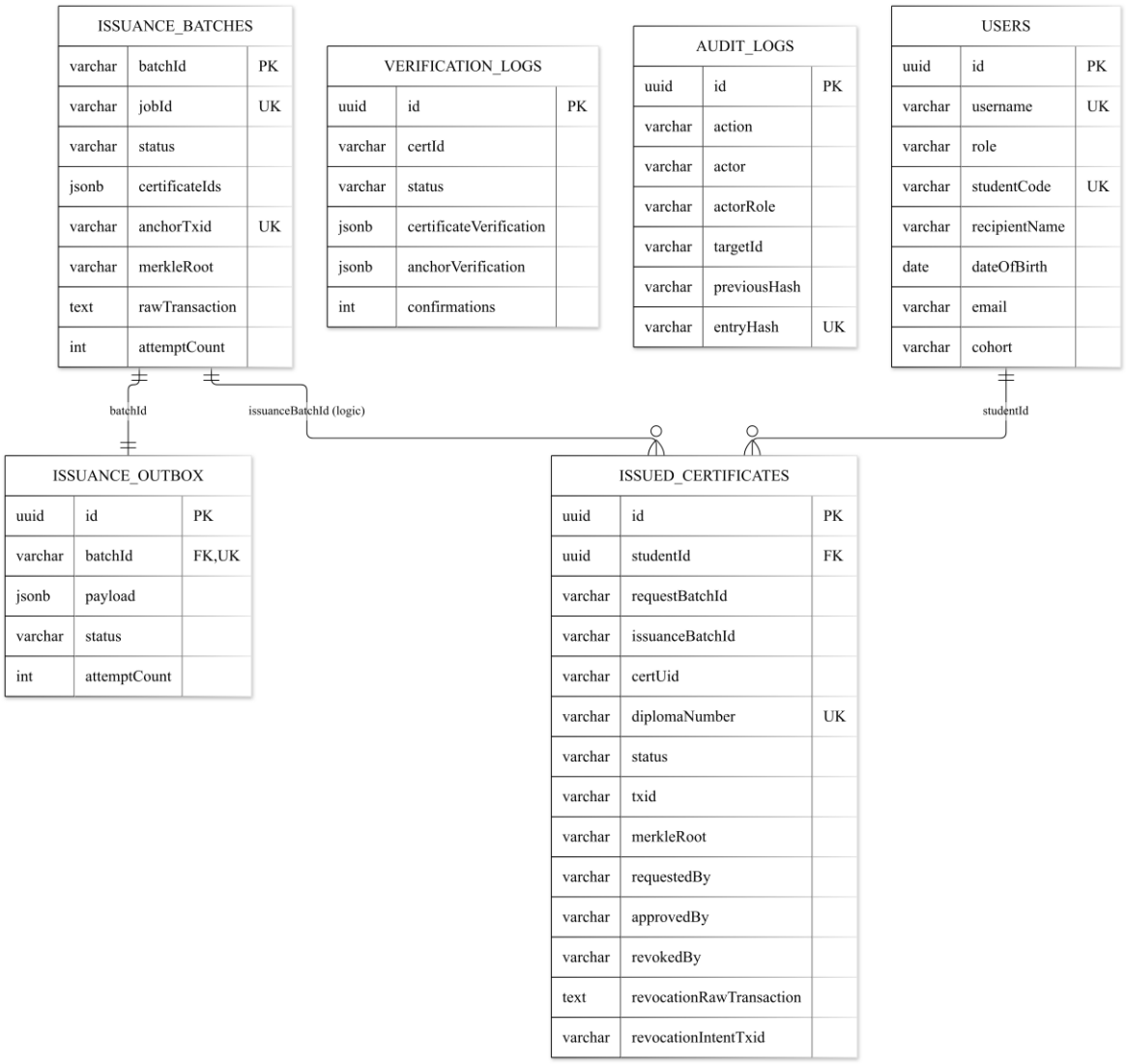
Biểu đồ trên Hình 2.7 thể hiện sự phân tách trách nhiệm rõ ràng giữa các phân hệ tham gia xác minh. Dịch vụ xác minh (Verifier Service) phối hợp với bộ chuyển đổi neo (Anchor Adapter) để kiểm tra cấu trúc ngữ nghĩa JSON-LD, bằng chứng đường dẫn Merkle và tính hợp lệ của giao dịch neo OP_RETURN trên nút Bitcoin Core. Chỉ sau khi tầng kiểm tra mật mã và giao dịch neo xác nhận hợp lệ, dịch vụ backend mới tiếp tục truy vấn mã định danh certUid trong cơ sở dữ liệu PostgreSQL để đối chiếu trạng thái thu hồi, ghi nhận bản ghi nhật ký tối thiểu (đã lược bỏ thông tin cá nhân nhạy cảm) và trả về một trong bốn kết luận chuẩn hóa (VALID, INVALID, REVOKED hoặc INDETERMINATE) cho người dùng.

Chứng thư được biểu diễn bằng JSON-LD theo mô hình W3C Verifiable Credentials. Đồ án dùng VC Data Model v2.0 để mô tả quan hệ logic giữa Issuer, Holder và Verifier [17]. Tệp Blockcerts V3 trong cấu hình triển khai thực tế sử dụng context VC v1.1 cùng namespace Blockcerts và bộ chữ ký MerkleProof2019 [4], [5], [19]. Trong cấu trúc đối tượng chứng thư JSON, trường proof.type được định nghĩa chính xác là MerkleProof2019 (thay vì bộ DataIntegrityProof tổng quát); trường proofValue chứa chuỗi mã hóa đường dẫn Merkle (Merkle proof path) phục vụ quá trình giải mã bằng LDMerkleProof2019 và tính lại giá trị băm gốc Merkle Root để đối chiếu với giao dịch neo trên Blockchain. Các nguyên tắc toàn vẹn dữ liệu được đối chiếu chặt chẽ với khuyến nghị W3C Data Integrity [25].

2.8. Thiết kế dữ liệu ngoài chuỗi

2.8.1. Mô hình dữ liệu đang triển khai

Dữ liệu nghiệp vụ ngoài chuỗi của hệ thống được tổ chức trong hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL gồm sáu bảng vật lý. Cấu trúc chi tiết các trường khóa cùng mối quan hệ giữa các bảng dữ liệu đang triển khai được thể hiện trên Hình 2.8.



Hình 2.8. Mô hình dữ liệu vật lý hiện tại và liên kết logic

Sơ đồ quan hệ thực thể trên Hình 2.8 phân biệt rõ hai loại liên kết trong thiết kế cơ sở dữ liệu: các liên kết có ràng buộc khóa ngoại vật lý cưỡng chế (foreign key constraint) và các liên kết logic phục vụ truy vết nghiệp vụ. Cụ thể, trường studentId trong bảng issued_certificates liên kết khóa ngoại chặt chẽ tới bảng users; trường batchId trong bảng issuance_outbox là khóa ngoại đồng thời có ràng buộc duy nhất (unique) tham chiếu tới bảng issuance_batches. Ngược lại, mối liên hệ giữa issuanceBatchId với batchId, hoặc trường certId trong bảng verification_logs, cùng các thuộc tính actor và targetId trong bảng audit, được quản lý ở mức logic ứng dụng nhằm tối ưu hóa hiệu năng ghi và giảm thiểu sự phụ thuộc ràng buộc cứng giữa các phân hệ.

Cấu trúc các bảng dữ liệu vật lý hiện hành cùng khóa định danh và ràng buộc toàn vẹn được tổng hợp trong Bảng 2.19.

Bảng 2.19. Các bảng dữ liệu vật lý hiện tại

Bảng	Khóa/định danh	Nhóm dữ liệu chính	Ràng buộc/ý nghĩa
users	id; username unique; studentCode unique có điều kiện	Tài khoản, role; hồ sơ Student	role ∈ maker/checker/student; passwordHash không trả qua API
issued_certificates	id; diplomaNumber unique; certUid; requestBatchId; issuanceBatchId	Ảnh chụp dữ liệu văn bằng; actor; trạng thái; anchor; thu hồi	studentId FK users; trạng thái issued/revoked phải có certUid, txid, root
issuance_batches	batchId PK; jobId unique; anchorTxid unique	Danh sách certificateIds, trạng thái lô, raw transaction, root, mốc thời gian	Checkpoint worker và idempotency theo lần duyệt
issuance_outbox	id; batchId unique/FK	Payload enqueue, pending/dispatched, attempt, lỗi	Mỗi batch tối đa một outbox; xóa cascade theo batch
verification_logs	id; certId logic	Trạng thái và kết quả kỹ thuật đã tối thiểu hóa	Không lưu PII không cần thiết; retention cấu hình
audit_logs	id; entryHash unique	Action, actor, role, target, txid, previousHash	Chuỗi băm phát hiện can thiệp (tamper-evident), không phải kho bất biến tuyệt đối

2.8.2. Lựa chọn định danh và ánh xạ kết quả

Quyền sở hữu chứng thư của người học được xác định chặt chẽ thông qua khóa ngoại studentId đối chiếu với trường định danh sub trong mã thông báo JWT, tuyệt đối không dựa vào chuỗi họ tên hiển thị vì họ tên giữa các sinh viên có thể trùng nhau hoặc thay đổi theo thời gian. Khi Maker khởi tạo phiếu yêu cầu, dịch vụ backend sẽ truy xuất hồ sơ người học từ bảng users và lưu bản chụp dữ liệu (snapshot) của các trường thông tin cần thiết vào bảng issued_certificates. Cơ chế này vừa duy trì liên kết định danh ổn định, vừa bảo toàn nguyên vẹn nội dung văn bằng tại thời điểm phát hành ngay cả khi thông tin hồ sơ gốc của người học được cập nhật về sau.

Hệ thống không dùng tên tệp do Blockcerts sinh tự động làm định danh vì tên có thể xung đột. Mỗi dòng roster nhận một nonce xác định từ 32 ký tự thập lục phân đầu tiên của SHA-256 trên chuỗi gồm tiền tố hệ thống, issuanceBatchId và khóa bản ghi. Worker đọc nonce trong tệp kết quả để ánh xạ từng chứng thư về đúng bản ghi cơ sở dữ liệu.

2.8.3. Mô hình chuẩn hóa đề xuất cho giai đoạn sau

Trong nguyên mẫu, một bản ghi issued_certificates vừa biểu diễn yêu cầu đang xử lý vừa lưu kết quả chứng thư sau phát hành. Thiết kế này phù hợp với giai đoạn thử nghiệm nhưng trộn lẫn hai vòng đời. Ở giai đoạn sau, có thể tách thành issuance_requests, certificates và revocation_requests, đồng thời bổ sung academic_record để liên kết với hệ thống đào tạo. Cách phân tách này giúp áp dụng chính sách lưu giữ độc lập và thuận lợi hơn cho kiểm toán.

2.8.4. Nguồn dữ liệu chuẩn đối với thu hồi và audit

Trong kiến trúc hiện tại, danh sách thu hồi công khai (Revocation List) được tổng hợp trực tiếp từ các bản ghi có trạng thái revoked và trường revocationTxid trong cơ sở dữ liệu PostgreSQL. Mặc dù giao dịch OP_RETURN thu hồi tạo ra một bằng chứng mật mã độc lập trên sổ cái Bitcoin, nguyên mẫu hiện chưa tích hợp bộ lập chỉ mục (indexer) chuyên dụng để tự động tái dựng toàn bộ trạng thái thu hồi thuần túy từ dữ liệu chuỗi khối. Tương tự, bảng nhật ký kiểm toán (audit_logs) sử dụng cặp trường previousHash và entryHash để phát hiện mọi hành vi chỉnh sửa hoặc xóa bỏ bản ghi ở phần giữa chuỗi; tuy nhiên, người có quyền quản trị cơ sở dữ liệu (DBA) vẫn có khả năng cắt bỏ các bản ghi ở cuối hoặc tính toán lại toàn bộ chuỗi băm nếu giá trị băm đầu chuỗi (head hash) chưa được neo định kỳ lên nguồn lưu trữ độc lập bên ngoài. Bên cạnh đó, do một số sự kiện nhật ký được ghi nhận sau khi giao dịch nghiệp vụ đã hoàn tất xác nhận (commit), việc bảo đảm tính nguyên tử tuyệt đối giữa thao tác nghiệp vụ và bản ghi kiểm toán vẫn là một giới hạn kỹ thuật cần tiếp tục hoàn thiện.

2.9. Thiết kế API và phân quyền

2.9.1. Quy ước chung

Các giao diện lập trình ứng dụng (API) trao đổi dữ liệu thống nhất theo định dạng JSON và sử dụng cơ chế xác thực JWT Bearer trong tiêu đề HTTP Authorization. Đường ống kiểm tra dữ liệu đầu vào (ValidationPipe) được kích hoạt đồng thời ba tùy chọn transform, whitelist và forbidNonWhitelisted nhằm tự động loại bỏ và từ chối mọi thuộc tính lạ nằm ngoài đặc tả DTO. Định danh của tác nhân thực hiện các thao tác nhạy cảm như phê duyệt, từ chối, tạo tài khoản hay thu hồi chứng thư luôn được trích xuất trực tiếp từ đối tượng req.user sau khi JwtStrategy xác thực chữ ký token, tuyệt đối không tiếp nhận từ phần thân (body) của yêu cầu gửi lên. Về thứ tự kiểm soát truy cập, lớp bảo vệ xác thực (AuthGuard) luôn được thực thi trước lớp kiểm tra vai trò (RolesGuard) [26]. Ngoài ra, các đường dẫn tĩnh như batch/request, batch/approve, holder/certificates và revoke/check đều được khai báo ưu tiên trước các đường dẫn chứa tham số động (:id) nhằm ngăn ngừa xung đột định tuyến trong NestJS.

2.9.2. Đặc tả endpoint chính

Danh sách các điểm cuối (endpoint) chính của tầng giao diện lập trình ứng dụng được đặc tả tại Bảng 2.20 (danh mục đối chiếu đầy đủ với tầng điều khiển được trình bày tại Phụ lục 1).

Bảng 2.20. Đặc tả các endpoint chính

Phương thức	Endpoint	Quyền	Chức năng
POST	/api/auth/login	Công khai	Đăng nhập, nhận JWT
POST	/api/auth/register	Checker	Tạo tài khoản
GET	/api/admin/users	Checker	Liệt kê tài khoản, không passwordHash
POST	/api/admin/users	Checker	Tạo tài khoản qua cổng nghiệp vụ
PATCH	/api/admin/users/:id/student-profile	Checker	Cập nhật hồ sơ Student cho phép
GET	/api/audit	Maker, Checker	Maker xem theo phạm vi; Checker xem toàn bộ
GET	/api/audit/integrity	Maker, Checker	Kiểm tra chuỗi hash audit
POST	/api/issue/request	Maker	Tạo một phiếu

Phương thức	Endpoint	Quyền	Chức năng
POST	/api/issue/batch/request	Maker	Tạo nhóm 1–500 phiếu
GET	/api/issue/students/:studentCode	Maker	Tra hồ sơ Student chỉ đọc
POST	/api/issue/:id/approve	Checker	Duyệt một phiếu
POST	/api/issue/batch/approve	Checker	Duyệt tập phiếu
POST	/api/issue/batch/:batchId/retry	Checker	Retry lô theo issuanceBatchId
POST	/api/issue/:id/reject	Checker	Từ chối phiếu pending
GET	/api/issue	Maker, Checker	Danh sách; Maker lọc requestedBy
GET	/api/issue/:id	Maker, Checker	Chi tiết; Maker chỉ phiếu của mình
GET	/api/issue/holder/certificates	Student	Chứng thư theo userId hiện tại
POST	/api/revoke/check	Công khai	Tra trạng thái theo certUid
POST	/api/revoke/:id	Checker	Thu hồi chứng thư
POST	/api/verify	Công khai	Xác minh JSON Blockcerts
GET	/api/blockcerts/issuers/kma/profile.json	Công khai	Issuer Profile
GET	/api/blockcerts/issuers/kma/revocation-list.json	Công khai	Revocation List
GET	/health	Công khai	Liveness tối thiểu

2.9.3. Ma trận phân quyền

Ma trận phân quyền truy cập chức năng giữa các vai trò Maker, Checker, Student và tác nhân công khai được tổng hợp tại Bảng 2.21.

Bảng 2.21. Ma trận phân quyền chức năng

Chức năng	Maker	Checker	Student	Công khai/Hệ thống
Đăng nhập	Có	Có	Có	Không
Quản lý tài khoản/hồ sơ Student	Không	Có	Không	Không

Chức năng	Maker	Checker	Student	Công khai/Hệ thống
Tra cứu Student để lập phiếu	Có	Có qua quản trị	Không	Không
Tạo phiếu đơn/lô	Có	Không	Không	Không
Xem phiếu/chứng thư nghiệp vụ	Chỉ phiếu requestedBy của mình	Toàn bộ	Chỉ issued/revoked theo userId	Không
Duyệt/từ chối/retry	Không	Có	Không	Worker xử lý lô đã duyệt
Thu hồi	Không	Có	Không	Chỉ tra cứu công khai
Xem audit	Theo actor/phiếu của mình	Toàn bộ	Không	Hệ thống ghi tự động
Xác minh JSON	Có thể dùng như công khai	Có thể dùng như công khai	Có thể dùng như công khai	Có, không đăng nhập
Issuer Profile/Revocation List/health	Đọc	Đọc	Đọc	Đọc

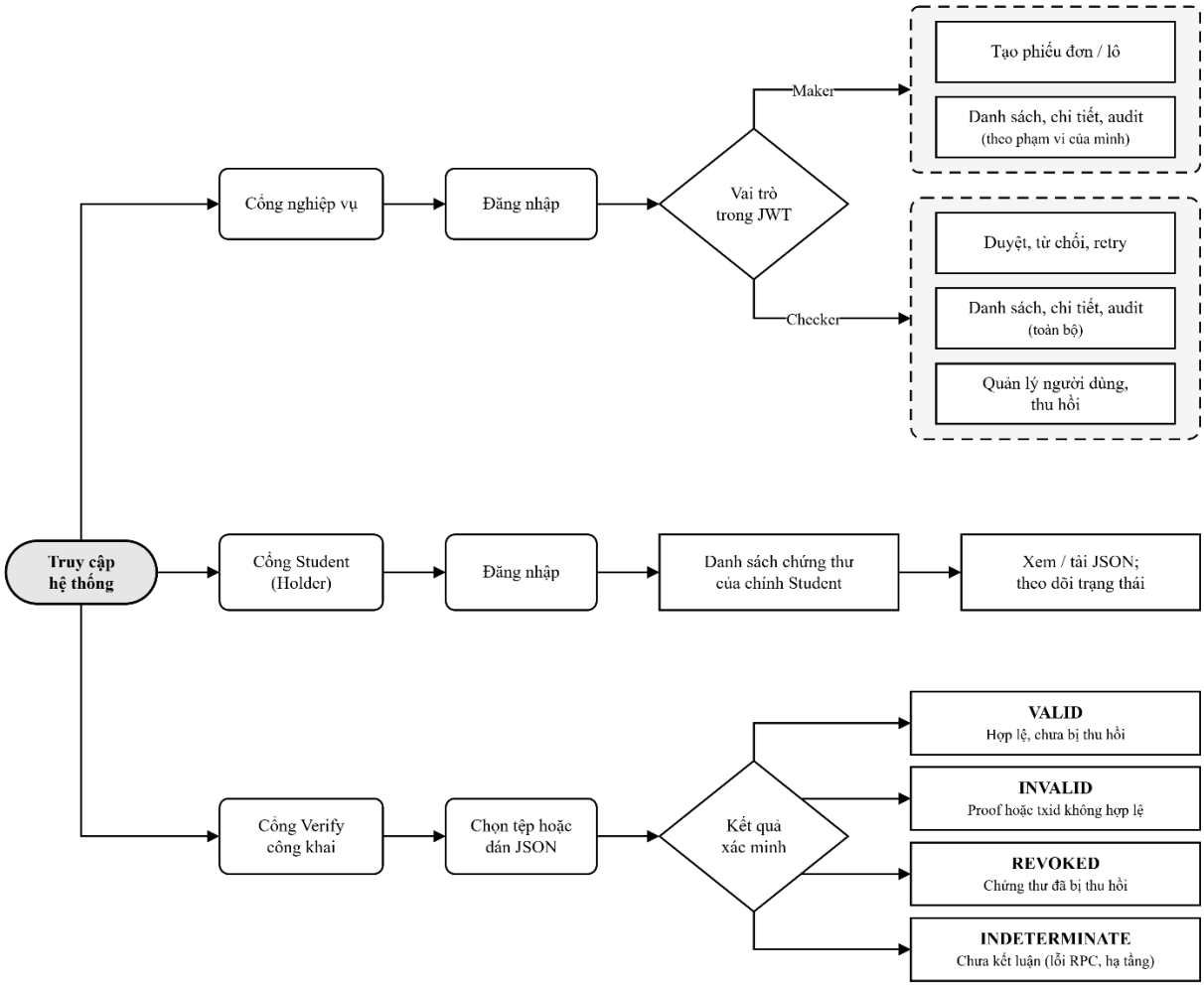
2.10. Thiết kế ba cổng giao diện

2.10.1. Nguyên tắc thiết kế

Ba cổng giao diện người dùng cùng kết nối tới một dịch vụ backend duy nhất nhưng được biên dịch và triển khai thành ba ứng dụng Web độc lập: cổng nghiệp vụ (frontend-admin) dùng chung cho Maker và Checker, cổng người học (frontend-client) dành cho Student và cổng xác minh (frontend-verify) phục vụ tra cứu công khai. Mặc dù thanh điều hướng trên giao diện tự động thích ứng theo vai trò người dùng để giảm thiểu thao tác nhầm lẫn, tầng backend vẫn là nơi quyết định cuối cùng về quyền truy cập. Đối với cổng Verify, ứng dụng chuyển tiếp toàn bộ nội dung tệp JSON chứng thư về dịch vụ xác minh tại backend thay vì tự xử lý trên trình duyệt, qua đó bảo đảm quy trình kiểm tra mật mã và đối chiếu trạng thái thu hồi diễn ra nhất quán.

2.10.2. Sơ đồ điều hướng

Cấu trúc phân nhánh màn hình và luồng di chuyển của từng nhóm tác nhân trên ba cổng giao diện Web được tổng hợp trong sơ đồ điều hướng trên Hình 2.9.



Hình 2.9. Sơ đồ điều hướng ba cổng giao diện

Như thể hiện trên Hình 2.9, luồng điều hướng của hệ thống được tách thành ba nhánh rõ rệt ngay từ điểm truy cập. Tại cổng nghiệp vụ, sau khi đăng nhập thành công, giao diện phân nhánh chức năng dựa trên trường vai trò (role) trong mã thông báo JWT: Maker được dẫn tới màn hình lập phiếu và danh sách yêu cầu thuộc phạm vi cá nhân, trong khi Checker truy cập các màn hình phê duyệt, thu hồi, quản lý người dùng và nhật ký kiểm toán toàn cục. Tại cổng Holder, người học sau khi xác thực chỉ được điều hướng tới danh sách chứng thư gắn trực tiếp với định danh sub của mình để xem chi tiết hoặc tải tệp JSON. Riêng cổng Verify công khai cho phép người dùng tải lên hoặc dán nội dung JSON chứng thư mà không cần qua bước đăng nhập, sau đó hiển thị trực quan một trong bốn trạng thái kết luận kỹ thuật (VALID, INVALID, REVOKED hoặc INDETERMINATE).

2.10.3. Đặc tả màn hình và trạng thái

Đặc tả chi tiết các màn hình làm việc và trạng thái hiển thị trên ba cổng giao diện được trình bày tại Bảng 2.22.

Bảng 2.22. Đặc tả màn hình và trạng thái giao diện

Cổng/Màn hình	Người dùng	Dữ liệu/Thao tác	Trạng thái cần thể hiện
Nghiệp vụ – Đăng nhập	Maker, Checker	Username, password	Đang gửi; sai thông tin; token hết hạn
Nghiệp vụ – Lập phiếu	Maker	Tra Student; nhập dữ liệu văn bằng; tạo đơn/lô	Hồ sơ thiếu; trùng số; pending_approval
Nghiệp vụ – Danh sách/chi tiết	Maker, Checker	Lọc trạng thái/requestBatchId; xem actor, batch, txid	pending, queued, processing, issued, failed, revoked và trạng thái phục hồi
Nghiệp vụ – Duyệt	Checker	Chọn một/tập phiếu; duyệt/từ chối; retry	Khóa khi không còn pending; hiển thị issuanceBatchId/jobId
Nghiệp vụ – Người dùng	Checker	Tạo user; cập nhật hồ sơ Student	Trùng username/studentCode; hợp lệ/không hợp lệ
Nghiệp vụ – Thu hồi	Checker	Nhập lý do; xác nhận thao tác	pending/reconciliation/revoked; không lặp giao dịch
Nghiệp vụ – Audit	Maker, Checker	Danh sách và integrity summary	Maker theo phạm vi; cảnh báo chuỗi hỏng
Holder – Ví chứng thư	Student	Danh sách; xem/tải JSON	issued và revoked; tệp thiếu; chưa có dữ liệu
Verify – Xác minh	Mọi người	Chọn/dán JSON; gửi kiểm tra	VALID, INVALID, REVOKED, INDETERMINATE; lỗi định dạng/hạ tầng

2.11. An toàn triển khai và vận hành

2.11.1. Xác thực, phân quyền và bảo vệ API

Mật khẩu người dùng được băm một chiều bằng thuật toán bcrypt kèm chuỗi muối (salt) ngẫu nhiên; mã thông báo JWT được thiết lập thời hạn hiệu lực ngắn và bắt buộc thẩm tra chữ ký tại backend trong mỗi yêu cầu truy cập [26], [27]. Đặc biệt, chiến lược xác thực JwtStrategy thực hiện truy vấn lại thông tin vai trò từ cơ sở dữ liệu nhằm giảm thiểu rủi ro sử dụng lại mã thông báo cũ sau khi quyền hạn của tài khoản đã thay đổi. Cơ chế cô lập theo đối tượng (object isolation) giới hạn Maker chỉ được truy cập các phiếu do chính mình khởi tạo, Student chỉ đọc được các chứng thư thuộc quyền sở hữu cá nhân và chỉ Checker mới được thực thi các thao tác phê duyệt hoặc quản trị. Bên cạnh đó, hệ thống kết hợp bộ lọc bảo vệ tiêu đề HTTP (Helmet), cơ chế giới hạn tần suất gọi API (throttling), danh sách nguồn truy cập hợp lệ (CORS allowlist) và giới hạn dung lượng gói tin đầu vào làm các lớp phòng thủ bổ sung.

2.11.2. Ranh giới mạng, secret và dữ liệu nhạy cảm

Trong mô hình triển khai thử nghiệm, ranh giới mạng được thiết lập theo kiến trúc phân tầng: các cổng dịch vụ hạ tầng (PostgreSQL, Redis và Bitcoin RPC) chỉ được ánh xạ trên địa chỉ nội bộ loopback (127.0.0.1) của máy chủ vật lý nhằm phục vụ công tác quản trị và gỡ lỗi tại chỗ, trong khi các container dịch vụ giao tiếp với nhau thông qua mạng cầu nối Docker nội bộ (bridge network); máy chủ Nginx đóng vai trò là cổng vào duy nhất tiếp nhận lưu lượng từ bên ngoài. Về quản lý thông tin bí mật, khóa riêng phát hành (WIF) được gắn kết (mount) an toàn từ tệp lưu trữ có chế độ phân quyền nghiêm ngặt (0600) dành riêng cho thành phần ký số, tuyệt đối không truyền trực tiếp qua biến môi trường dạng bản rõ; các tham số nhạy cảm khác như mật khẩu cơ sở dữ liệu, khóa bí mật JWT và thông tin xác thực RPC được nạp từ cấu hình runtime và loại trừ hoàn toàn khỏi kho mã nguồn Git. Các thư mục và tệp tạm phát sinh trong quá trình xác minh đều được giới hạn quyền truy cập (0600), đồng thời nhật ký xác minh chỉ lưu giữ các trường kỹ thuật tối thiểu. Ngoài ra, phân hệ tải tài nguyên an toàn (safe-fetch) chủ động chặn toàn bộ dải địa chỉ IP nội bộ (loopback, private, link-local và cloud metadata), thẩm tra lại địa chỉ đích sau mỗi bước chuyển hướng (redirect), cũng như áp đặt ngưỡng kích thước phản hồi, số lần chuyển hướng tối đa và thời gian chờ (timeout) nhằm ngăn chặn hiệu quả tấn công giả mạo yêu cầu phía máy chủ (SSRF) [28].

2.11.3. Bảo vệ trạng thái, outbox và phục hồi

Để bảo vệ tính nhất quán của trạng thái khi xảy ra truy cập đồng thời, thao tác phê duyệt sử dụng cơ chế khóa bi quan (pessimistic lock) bên trong giao dịch cơ sở dữ liệu, trong khi thao tác từ chối áp dụng câu lệnh cập nhật có điều kiện theo trạng thái pending_approval. Trạng thái của các phiếu yêu cầu, bản ghi lô phát hành (issuance_batches) và bản ghi sự kiện hàng đợi (issuance_outbox) bắt buộc phải được xác nhận (commit) nguyên tử trong cùng một giao dịch PostgreSQL. Bộ điều phối (Dispatcher) chỉ đẩy công việc vào hàng đợi Redis sau khi giao dịch cơ sở dữ liệu đã hoàn tất thành công và hỗ trợ cơ chế thử lại tự động nếu kết nối hàng đợi gặp sự cố tạm thời. Ở chiều xử lý nền, tiến trình Worker kết hợp mã công việc (jobId) cố định, khóa tư vấn (advisory lock), điểm kiểm soát trạng thái và dữ liệu giao dịch thô đã lưu để ngăn chặn việc tạo ra giao dịch neo trùng lặp. Tương tự, quy trình thu hồi chứng thư cũng bắt buộc ghi nhận điểm kiểm soát bền vững vào cơ sở dữ liệu trước khi phát sóng giao dịch lên mạng Bitcoin.

2.11.4. Audit và giới hạn tính toàn vẹn

Quá trình ghi nhận nhật ký kiểm toán (Audit Log) được tuần tự hóa bằng cơ chế khóa tư vấn cấp giao dịch (advisory transaction lock) của PostgreSQL nhằm tránh xung đột nhánh băm khi có nhiều thao tác đồng thời. Mỗi bản ghi nhật ký lưu giữ giá trị băm của bản ghi liền trước (previousHash) và giá trị băm hiện tại (entryHash) được tính toán trên chuỗi JSON đã chuẩn hóa thứ tự khóa (canonical JSON). Điểm cuối kiểm tra tính toàn vẹn (/api/audit/integrity) cho phép duyệt và tính toán lại toàn bộ chuỗi băm để phát hiện kịp thời các can thiệp sửa đổi hoặc xóa bỏ bản ghi ở giữa chuỗi. Trong thiết kế của nguyên mẫu, để tránh làm gián đoạn luồng nghiệp vụ chính khi phát sinh lỗi ghi nhật ký, hệ thống ghi nhận cảnh báo AUDIT_WRITE_FAILED vào luồng nhật ký hệ thống; tuy nhiên, khi triển khai trong môi trường thực tế, hệ thống cần kết hợp kho lưu trữ chỉ cho phép ghi nối tiếp (append-only) hoặc đẩy nhật ký sang phân hệ giám sát độc lập, đồng thời định kỳ neo giá trị băm đầu chuỗi (head hash) ra bên ngoài.

2.11.5. Giới hạn của môi trường pilot

Nguyên mẫu hiện tại vận hành qua giao thức HTTP bên trong mạng thử nghiệm cô lập và sử dụng mạng Bitcoin regtest. Các biện pháp an toàn cấp độ sản phẩm thương mại như mã hóa đường truyền HTTPS/HSTS, xác thực đa nhân tố (MFA), hệ thống quản lý bí mật tập trung (Secret Manager), mô-đun bảo mật phần cứng (HSM/KMS), cơ chế luân chuyển khóa tự động, quy trình sao lưu và diễn tập phục hồi định kỳ, kiến trúc sẵn sàng cao (HA), tường lửa ứng dụng Web (WAF) cũng như hoạt động kiểm thử xâm nhập độc lập đều nằm ngoài phạm vi thực nghiệm của đồ án. Đây là những điều kiện kỹ thuật bắt buộc phải bổ sung trước khi đưa hệ thống vào vận hành thực tế. Ngoài ra, điểm cuối /health hiện chỉ phản ánh trạng thái sống (liveness) của tiến trình backend; việc đánh giá mức độ sẵn sàng tổng thể (readiness) cần kết hợp kịch bản kiểm tra kết nối tới PostgreSQL, Redis, Bitcoin Core, Nginx và bộ công cụ Blockcerts.

2.12. Tiêu chí chấp nhận và ma trận truy vết

2.12.1. Tiêu chí chấp nhận

Để làm cơ sở đánh giá ở giai đoạn thực nghiệm, các tiêu chí chấp nhận hệ thống (từ AC-01 đến AC-13) được xác lập tại Bảng 2.23.

Bảng 2.23. Tiêu chí chấp nhận

Mã	Tiêu chí	Bằng chứng/điểm kiểm tra
AC-01	Chứng thư gốc trả VALID	Functional E2E và full verifier
AC-02	Sửa nội dung, proof hoặc txid không còn VALID	Các ca tamper trong kịch bản xác minh
AC-03	Maker tự duyệt và Student/Maker thu hồi bị 403	Functional E2E/RBAC
AC-04	Maker B không đọc phiếu/audit của Maker A; Student không đọc chứng thư Student khác	Functional E2E object isolation
AC-05	Duyệt từng phần cùng requestBatchId tạo issuanceBatchId khác nhau	Functional E2E và migration tách batch
AC-06	Lô 10, 100, 500 phát hành đủ và mỗi lô dùng một anchor	Benchmark và artifact audit
AC-07	Lỗi/khởi động lại sau broadcast không tạo giao dịch thứ hai	Recovery E2E và checkpoint txid
AC-08	Checker thu hồi; Verify trả REVOKED; retry dùng giao dịch cũ	Kịch bản thu hồi và unit test idempotency

Mã	Tiêu chí	Bằng chứng/điểm kiểm tra
AC-09	Revocation List và audit phản ánh trạng thái; integrity hợp lệ	API và backend test
AC-10	Lỗi RPC/hạ tầng được phân loại INDETERMINATE, proof/txid hỏng là INVALID	16 ca kiểm thử verifier
AC-11	Safe-fetch chặn URL/IP/redirect nguy hiểm	10 ca regression SSRF trong verifier suite
AC-12	Build và bộ kiểm thử hiện hành đạt	Backend 12/12 suite, 49/49 test; verifier 16/16 tại lần kiểm tra 30/09/2026
AC-13	Bộ bằng chứng công bố có manifest/checksum và cây nguồn sạch	final-revalidation-20260930, sourceDirty=false

2.12.2. Ma trận FR–use case–thành phần–API–dữ liệu–kiểm thử

Sự liên kết xuyên suốt từ yêu cầu chức năng (FR), ca sử dụng (UC), thành phần API, bảng dữ liệu đến tiêu chí chấp nhận được tổng hợp trong ma trận truy vết tại Bảng 2.24.

Bảng 2.24. Ma trận truy vết yêu cầu–thiết kế–kiểm thử

FR	Use case	Thành phần/API chính	Dữ liệu chính	Tiêu chí
FR-01	UC-01	AuthController, JwtStrategy, RolesGuard; /auth/login	users, audit_logs	AC-03, AC-12
FR-02	UC-08	Admin/Auth service; /admin/users, /auth/register	users, audit_logs	AC-12
FR-03	UC-02	IssuanceService; /issue/students/:studentCode	users	AC-04, AC-12
FR-04	UC-02	/issue/request	issued_certificates	AC-03, AC-12
FR-05	UC-02	/issue/batch/request	requestBatchId	AC-05, AC-06
FR-06	UC-03	approve/reject endpoints, transaction/conditional update	issued_certificates, issuance_batches, outbox	AC-03, AC-05

FR	Use case	Thành phần/API chính	Dữ liệu chính	Tiêu chí
FR-07	UC-04	Dispatcher, BullMQ, worker, cert-tools, cert-issuer	issuance_batches, issuance_outbox, JSON runtime	AC-06, AC-07
FR-08	UC-03/08	GET /issue, /issue/:id, /audit	requestedBy, audit_logs	AC-04
FR-09	UC-05	/issue/holder/certificates	studentId, JSON runtime	AC-04
FR-10	UC-06	/verify, verifier service, anchor adapter	verification_logs	AC-01, AC-02, AC-10, AC-11
FR-11	UC-07	/revoke/:id, reconciler, /revoke/check	checkpoint thu hồi, revocationTxid	AC-03, AC-07, AC-08
FR-12	UC-08	IssuerController	issued_certificates revoked	AC-08, AC-09
FR-13	UC-08	AuditService, /audit/integrity	previousHash, entryHash	AC-09, AC-12
FR-14	UC-08	/health và script health/readiness	Không lưu thêm	AC-12, AC-13

2.13. Kết luận chương 2

Chương 2 đã cụ thể hóa các cơ sở lý thuyết ở Chương 1 thành bản thiết kế kiến trúc và nghiệp vụ toàn diện cho hệ thống: xác định rõ quyền hạn của bốn nhóm tác nhân (Maker, Checker, Student và Người xác minh); tổ chức ba cổng giao diện Web tách biệt; áp dụng cơ chế kiểm soát chéo Maker–Checker kết hợp phân quyền theo vai trò (RBAC) và cô lập dữ liệu theo đối tượng; phân tách định danh nhóm yêu cầu (requestBatchId) với định danh đợt phát hành (issuanceBatchId); thiết kế mẫu Transactional Outbox, điểm kiểm soát trạng thái (checkpoint) và cơ chế bảo đảm tính lũy đẳng cho cả hai luồng phát hành và thu hồi; xây dựng quy trình xác minh bốn trạng thái tích hợp bộ lọc chống tấn công SSRF (safe-fetch); đồng thời thiết kế lược đồ sáu bảng dữ liệu ngoài chuỗi cùng chuỗi băm nhật ký kiểm toán (Audit Hash Chain). Hệ thống sơ đồ và bảng biểu đặc tả trong chương đã mô hình hóa đầy đủ kiến trúc thành phần, luồng tuần tự phát hành theo lô, máy trạng thái kép, quy trình xác minh công khai, mô hình dữ liệu vật lý và cấu trúc điều hướng giao diện.

Xuyên suốt bản thiết kế, nguyên tắc tối thiểu hóa dữ liệu trên chuỗi được tuân thủ nghiêm ngặt: hệ thống chỉ neo giá trị gốc Merkle lên mạng Bitcoin regtest, không sử dụng hợp đồng thông minh và phân định rõ ranh giới giữa tệp chứng thư kỹ thuật Blockcerts với văn bằng số có giá trị pháp lý. Chương cũng đã nhận diện khách quan các giới hạn của mô hình nguyên mẫu về cơ chế bảo vệ khóa ký, tính nguyên tử của nhật ký kiểm toán, nguồn dữ liệu danh sách thu hồi, yêu cầu bảo vệ dữ liệu cá nhân và phạm vi môi trường thử nghiệm. Toàn bộ các đặc tả thiết kế và tiêu chí chấp nhận xác lập tại Chương 2 là cơ sở trực tiếp để triển khai xây dựng, kiểm thử tự động và đánh giá hiệu năng hệ thống tại Chương 3.

CHƯƠNG 3. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

3.1. Môi trường và mô hình triển khai thực nghiệm

3.1.1. Môi trường phần cứng và thông số phòng thí nghiệm

Môi trường thực nghiệm được triển khai trên máy chủ chạy hệ điều hành Ubuntu Server 22.04.5 LTS, trang bị bộ vi xử lý Intel Xeon Gold 6130 (4 vCPU, xung nhịp 2,10 GHz) và 19 GiB bộ nhớ RAM. Mạng Bitcoin được vận hành ở chế độ kiểm thử hồi quy cục bộ (regtest) nhằm chủ động điều khiển quá trình sinh khối và kiểm thử toàn diện các kịch bản nghiệp vụ mà không phát sinh chi phí giao dịch thực tế [9]. Toàn bộ phép đo trong chương này được thực hiện trên phiên bản mã nguồn ổn định (Release v1.0) với cây thư mục sạch và bộ tệp bằng chứng có thể kiểm chứng độc lập.

Nội dung chương này trình bày tuần tự quy trình thiết lập hệ thống từ máy chủ Ubuntu ban đầu đến khi vận hành đầy đủ trên giao diện Web, tiếp theo là các kịch bản thực nghiệm về phát hành, xác minh, thu hồi, phục hồi sau sự cố và đánh giá hiệu năng xử lý theo lô. Danh mục các thành phần phần mềm cùng phiên bản và vai trò tương ứng trong môi trường thực nghiệm được tổng hợp tại Bảng 3.1.

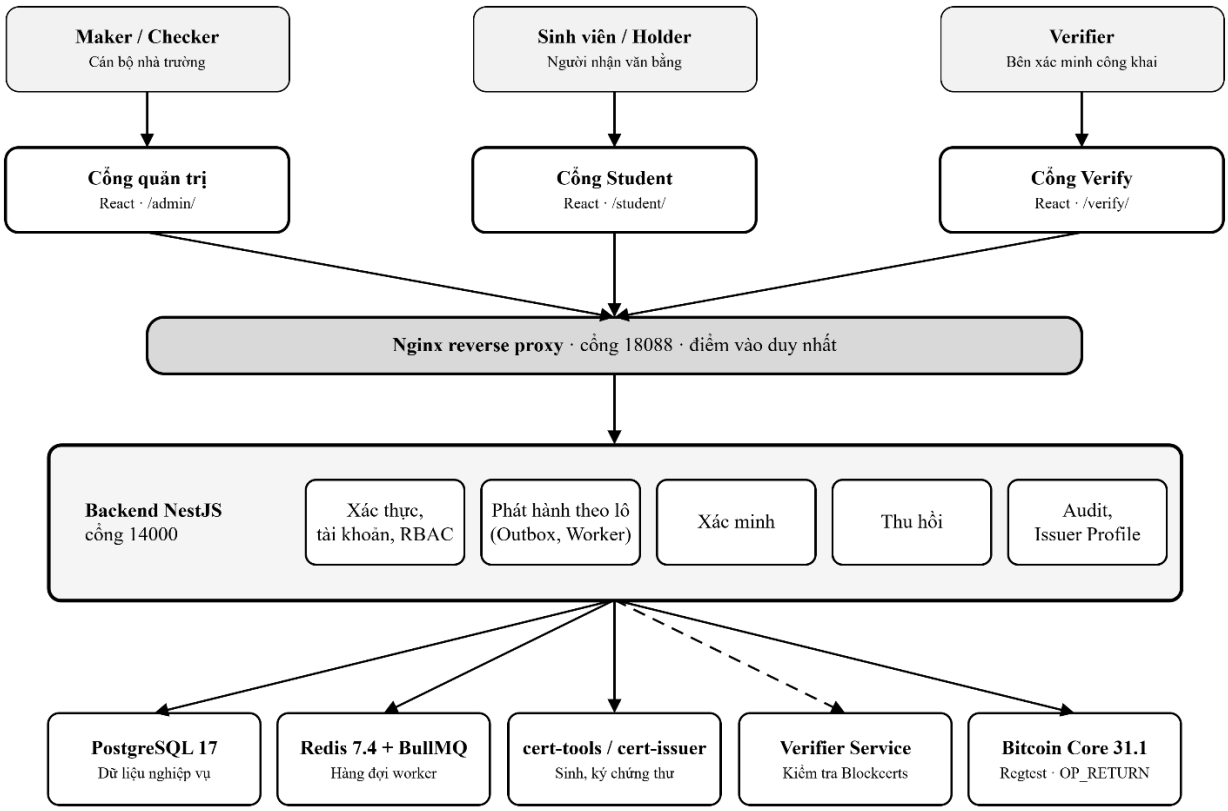
Bảng 3.1. Thành phần phần mềm của môi trường thực nghiệm

Thành phần	Phiên bản/dòng phiên bản	Vai trò
Node.js	22 trở lên (kịch bản triển khai tự động không cố định phiên bản vá)	Runtime backend và build frontend
NestJS	11	REST API, JWT, RBAC và nghiệp vụ
PostgreSQL	17-bookworm	Dữ liệu nghiệp vụ, trạng thái và log
Redis	7.4-bookworm	Nền lưu trữ cho BullMQ
BullMQ	6 (^6.3.1)	Hàng đợi phát hành và retry
Bitcoin Core	31.1	Sổ cái regtest và RPC
cert-tools	3.0.2	Sinh chứng thư unsigned từ roster
cert-issuer	3.13.1	Tạo Merkle proof và neo Bitcoin
React / Vite	19 / 8 (^19.2.8 / ^8.2.2)	Bộ thư viện xây dựng và đóng gói ba cổng giao diện Web

Thành phần	Phiên bản/dòng phiên bản	Vai trò
Nginx	Bản hệ thống Ubuntu	Reverse proxy và base path

3.1.2. Kiến trúc triển khai hệ thống

Kiến trúc triển khai tổng thể của hệ thống trong môi trường thực nghiệm được minh họa trên Hình 3.1. Ở tầng giao diện, hệ thống tách thành ba ứng dụng Web độc lập theo nhóm tác nhân: cổng nghiệp vụ dùng chung cho người lập phiếu (Maker) và người phê duyệt (Checker) tại đường dẫn /admin/ (trên sơ đồ ký hiệu là khối Cổng quản trị tương ứng với thư mục mã nguồn frontend-admin, không phải một vai trò quản trị viên độc lập), cổng người học (Cổng Student) tại đường dẫn /student/ và cổng xác minh công khai (Cổng Verify) tại đường dẫn /verify/. Toàn bộ lưu lượng từ ba cổng giao diện đi qua máy chủ Nginx reverse proxy (cổng 18088 trong stack kiểm thử tái lập) trước khi chuyển tiếp tới dịch vụ trung tâm Backend NestJS (cổng 14000), nơi tập trung xử lý xác thực, phân quyền theo vai trò (RBAC), phát hành theo lô (Outbox, Worker), xác minh, thu hồi và nhật ký kiểm toán (Audit, Issuer Profile). Ở tầng hạ tầng bên dưới, hệ quản trị cơ sở dữ liệu PostgreSQL 17 lưu trữ toàn bộ trạng thái nghiệp vụ, Redis 7.4 kết hợp với BullMQ điều phối hàng đợi tác vụ phát hành bất đồng bộ, bộ công cụ cert-tools và cert-issuer đảm nhiệm việc khởi tạo và ký chứng thư, dịch vụ Verifier Service thực hiện kiểm tra chứng thư Blockcerts, và nút Bitcoin Core 31.1 vận hành sổ cái regtest để ghi nhận giao dịch neo qua trường OP_RETURN. Việc tách biệt này bảo đảm mọi quy tắc kiểm soát truy cập đều được cưỡng chế chặt chẽ tại tầng dịch vụ phía máy chủ thay vì phụ thuộc vào trạng thái hiển thị trên giao diện người dùng.



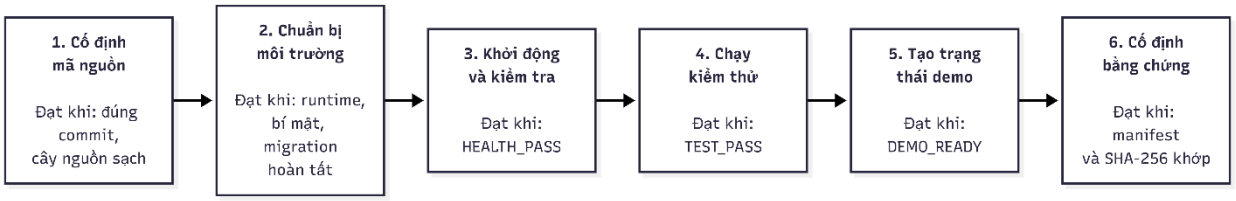
Hình 3.1. Kiến trúc triển khai của hệ thống trong môi trường thực nghiệm

3.2. Quy trình cài đặt và triển khai hệ thống trên môi trường LAB

Quy trình triển khai và tái lập thực nghiệm được thiết kế theo cơ chế cổng kiểm soát trạng thái tuần tự (minh họa trên Hình 3.2), bao gồm sáu giai đoạn liên tiếp: (1) cố định mã nguồn tại đúng mã commit quy định với cây thư mục sạch; (2) chuẩn bị môi trường thực thi, sinh cấu hình bí mật và áp dụng các bản di trú cơ sở dữ liệu (migration); (3) khởi động dịch vụ và kiểm tra tính sẵn sàng cho đến khi đạt trạng thái HEALTH_PASS; (4) thực thi bộ kiểm thử tự động và đạt trạng thái TEST_PASS; (5) làm sạch dữ liệu tạm để đưa hệ thống về trạng thái trình diễn DEMO_READY; và (6) đóng băng bộ bằng chứng thực nghiệm kèm mã băm đối chiếu SHA-256. Hệ thống chỉ chuyển sang bước kế tiếp khi bước hiện tại thỏa mãn đầy đủ tiêu chí chấp nhận; danh sách lệnh tra cứu nhanh được tổng hợp tại Phụ lục 3.

Trong các mục tiếp theo, từng bước triển khai được trình bày thống nhất gồm ba phần: mục tiêu thực hiện, câu lệnh thực thi và kết quả ghi nhận để đối chiếu. Ký hiệu $ ở đầu dòng biểu thị dấu nhắc của cửa sổ dòng lệnh (terminal), người vận hành không nhập ký tự này khi gõ lệnh. Toàn bộ mật khẩu cơ sở dữ liệu, khóa bí mật JWT, thông tin xác thực giao diện RPC của Bitcoin và khóa riêng phát hành định

dạng WIF đều được sinh tự động trong thư mục thời gian chạy (runtime) và tuyệt đối không đưa vào nội dung báo cáo.



Hình 3.2. Sơ đồ quy trình triển khai hệ thống trên môi trường phòng thí nghiệm

3.2.1. Chuẩn bị môi trường hệ điều hành và các gói phụ thuộc

Máy chủ phòng thí nghiệm cần có Git, Node.js/npm, Docker Engine, Docker Compose và Nginx. Trước khi dựng stack, cần kiểm tra Docker daemon, CPU, RAM, dung lượng đĩa và các cổng dự kiến. Nếu Docker chưa hoạt động hoặc cổng đã bị sử dụng, phải dừng quy trình để xử lý; không tự ý đổi cổng vì Issuer Profile, CORS và các URL công khai phụ thuộc vào cấu hình thống nhất.

Bước 1 – Cập nhật Ubuntu và cài các công cụ nền

Thực hiện các câu lệnh dưới đây:

```
$ sudo apt update
$ sudo apt install -y git curl ca-certificates gnupg nginx python3 python3-pip
openssl jq
```

Kết quả ghi nhận khi thực thi thành công:

```
Các gói được cài đặt hoặc xác nhận đang ở phiên bản sẵn có trong Ubuntu.
Lệnh kết thúc với mã 0; không xuất hiện lỗi "command not found".
```

Bước 2 – Cài Docker Engine và Docker Compose plugin

Thực hiện lần lượt các câu lệnh sau:

```
$ sudo install -m 0755 -d /etc/apt/keyrings
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
$ echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo ${VERSION_CODENAME}) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
$ sudo apt update
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
$ sudo usermod -aG docker "$USER"
```

Sau khi thêm tài khoản vào nhóm docker, đăng xuất/đăng nhập lại phiên shell (hoặc mở terminal mới) để quyền nhóm có hiệu lực. Sau đó kiểm tra:

```
$ docker --version
$ docker compose version
$ docker info >/dev/null && echo DOCKER_OK
```

Kết quả ghi nhận khi thực thi thành công:

```
Docker version ...
Docker Compose version v2... hoặc mới hơn
DOCKER_OK
```

Bước 3 – Cài Node.js và kiểm tra các công cụ

Bộ kịch bản triển khai yêu cầu Node.js/npm khả dụng nhưng không khóa cứng bản runtime. Trên Ubuntu 22.04 có thể dùng Node.js 22 LTS để tái lập; nếu dùng phiên bản tương thích khác, phải ghi rõ kết quả node --version trong bộ bằng chứng.

```
$ curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
$ sudo apt install -y nodejs
$ node --version
$ npm --version
$ nginx -v
$ python3 --version
```

Kết quả ghi nhận khi thực thi thành công:

```
v22.x.x hoặc bản tương thích mới hơn đã được ghi nhận
npm tương thích với bản Node.js đang sử dụng
nginx version: nginx/...
Python 3.x.x
```

Bước 4 – Kiểm tra tài nguyên máy

Tiến hành chạy các lệnh dưới đây để ghi nhận tài nguyên trước khi thực nghiệm:

```
$ uname -a
$ nproc
$ free -h
$ df -h /
```

Kết quả của lượt tái kiểm tra chính thức:

```
Máy thực nghiệm chính thức: Ubuntu Server 22.04.5 LTS.
CPU khả dụng: 4 vCPU.
RAM khả dụng xấp xỉ 19 GiB.
Phân vùng hệ thống đủ dung lượng cho ảnh dựng, vùng lưu trữ và tệp kết quả.
```

3.2.2. Thiết lập cấu trúc thư mục và tải mã nguồn hệ thống

Repository phải được checkout đúng commit dùng cho lượt tái kiểm tra. Sau checkout, git status --short không được hiển thị thay đổi trong mã nguồn nghiệp vụ. Script source-manifest.py tạo dấu vân tay trên tập tệp được kiểm soát; các tệp kết quả sinh sau đó không thuộc fingerprint của mã nguồn.

Bước 1 – Tải mã nguồn repository về máy chủ LAB

Thực hiện câu lệnh:

```
$ cd ~
$ git clone https://github.com/ebkk24/DATN-AT190226.git
$ cd DATN-AT190226
$ git fetch --all --tags
```

Kết quả ghi nhận khi thực thi thành công:

```
Repository DATN-AT190226 được tạo trên máy và có thể truy cập các thư mục
backend, frontend-* và experiments/repro.
```

Bước 2 – Checkout đúng mốc mã nguồn dùng cho lượt tái kiểm tra

Thực hiện câu lệnh:

```
$ git checkout main  # hoặc git checkout tags/v1.0.0 (nếu đã tạo release tag)
$ git rev-parse HEAD
$ git status --short
```

Kết quả ghi nhận khi thực thi thành công:

```
Mã nguồn ở trạng thái phát hành (nhánh main hoặc release tag v1.0.0)
(git status --short không in thêm dòng nào, xác nhận mã nguồn ở trạng thái
sạch)
```

Nếu lệnh git status --short hiển thị tệp bị chỉnh sửa hoặc tệp lạ chưa được theo dõi trong phạm vi kiểm soát, lượt chạy đó không được sử dụng làm bằng chứng thực nghiệm chính thức; người vận hành phải làm sạch cây mã nguồn hoặc khởi tạo lại dấu vân tay mã nguồn (fingerprint) và ghi nhận rõ trong hồ sơ bằng chứng.

3.2.3. Khởi tạo cấu hình runtime và cơ chế bảo vệ bí mật

Tệp cấu hình mẫu được sao chép thành cấu hình runtime riêng. Mật khẩu PostgreSQL/Redis, JWT secret, thông tin xác thực Bitcoin RPC và WIF phát hành không được đưa vào Git hoặc ảnh chụp báo cáo. Tệp runtime chỉ cho phép quyền đọc cần thiết; WIF chỉ được cấp cho thành phần có nhiệm vụ ký. Issuer Profile chỉ công bố khóa công khai và endpoint cần thiết. Các nhóm cấu hình được tổng hợp tại Bảng 3.2.

Bước 1 – Kiểm tra không gian triển khai thực nghiệm và quyền thực thi script

Thực hiện câu lệnh:

```
$ chmod +x experiments/repro/scripts/*.sh
$ test ! -e experiments/repro/runtime/.env && echo RUNTIME_CLEAN || echo
RUNTIME_EXISTS
$ grep -n "datn-blockcerts-repro" experiments/repro/scripts/prepare.sh | head
```

Kết quả ghi nhận khi thực thi thành công:

```
RUNTIME_CLEAN (khi thiết lập môi trường mới).
Các dòng cấu hình chỉ tới namespace datn-blockcerts-repro.
```

Bước 2 – Không tự ghi bí mật vào mã nguồn

prepare.sh tự sinh mật khẩu PostgreSQL/Redis, thông tin xác thực Bitcoin RPC và JWT secret bằng OpenSSL; khóa phát hành regtest cũng được tạo trong runtime. Người thực hiện không cần ghi bí mật thật vào .env.example. Sau bước prepare, chỉ kiểm tra quyền tệp và không sao chép giá trị bí mật vào báo cáo. Phụ lục 2 trình bày khung cấu hình và mức nhạy cảm của các nhóm biến môi trường; danh mục các nhóm biến cấu hình thời gian chạy và nguyên tắc bảo vệ tương ứng được tổng hợp tại Bảng 3.2.

Bảng 3.2. Nhóm cấu hình runtime và nguyên tắc bảo vệ

Nhóm	Ví dụ	Yêu cầu
Cơ sở dữ liệu	POSTGRES_PASSWORD, DATABASE_URL	Sinh riêng cho môi trường; không công bố
Redis	REDIS_PASSWORD, REDIS_URL	Không ghi vào tệp kết quả công khai
JWT	JWT_SECRET, JWT_EXPIRES_IN	Secret sinh ngẫu nhiên; thời hạn token cấu hình riêng; không công bố secret
Bitcoin RPC	BITCOIN_RPC_USER/PASSWORD	Chỉ dùng trong mạng nội bộ
Blockcerts	Issuer URL, revocation URL, WIF	URL nhất quán; WIF tuyệt đối bí mật
Frontend	API base URL	Không chứa credential

3.2.4. Khởi chạy cơ sở dữ liệu, bộ nhớ đệm và nút Bitcoin regtest

Bộ experiments/repro tạo một stack tách biệt để không phụ thuộc trạng thái của môi trường phát triển trước đó. PostgreSQL, Redis và Bitcoin Core phải đạt health check trước khi backend được khởi chạy. Đây là readiness ở cấp triển khai, khác với /health của backend, vốn chỉ phản ánh liveness của tiến trình ứng dụng.

Bước 1 – Khởi tạo toàn bộ các dịch vụ hệ thống trên môi trường LAB

Trên máy chủ phòng thí nghiệm, tiến hành chạy lệnh sau. Tùy CPU, tốc độ mạng và cache image, bước này có thể mất vài phút:

```
$ cd ~/DATN-AT190226
$ bash experiments/repro/scripts/prepare.sh --reset
```

Kết quả ghi nhận khi thực thi thành công:

```
[1/8] Cài phụ thuộc, kiểm thử biên dịch và build từ mã nguồn hiện có
[2/8] Tạo cây dự án runtime tách biệt
[3/8] Khởi động PostgreSQL, Redis và Bitcoin regtest của stack riêng
[4/8] Tạo khóa phát hành dùng riêng cho regtest và cấp UTXO
[5/8] Sinh cấu hình Blockcerts cho URL và địa chỉ của stack kiểm thử
[6/8] Chạy 10 migration trên cơ sở dữ liệu kiểm thử
[7/8] Tạo cấu hình Nginx cổng 18088
[8/8] Kiểm tra phạm vi cách ly
PREPARE_PASS stack=datn-blockcerts-repro api=14000 web=18088 key_mode=600
```

Dòng PREPARE_PASS là cổng kiểm soát của bước chuẩn bị. Nếu script dừng trước đó, không chạy start.sh; cần xử lý nguyên nhân rồi thực hiện lại prepare.sh.

Bước 2 – Kiểm tra quyền của tệp bí mật

Thực hiện câu lệnh:

```
$ stat -c '%a %n' experiments/repro/runtime/.env
$ stat -c '%a %n'
experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Kết quả ghi nhận khi thực thi thành công:

```
600 experiments/repro/runtime/.env
600 experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Trong lượt tái kiểm tra, health.sh kiểm tra backend, Nginx, ba cổng Web, Issuer Profile và số migration trong PostgreSQL. Điều kiện đạt gồm ba container healthy, các endpoint trả kết quả thành công và đủ 10 migration được áp dụng.

3.2.5. Khởi tạo lược đồ cơ sở dữ liệu và biên dịch ứng dụng

Backend chạy với synchronize=false; schema được quản lý bằng migration [23]. Phiên bản hiện tại có 10 migration, trong đó migration cuối tách requestBatchId và issuanceBatchId. Sau migration, backend và ba frontend được build độc lập. Build thành công chỉ chứng minh mã có thể biên dịch và đóng gói; kiểm thử nghiệp vụ được thực hiện ở bước tiếp theo.

Bước 1 – Xác nhận ba container hạ tầng đang healthy

Thực hiện câu lệnh:

```
$ docker ps --filter 'name=datn-repro' \
  --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
$ for c in datn-repro-postgres datn-repro-redis \
  datn-repro-bitcoin-core; do
    echo -n "$c: "
    docker inspect -f '{{.State.Health.Status}}' "$c"
  done
```

Kết quả ghi nhận khi thực thi thành công:

```
datn-repro-postgres: healthy
datn-repro-redis: healthy
datn-repro-bitcoin-core: healthy
```

Bước 2 – Kiểm tra đủ 10 migration

Nạp biến môi trường runtime vào shell hiện tại rồi truy vấn bảng migration:

```
$ set -a
$ source experiments/repro/runtime/.env
$ set +a
$ docker exec \
  -e PGPASSWORD="$POSTGRES_PASSWORD" \
  datn-repro-postgres \
  psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -Atc 'select count(*) from typeorm_migrations'
```

Kết quả ghi nhận khi thực thi thành công:

```
10
```

Bước 3 – Kiểm tra sản phẩm build của ba frontend

Thực hiện câu lệnh:

```
$ test -f frontend-admin/dist/index.html && echo ADMIN_BUILD_OK
$ test -f frontend-client/dist/index.html && echo STUDENT_BUILD_OK
$ test -f frontend-verify/dist/index.html && echo VERIFY_BUILD_OK
$ test -d backend/dist && echo BACKEND_BUILD_OK
```

Kết quả ghi nhận khi thực thi thành công:

```
ADMIN_BUILD_OK
STUDENT_BUILD_OK
VERIFY_BUILD_OK
BACKEND_BUILD_OK
```

3.2.6. Cấu hình cặp khóa phát hành và mạng Blockcerts

Trước khi cho phép thao tác trên Web, môi trường phát hành phải chứng minh sự phối hợp đúng giữa Issuer Profile, khóa phát hành, Bitcoin RPC và cert-tools/cert-issuer. Preflight dùng dữ liệu giả lập để tạo chứng thư mẫu, neo Merkle root và kiểm tra receipt. Chỉ khi proof, chain=bitcoinRegtest và OP_RETURN khớp, môi trường mới được coi là sẵn sàng cho demo nghiệp vụ.

Bước 1 – Kiểm tra địa chỉ phát hành và Issuer Profile đã được sinh

Thực hiện câu lệnh:

```
$ grep '^ISSUING_ADDRESS=' experiments/repro/runtime/.env
$ cat experiments/repro/runtime/project/blockcerts/issuer/
issuing-address.txt
$ python3 -m json.tool \
  experiments/repro/runtime/project/blockcerts/issuer/profile.json \
  | head -n 40
```

Kết quả ghi nhận khi thực thi thành công:

```
ISSUING_ADDRESS=<địa_chỉ_Bitcoin_regtest>
<địa_chỉ_Bitcoin_regtest>
Issuer Profile chứa URL http://127.0.0.1:18088/api/blockcerts/issuers/kma/...
```

Bước 2 – Không hiển thị WIF; chỉ kiểm tra quyền truy cập

Thực hiện câu lệnh:

```
$ stat -c '%a %n'
experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Kết quả ghi nhận khi thực thi thành công:

```
600 experiments/repro/runtime/project/storage/credentials/pk_issuer.txt
```

Không chạy cat đối với pk_issuer.txt trong ảnh chụp hoặc phiên trình bày, vì đây là WIF phát hành của môi trường regtest. Chỉ địa chỉ công khai và Issuer Profile được phép xuất hiện trong bằng chứng.

Khóa riêng không được đưa vào profile hoặc frontend. Regtest cho phép chủ động tạo khối và tái lập kết quả, nhưng không phản ánh phí, mempool hoặc độ trễ của Bitcoin mainnet [9].

3.2.7. Triển khai dịch vụ backend và ba cổng giao diện Web

Sau khi hoàn tất biên dịch, máy chủ Nginx đảm nhiệm vai trò proxy ngược (reverse proxy), phục vụ ba đường dẫn gốc (base path) cho cổng nghiệp vụ (/admin/), cổng người học (/student/) và cổng xác minh công khai (/verify/), đồng thời chuyển tiếp các yêu cầu API tới dịch vụ backend. Cổng nghiệp vụ được dùng chung cho hai vai trò Maker và Checker; cổng người học chỉ hiển thị các chứng thư thuộc quyền sở hữu của sinh viên đang đăng nhập; trong khi cổng xác minh cho phép truy cập công khai không cần đăng nhập. Trước khi chạy bộ kiểm thử tích hợp đầu-cuối (E2E), toàn bộ điểm cuối giao diện Web được kiểm tra đồng thời với đường dẫn /health và hồ sơ đơn vị cấp (Issuer Profile). Các tiêu chí đánh giá hoàn thành giai đoạn triển khai được tổng hợp tại Bảng 3.3.

Bước 1 – Khởi động dịch vụ backend và Nginx reverse proxy

Thực hiện câu lệnh:

```
$ bash experiments/repro/scripts/start.sh
```

Kết quả ghi nhận khi thực thi thành công:

```
START_PASS api=14000 web=18088 fault_after_broadcast=0
```

Bước 2 – Chạy readiness/dependency check

Thực hiện câu lệnh:

```
$ bash experiments/repro/scripts/health.sh
```

Kết quả ghi nhận khi thực thi thành công:

```
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200 verify=200
```

Bước 3 – Kiểm tra trực tiếp các cổng web bằng HTTP

Thực hiện câu lệnh:

```
$ curl -I http://127.0.0.1:18088/admin/
$ curl -I http://127.0.0.1:18088/student/
$ curl -I http://127.0.0.1:18088/verify/
$ curl -fsS http://127.0.0.1:18088/api/blockcerts/issuers/kma/profile.json |
head
```

Kết quả ghi nhận khi thực thi thành công:

```
Ba yêu cầu HEAD trả HTTP 200.
Issuer Profile trả JSON và không yêu cầu đăng nhập.
```

Tổng hợp các hạng mục kiểm tra trạng thái dịch vụ và điều kiện xác nhận hoàn thành giai đoạn triển khai được trình bày tại Bảng 3.3.

Bảng 3.3. Điều kiện hoàn thành giai đoạn triển khai

Hạng mục	Điều kiện đạt	Ý nghĩa
/health	HTTP 200, backend đang chạy	Liveness của ứng dụng
PostgreSQL/Redis/Bitcoin	Container healthy	Phụ thuộc hạ tầng sẵn sàng
Migration	10/10 đã áp dụng	Schema khớp mã nguồn
Ba cổng Web (/admin/, /student/, /verify/)	Truy cập được qua Nginx	Cổng nghiệp vụ, cổng người học và cổng xác minh được phục vụ đúng base path
Issuer Profile	Truy cập công khai trong môi trường demo	Verifier có dữ liệu issuer

3.2.8. Khởi tạo tài khoản người dùng và kiểm tra phân quyền RBAC

Sau khi hệ thống đạt điều kiện sẵn sàng, tài khoản thử nghiệm được khởi tạo để thực hiện quy trình trên Web. Script bootstrap tạo Checker trực tiếp trong cơ sở

dữ liệu kiểm thử, sau đó dùng API được bảo vệ để tạo Maker, Student và Student thứ hai phục vụ kiểm tra isolation.

Thực hiện câu lệnh dưới đây:

```
$ node experiments/repro/scripts/prepare-accounts.mjs
```

Kết quả ghi nhận khi thực thi thành công:

```
{"ok":true,"roles":4,"privateMode":"600"}
```

Kiểm tra quyền tệp chứa tài khoản thử nghiệm:

```
$ stat -c '%a %n' experiments/repro/runtime/private/accounts.json
```

Kết quả mong đợi:

```
600 experiments/repro/runtime/private/accounts.json
```

Khi cần đăng nhập trên giao diện, người vận hành có thể đọc accounts.json ngay trên máy cục bộ. Tuy nhiên không chụp hoặc sao chép mật khẩu từ tệp này vào báo cáo, Git hoặc tệp kết quả công khai. Lưu ý: khi chạy run-tests.sh ở bước kế tiếp, script tự dừng/khởi động lại ứng dụng và chạy prepare-accounts.mjs thêm một lần; vì vậy accounts.json có thể được thay bằng bộ tài khoản mới. Trước khi demo web phải thực hiện bước reset demo ở Mục 3.5 rồi đọc lại tệp này.

3.2.9. Kiểm tra tính sẵn sàng và các tiêu chí an toàn trước vận hành

run-tests.sh lần lượt thực hiện unit/integration test backend, test verifier, bootstrap tài khoản, Functional E2E và Recovery E2E. Đây là cổng kiểm soát trước khi sử dụng số liệu benchmark. Script tự gọi start.sh, health.sh và prepare-accounts.mjs; không cần bootstrap tài khoản thủ công lần thứ hai trong cùng lượt.

Thực hiện câu lệnh:

```
$ bash experiments/repro/scripts/run-tests.sh
```

Dòng kết thúc của lượt chạy đạt:

```
TEST_PASS default=PASS benchmark=SKIPPED
```

Sau đó đọc tệp tổng hợp của lượt chạy:

```
$ python3 -m json.tool experiments/repro/runtime/results/summary.json
```

Kết quả của lượt tái kiểm tra chính thức:

```
{
  "default": "PASS",
  "functional": true,
  "recovery": true,
  "backend_suites": 12,
  "backend_tests": 49,
  "ssrf_tests": 10,
  "verification_status_tests": 6,
  "verifier_tests": 16
}
```

Nếu một test thất bại thì không bỏ qua bằng cờ cưỡng bức. Cần đọc log trong experiments/repro/runtime/logs/, xử lý nguyên nhân và chạy lại từ bước phù hợp.

3.2.10. Thiết lập công cụ đo lường và thu thập dữ liệu thực nghiệm

Benchmark cần nhiều thời gian hơn nên chỉ chạy sau khi kiểm thử mặc định đạt. Runner phát hành 10, 100 và 500 chứng thư, lặp mỗi kích thước ba lần, sau đó kiểm toán tệp kết quả và thực hiện xác minh đầy đủ.

Thực hiện câu lệnh:

```
$ bash experiments/repro/scripts/run-tests.sh --benchmark
```

Dòng kết thúc của lượt benchmark đạt:

```
TEST_PASS default=PASS benchmark=PASS
```

Đọc kết quả tổng hợp và kiểm toán tệp kết quả:

```
$ python3 -m json.tool experiments/repro/runtime/results/summary.json
$ python3 -m json.tool experiments/repro/runtime/results/benchmark-artifact-
audit.json | head -n 80
```

Kết quả của bộ bằng chứng chính thức:

```
summary.json (các trường cần đối chiếu):
{
  "benchmark": "PASS",
  "benchmark_runs": 9,
  "benchmark_certificates": 1830,
  "full_verifier_valid": 1830
}
benchmark-artifact-audit.json (kết quả tổng hợp):
{
  "passed": true,
  "fullVerifierValid": 1830
}
```

Các số liệu thời gian, thông lượng và tài nguyên được phân tích riêng ở Mục 3.9. Không dùng kết quả từ một working tree khác để thay cho bộ bằng chứng của cây nguồn đã chốt.

3.2.11. Quản trị vòng đời dịch vụ và bảo trì môi trường hệ thống

Sau khi hoàn tất thực nghiệm, chọn mức dừng phù hợp. Không chạy docker system prune vì lệnh này có thể ảnh hưởng đến các stack khác trên máy.

Chỉ dừng backend/Nginx nhưng giữ container và dữ liệu:

```
$ bash experiments/repro/scripts/stop.sh --app-only
```

Dừng cả container nhưng giữ volume:

```
$ bash experiments/repro/scripts/stop.sh --down
```

Quy trình giải phóng tài nguyên và dọn dẹp môi trường thực nghiệm:

```
$ bash experiments/repro/scripts/stop.sh --purge
```

Khi thực hiện dọn dẹp, chỉ các tài nguyên trong phạm vi môi trường lab bị giải phóng, bảo đảm an toàn và tính độc lập của các dịch vụ khác; stack chính tại cổng 4000/8088 không thuộc phạm vi kịch bản tái lập.

3.3. Hiện thực backend, dữ liệu và hàng đợi

3.3.1. Các mô-đun nghiệp vụ

Dịch vụ backend được tổ chức thành các mô-đun nghiệp vụ độc lập gồm: xác thực (Auth), quản lý người dùng (Admin/Users), phát hành (Issuance), xác minh (Verification), thu hồi (Revocation), nhật ký kiểm toán (Audit) và hồ sơ đơn vị cấp (Issuer Profile). Trên các điểm cuối API cần bảo vệ, lớp chắn xác thực (AuthGuard) luôn được kích hoạt trước lớp chắn phân quyền (RolesGuard) nhằm bảo đảm thông tin định danh người dùng trong mã thông báo JWT (req.user) được giải mã hoàn chỉnh trước khi đối chiếu vai trò. Hệ thống không thiết lập vai trò quản trị viên (Admin) độc lập; hai vai trò Maker và Checker sử dụng chung cổng nghiệp vụ nhưng bị phân tách quyền hạn nghiêm ngặt tại tầng API: Checker nắm quyền khởi tạo tài khoản, phê duyệt phát hành và thu hồi chứng thư; Maker chỉ có quyền lập phiếu yêu cầu; Student chỉ được phép truy xuất các chứng thư thuộc về chính mình; còn người xác minh (Verifier) là tác nhân công khai không cần đăng nhập.

3.3.2. Mô hình dữ liệu và liên kết người học

Trong mô hình dữ liệu của nguyên mẫu, bảng issued_certificates đóng vai trò lưu trữ thống nhất cả thông tin phiếu yêu cầu ban đầu lẫn kết quả chứng thư sau khi phát hành. Quyền sở hữu chứng thư của người học được định danh duy nhất thông qua khóa ngoại studentId kết hợp với mã sinh viên (studentCode); trường họ tên người nhận (recipientName) chỉ đóng vai trò bản chụp dữ liệu (snapshot) phục vụ hiển thị trên văn bằng, giúp hệ thống phân biệt chính xác các sinh viên trùng họ tên. Để hỗ trợ nghiệp vụ xét duyệt linh hoạt, hệ thống tách bạch mã lô yêu cầu (requestBatchId) dùng để nhóm các phiếu do Maker khởi tạo trong cùng một đợt với mã lô phát hành (issuanceBatchId) chỉ được sinh ra tại thời điểm Checker thực hiện phê duyệt. Nhờ cơ chế này, một lô yêu cầu ban đầu có thể được tách thành nhiều lô phát hành độc lập khi Checker thực hiện phê duyệt từng phần. Vai trò và các thuộc

tính cốt lõi của sáu bảng dữ liệu tham gia trực tiếp vào luồng phát hành được tổng hợp tại Bảng 3.4.

Bảng 3.4. Các thực thể dữ liệu trực tiếp tham gia luồng phát hành

Bảng	Vai trò trong luồng	Điểm quan trọng
users	Tài khoản, role, Student profile	studentCode duy nhất; không trả passwordHash
issued_certificates	Phiếu + tệp chứng thư sau phát hành	studentId, requestBatchId, issuanceBatchId, txid, trạng thái, checkpoint thu hồi
issuance_batches	Trạng thái một lần phát hành	jobId ổn định, giao dịch thô, anchorTxid, attemptCount
issuance_outbox	Giao hàng job DB → Redis/BullMQ	pending/dispatched; retry nếu enqueue lỗi
verification_logs	Kết quả xác minh	Không lưu PII thô trong lượt E2E cuối
audit_logs	Actor/action/target	Chuỗi previousHash/entryHash phát hiện sửa cục bộ

3.3.3. Outbox giao dịch và phục hồi phát hành

Khi Checker phê duyệt một hoặc nhiều phiếu yêu cầu, việc cập nhật trạng thái các bản ghi chứng thư cùng việc khởi tạo bản ghi lô phát hành (IssuanceBatch) và bản ghi hàng đợi trung gian (IssuanceOutbox) được thực thi nguyên tử trong cùng một giao dịch cơ sở dữ liệu PostgreSQL. Chỉ sau khi giao dịch cơ sở dữ liệu đã xác nhận (commit) thành công, bộ điều phối nền (Dispatcher) mới đọc bản ghi từ bảng outbox để đẩy công việc vào hàng đợi BullMQ. Trong trường hợp Redis hoặc BullMQ gặp sự cố tạm thời khiến thao tác đẩy hàng đợi thất bại, bản ghi outbox vẫn giữ trạng thái chờ (pending) để bộ điều phối tự động gửi lại ở chu kỳ tiếp theo. Mỗi công việc phát hành sử dụng một mã định danh ổn định (jobId = issuance-<batchId>), giới hạn tối đa 5 lần thử lại (attempts = 5) với thời gian chờ tăng theo hàm mũ (exponential backoff), qua đó ngăn chặn việc sinh trùng lặp các tác vụ phát hành tương đương.

Tiến trình Worker thực thi phát hành duy trì điểm kiểm soát trạng thái (checkpoint) theo từng lô. Trước khi phát sóng giao dịch lên mạng Bitcoin regtest, dữ liệu giao dịch thô đã ký (rawTransaction) cùng mã giao dịch dự kiến (anchorTxid)

và giá trị gốc Merkle (merkleRoot) được ghi bền vững vào PostgreSQL. Nếu tiến trình Worker gặp sự cố dừng đột ngột ngay sau khi nút Bitcoin đã tiếp nhận giao dịch, cơ chế đối soát và phục hồi sẽ truy xuất lại đúng giao dịch đã lưu tại điểm kiểm soát thay vì ký và phát sóng một giao dịch mới. Kết quả kiểm thử phục hồi đầu-cuối (Recovery E2E) ngày 30/09/2026 xác nhận mã giao dịch (txid) được giữ nguyên hoàn toàn qua các lần thử lại và chiều cao khối chỉ tăng đúng một đơn vị.

3.3.4. Audit và dữ liệu xác minh

Mô-đun kiểm toán ghi nhận đầy đủ định danh tác nhân (actor), vai trò (actorRole), hành động (action) và đối tượng tác động (targetId) cho toàn bộ bước nghiệp vụ trọng yếu. Cơ chế liên kết chuỗi băm SHA-256 giữa bản ghi hiện tại (entryHash) và bản ghi liền trước (previousHash) giúp phát hiện kịp thời các hành vi chỉnh sửa hoặc xóa trái phép bản ghi ở giữa chuỗi. Tuy nhiên, cần nhìn nhận khách quan rằng chuỗi băm này chỉ mang tính chất phát hiện can thiệp (tamper-evident) trong phạm vi cơ sở dữ liệu chứ không tạo ra tính bất biến tuyệt đối: người quản trị cơ sở dữ liệu có đặc quyền cao vẫn có thể tính toán lại toàn bộ chuỗi băm hoặc cắt bỏ các bản ghi ở cuối chuỗi nếu giá trị băm đỉnh chuỗi (headHash) chưa được neo định kỳ sang một kho lưu trữ chỉ-ghi (append-only) độc lập. Ngoài ra, một số bản ghi nhật ký được ghi sau khi giao dịch nghiệp vụ đã hoàn tất để không làm gián đoạn luồng xử lý chính, do đó đây là giới hạn thiết kế cần được nêu rõ của nguyên mẫu.

Bên cạnh nhật ký kiểm toán, kịch bản kiểm thử chức năng đầu-cuối (Functional E2E) còn xác minh rằng bảng nhật ký xác minh (verification_logs) đã được lược bỏ thông tin định danh cá nhân, tuyệt đối không lưu trữ họ tên, địa chỉ thư điện tử hoặc mã sinh viên trích từ chứng thư thử nghiệm. Đồng thời, các tệp chứng thư JSON lưu trong không gian làm việc trên máy chủ được thiết lập chế độ phân quyền tệp 0600 (chỉ tài khoản dịch vụ sở hữu mới có quyền đọc/ghi). Các biện pháp kiểm soát này giúp giảm thiểu rủi ro rò rỉ dữ liệu cá nhân ngoài chuỗi khối, dù trong môi trường vận hành thực tế vẫn cần kết hợp thêm chính sách giới hạn thời gian lưu trữ (retention), mã hóa bản sao lưu và giám sát truy cập tập trung.

3.4. Hiện thực phát hành, xác minh và thu hồi Blockcerts

3.4.1. Phát hành theo lô và gom lô Merkle

Trong luồng phát hành, Maker có thể khởi tạo phiếu yêu cầu đơn lẻ hoặc tạo theo lô với quy mô tối đa 500 hồ sơ trong một lần gọi API. Dịch vụ backend thực hiện kiểm tra tính hợp lệ của dữ liệu đầu vào, đối chiếu hồ sơ người học (Student) trong cơ sở dữ liệu và khởi tạo các bản ghi ở trạng thái chờ phê duyệt (pending_approval). Khi Checker thực hiện phê duyệt, nhóm bản ghi được chọn sẽ

được cấp một mã lô phát hành (issuanceBatchId) mới và chuyển sang trạng thái xếp hàng (queued). Tiếp theo, tiến trình Worker chia danh sách hồ sơ trong lô thành các cụm (chunk) gồm 10 hồ sơ, khởi chạy song song tối đa 16 cụm để gọi công cụ cert-tools sinh các tệp chứng thư chưa ký (unsigned certificates), sau đó tập hợp toàn bộ đầu ra để gọi cert-issuer một lần duy nhất. Kết quả của quá trình này là một cây Merkle chung và đúng một giao dịch neo trên mạng Bitcoin cho toàn bộ lô phát hành.

Sau khi phát hành, mỗi chứng thư điện tử sở hữu mã định danh (certUid), giá trị băm mục tiêu (targetHash) và đường dẫn bằng chứng Merkle (Merkle proof) riêng biệt, nhưng chia sẻ chung giá trị gốc Merkle (Merkle root) và mã giao dịch Bitcoin (txid) với các chứng thư khác trong cùng lô phát hành. Chỉ duy nhất giá trị gốc Merkle được nhúng vào trường OP_RETURN của giao dịch Bitcoin; toàn bộ dữ liệu học tập và thông tin cá nhân của người học được giữ ngoài chuỗi khối. Do nhiều chứng thư trong cùng một lô cùng tham chiếu tới một giao dịch neo, trường txid trong bảng chứng thư không áp đặt ràng buộc duy nhất (unique constraint) ở cấp độ từng bản ghi văn bằng.

3.4.2. Xác minh công khai

Khi người dùng tải tệp chứng thư lên cổng xác minh công khai, giao diện gửi dữ liệu JSON tới điểm cuối POST /api/verify. Dịch vụ backend phối hợp với Verifier Service và bộ chuyển đổi neo regtest (Regtest Anchor Adapter) để kiểm tra tuần tự cấu trúc ngữ nghĩa của chứng thư, giá trị băm mục tiêu, đường dẫn bằng chứng Merkle, giao dịch neo trên mạng Bitcoin, thông tin khóa công khai trong Issuer Profile và trạng thái thu hồi hiện hành. Kết quả xác minh được phân loại tường minh thành bốn trạng thái: hợp lệ (VALID), không hợp lệ (INVALID), đã thu hồi (REVOKED) và không đủ dữ kiện kết luận (INDETERMINATE). Trạng thái INVALID chỉ được trả về khi có bằng chứng mật mã hoặc dữ liệu khẳng định nội dung chứng thư, bằng chứng Merkle hoặc giao dịch neo bị sai lệch; ngược lại, các lỗi kết nối mạng, quá thời gian chờ (timeout) hoặc thiếu dữ kiện truy vấn tạm thời đều được phân loại vào INDETERMINATE nhằm tránh đưa ra kết luận âm tính giả.

Để ngăn chặn nguy cơ tấn công giả mạo yêu cầu phía máy chủ (SSRF) thông qua các đường dẫn URL nhúng trong chứng thư đầu vào, dịch vụ xác minh sử dụng cơ chế tải tài nguyên an toàn (safeFetch): tự động chặn các địa chỉ vòng lặp nội bộ (loopback), dải IP riêng (private network), địa chỉ liên kết cục bộ (link-local) và địa chỉ dịch vụ siêu dữ liệu đám mây (metadata IP); đồng thời kiểm tra lại địa chỉ đích sau mỗi bước chuyển hướng HTTP (redirect), giới hạn thời gian chờ, dung lượng

phản hồi tối đa và thẩm định toàn bộ danh sách địa chỉ IP khi máy chủ DNS trả về nhiều bản ghi. Bộ kiểm thử hồi quy của dịch vụ xác minh gồm 10 ca kiểm thử chống SSRF và 6 ca kiểm thử phân loại trạng thái, đạt kết quả tuyệt đối 16/16 ca. Dù vậy, kết quả này chỉ chứng minh hiệu lực bảo vệ đối với các kịch bản tấn công đã định nghĩa và không thay thế cho hoạt động kiểm thử xâm nhập độc lập [28].

3.4.3. Thu hồi có điểm kiểm soát và đối soát trạng thái

Cơ chế thu hồi là phần mở rộng của nguyên mẫu so với luồng phát hành chuẩn: trong khi giao dịch phát hành chỉ neo Merkle Root của toàn bộ lô, giao dịch thu hồi do Checker phê duyệt sẽ phát sóng một giao dịch Bitcoin riêng chứa payload REVOKE:<certUid> trong đầu ra OP_RETURN. Chuỗi định danh certUid (UUID) này đóng vai trò neo bằng chứng thu hồi độc lập trên sổ cái; tuy nhiên chuỗi này là quy ước riêng của nguyên mẫu, verifier Blockcerts bên ngoài không mặc nhiên diễn giải được nếu không có module tương thích. Dưới góc độ quyền riêng tư, certUid là định danh giả danh có tính liên kết (linkable), do đó trạng thái thu hồi cần được đối soát cẩn trọng giữa chuỗi khối và cơ sở dữ liệu.

Để tránh tạo giao dịch thứ hai khi có lỗi, service dùng advisory lock để tuần tự hóa việc sử dụng UTXO. Khi bắt đầu, bản ghi chuyển sang revocation_pending. Raw transaction đã ký và revocationIntentTxid được lưu bền vững trước sendrawtransaction. Nếu timeout hoặc crash, trạng thái chuyển thành revocation_reconciliation_required; tiến trình nền chỉ tra cứu hoặc phát lại đúng raw transaction đã lưu. Khi giao dịch có ít nhất một confirmation, bản ghi chuyển sang revoked và revocationTxid được chốt bằng intent txid. Thứ tự này cần thiết vì PostgreSQL và Bitcoin không nằm trong cùng một giao dịch ACID.

3.5. Thực nghiệm quy trình nghiệp vụ trên ba cổng giao diện Web

Mục này trình bày quy trình thực nghiệm trên ba cổng Web: tạo tài khoản, Maker lập phiếu, Checker phê duyệt, Worker phát hành, Student tải chứng thư, Verify kiểm tra, sửa JSON để kiểm tra toàn vẹn và thu hồi. Hình 3.3–3.12 được chụp trong cùng một lượt demo sạch sau demo-reset.sh --quick, dùng một Student và số hiệu KMA-2026-0001. Các kết luận về an toàn, phục hồi và hiệu năng vẫn dựa trên mã nguồn và kết quả E2E ngày 30/09/2026, không chỉ dựa vào ảnh giao diện.

Bước chuẩn bị – Đưa môi trường về trạng thái demo sạch

Trước khi thao tác trên Web, môi trường được làm sạch dữ liệu cũ, tạo lại bộ tài khoản thử nghiệm và kiểm tra health. Không dùng --full cho mỗi lượt demo vì chế độ này xóa toàn bộ volume/runtime.

Thực hiện câu lệnh:

```
$ bash experiments/repro/scripts/demo-reset.sh --dry-run
$ bash experiments/repro/scripts/demo-reset.sh --quick
```

Kết quả mong đợi:

```
DEMO_RESET_DRY_RUN scope=PASS containers=PASS schema=PASS mode=quick
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200 verify=200
DEMO_READY mode=quick health=PASS accounts=READY data=CLEAN
```

Sau khi xuất hiện DEMO_READY, accounts.json đã được tạo lại và là bộ credential phải dùng cho toàn bộ phần Web bên dưới. Không dùng credential đã đọc trước khi chạy run-tests.sh hoặc benchmark.

Địa chỉ truy cập: trên máy chủ phòng thí nghiệm hoặc từ trình duyệt mạng nội bộ, dùng http://127.0.0.1:18088/admin/ cho Maker/Checker, http://127.0.0.1:18088/student/ cho sinh viên và http://127.0.0.1:18088/verify/ để xác minh công khai. Khi truy cập hệ thống chính từ máy khác, dùng http://<IP>:8088/admin/, http://<IP>:8088/student/ và http://<IP>:8088/verify/; không dùng 127.0.0.1. <IP> là 192.168.4.69 trong mạng LAN hoặc 100.105.56.19 qua Tailscale.

3.5.1. Khởi tạo tài khoản Maker, Checker và Student

Bước 1 – Xác nhận tài khoản và dữ liệu Student dùng cho demo. Script demo-reset.sh --quick đã tạo một Checker, một Maker, một Student chính và một Student thứ hai để kiểm tra isolation. Tài khoản không được đăng ký công khai; Checker là vai trò có quyền tạo tài khoản.

Kiểm tra tên đăng nhập và mã sinh viên mà không in mật khẩu ra terminal:

```
$ python3 - <<'PY'
import json
p = "experiments/repro/runtime/private/accounts.json"
a = json.load(open(p))
for role in ("checker", "maker", "student"):
    print(role, "=", a[role]["username"])
print("studentCode =", a["student"]["studentCode"])
PY
```

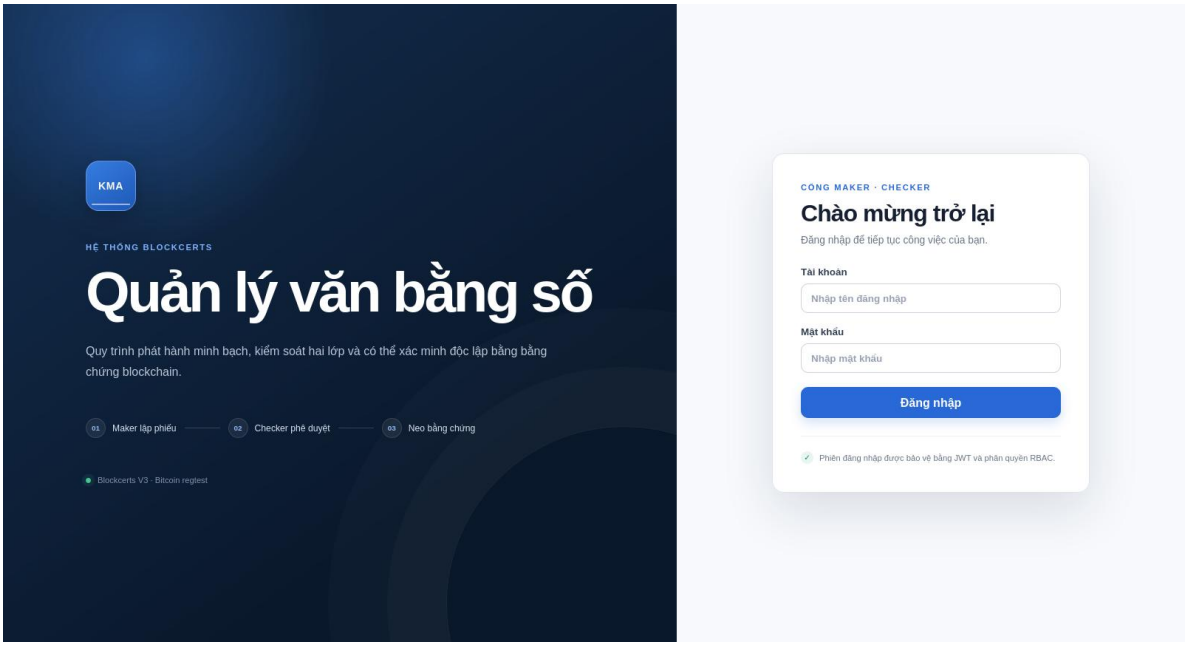
Kết quả: terminal in ra username của ba vai trò và studentCode của Student. Khi cần đăng nhập, mở accounts.json trực tiếp trên máy lab để lấy password tương ứng; không chụp mật khẩu vào báo cáo.

3.5.2. Maker đăng nhập cổng nghiệp vụ, tra cứu hồ sơ người học và lập phiếu yêu cầu

Bước 2 – Maker đăng nhập vào cổng nghiệp vụ. Người vận hành mở trình duyệt tại địa chỉ dưới đây và đăng nhập bằng thông tin tài khoản maker trong tệp accounts.json:

```
http://127.0.0.1:18088/admin/
```

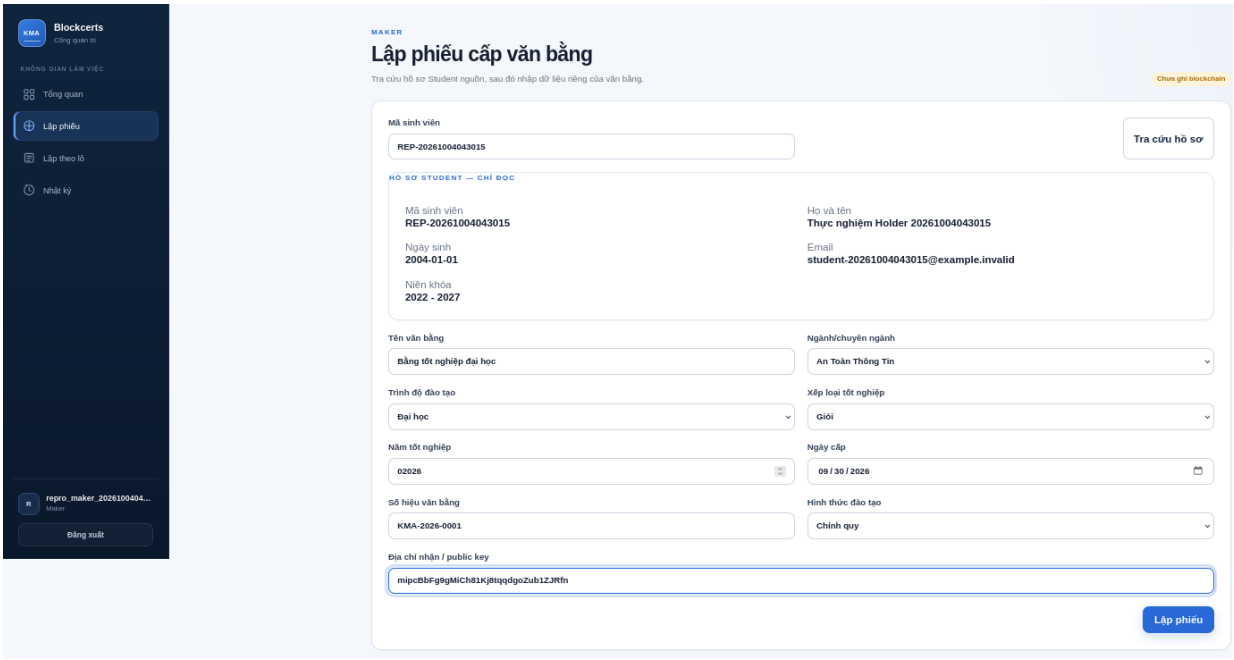
Kết quả ghi nhận: giao diện cổng nghiệp vụ (CỔNG MAKER · CHECKER) mở thành công; thanh điều hướng của tài khoản Maker chỉ hiển thị các chức năng tổng quan, lập phiếu đơn, lập theo lô và xem nhật ký trong phạm vi cá nhân, hoàn toàn không hiển thị chức năng phê duyệt, quản lý người dùng hoặc thu hồi văn bằng. Màn hình đăng nhập cổng nghiệp vụ được minh họa trên Hình 3.3.



Hình 3.3. Màn hình đăng nhập cổng nghiệp vụ dùng chung cho Maker và Checker

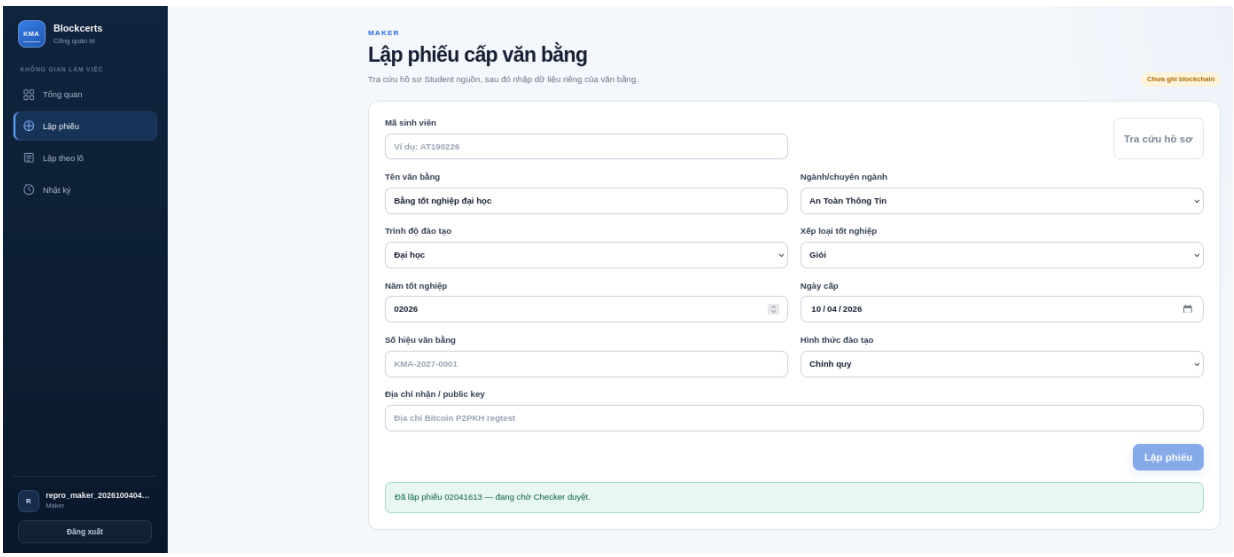
Bước 3 – Maker tra cứu hồ sơ người học và nhập thông tin lập phiếu cấp văn bằng theo Hình 3.4. Tại màn hình lập phiếu, Maker nhập mã sinh viên (studentCode) lấy ở Bước 1 vào ô tra cứu và nhấn nút Tra cứu hồ sơ để hệ thống hiển thị khung Hồ sơ Student — Chỉ đọc gồm họ tên, ngày sinh, email và niên khóa. Tiếp theo, Maker nhập các trường thông tin nghiệp vụ của văn bằng gồm: tên văn bằng (Bằng tốt nghiệp đại học), ngành/chuyên ngành (An toàn thông tin), trình độ đào tạo (Đại học), xếp loại tốt nghiệp (Giỏi), năm tốt nghiệp (2026), ngày cấp (30/09/2026), số hiệu văn bằng (KMA-2026-0001), hình thức đào tạo (Chính quy) và địa chỉ nhận khóa công khai P2PKH trên mạng regtest. Thiết kế giao diện khóa cứng vùng thông tin

cá nhân của người học, không cho phép Maker chỉnh sửa hồ sơ gốc của sinh viên trong lúc lập phiếu.



Hình 3.4. Maker đối chiếu hồ sơ Student và nhập dữ liệu văn bằng

Sau khi nhấn nút Lập phiếu, dịch vụ backend khởi tạo bản ghi yêu cầu ở trạng thái chờ phê duyệt (pending_approval). Lúc này hệ thống mới chỉ gán mã lô yêu cầu (requestBatchId), trong khi mã lô phát hành (issuanceBatchId) chưa được sinh ra cho đến khi có quyết định phê duyệt của Checker. Giao diện hiển thị thông báo tạo phiếu thành công và đang chờ Checker phê duyệt như minh họa trên Hình 3.5.



Hình 3.5. Phiếu được tạo và chờ Checker phê duyệt

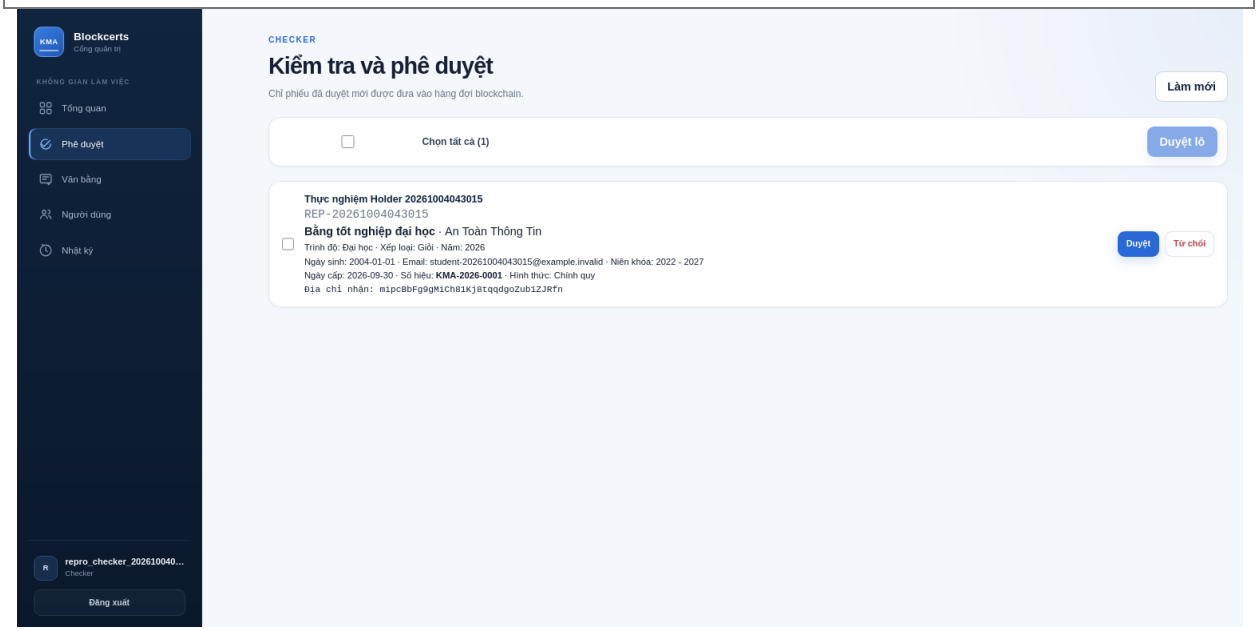
3.5.3. Checker kiểm tra, phê duyệt và Worker phát hành chứng thư

Bước 4 – Checker đăng nhập và kiểm tra phê duyệt phiếu yêu cầu (minh họa trên Hình 3.6). Người vận hành đăng xuất tài khoản Maker trên cổng nghiệp vụ và đăng nhập lại bằng tài khoản checker trong tệp accounts.json. Tại mục Phê duyệt,

Checker kiểm tra danh sách các phiếu đang chờ duyệt, đối chiếu thông tin người lập phiếu, bản chụp hồ sơ sinh viên (REP-20261004043015) cùng các thuộc tính văn bằng (KMA-2026-0001). Đối với một lô gồm nhiều phiếu yêu cầu, Checker có thể chọn duyệt toàn bộ hoặc duyệt từng phần; mỗi thao tác phê duyệt sẽ sinh ra một mã issuanceBatchId riêng biệt nhưng vẫn bảo toàn mã requestBatchId ban đầu để truy vết nguồn gốc.

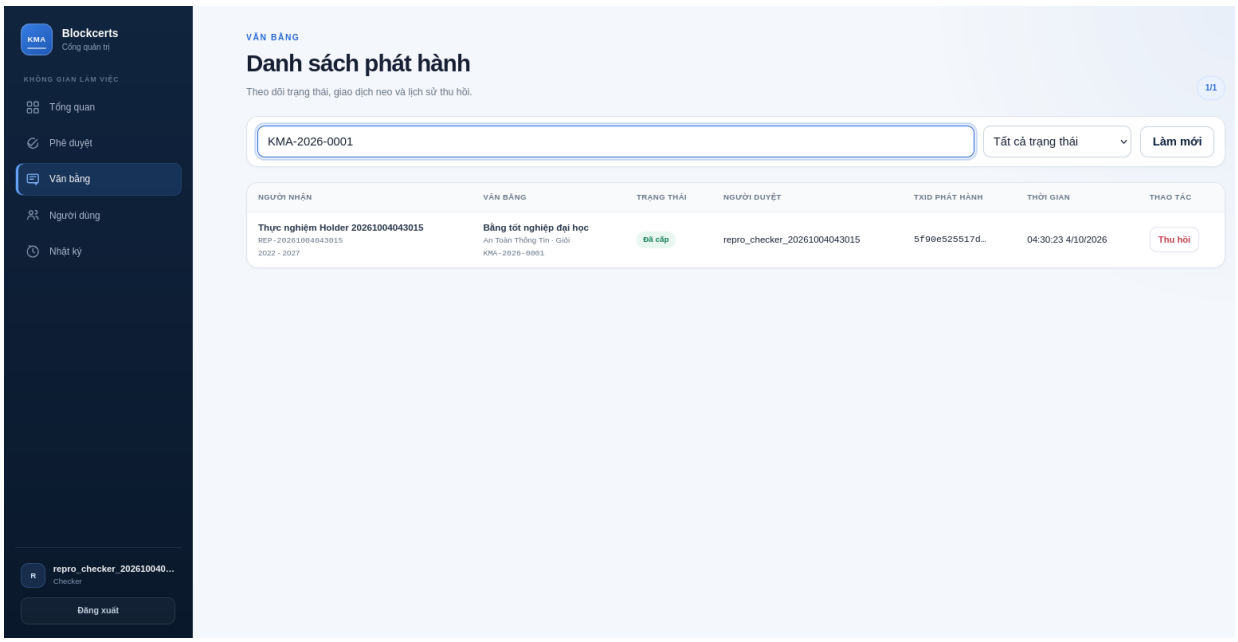
Địa chỉ sử dụng:

```
http://127.0.0.1:18088/admin/
```



Hình 3.6. Checker kiểm tra phiếu đang chờ phê duyệt

Bước 5 – Theo dõi tiến trình Worker phát hành chứng thư. Ngay sau khi Checker nhấn nút Duyệt, trạng thái bản ghi chuyển tuần tự qua queued rồi processing. Bộ điều phối và tiến trình Worker chạy nền tự động sinh tệp chứng thư JSON-LD, dựng cây Merkle, phát sóng giao dịch neo lên mạng Bitcoin regtest và cập nhật trạng thái bản ghi thành đã cấp (issued, hiển thị nhãn Đã cấp trên giao diện). Người vận hành không cần nhấn phê duyệt lặp lại khi tác vụ đang xử lý. Khi hoàn tất, màn hình Danh sách phát hành hiển thị bản ghi văn bằng ở trạng thái Đã cấp kèm theo mã người duyệt, mã giao dịch phát hành (5f90e525517d...) và nút thao tác Thu hồi như minh họa trên Hình 3.7.



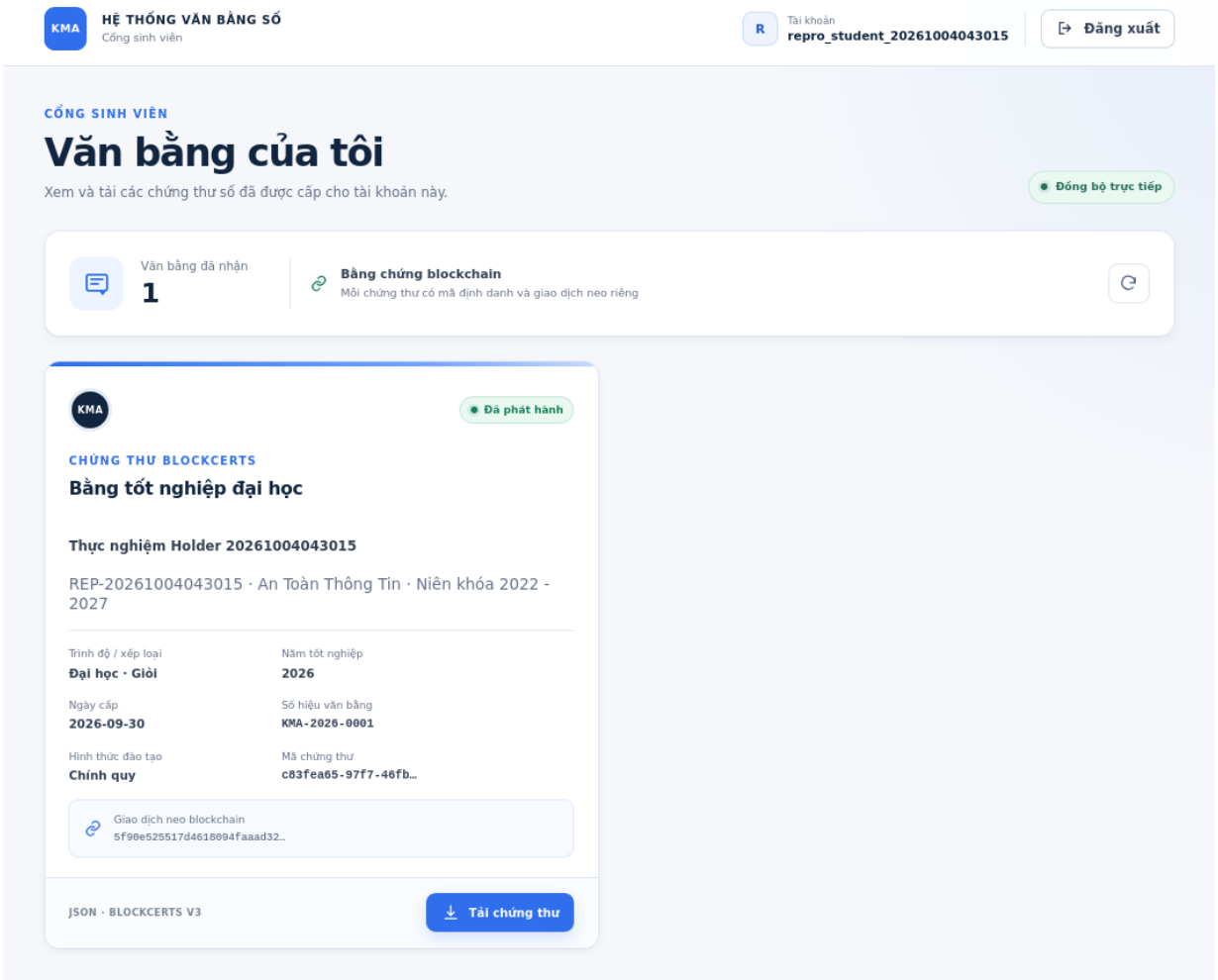
Hình 3.7. Danh sách văn bằng sau khi Worker phát hành

3.5.4. Người học tra cứu và tải chứng thư điện tử

Bước 6 – Người học đăng nhập cổng sinh viên (frontend-client) để tra cứu và tải chứng thư. Khi người học truy cập địa chỉ dưới đây và đăng nhập bằng tài khoản student, dịch vụ backend trích xuất định danh chủ thể (subject) trực tiếp từ mã thông báo JWT đã ký để lọc các chứng thư có khóa ngoại studentId khớp với tài khoản hiện tại, tuyệt đối không sử dụng tham số truy vấn do phía trình duyệt truyền lên làm căn cứ phân quyền:

```
http://127.0.0.1:18088/student/
```

Kết quả ghi nhận: người học nhìn thấy đúng chứng thư văn bằng KMA-2026-0001 vừa được phát hành (kèm mã chứng thư c83fea65-97f7-46fb... và mã giao dịch neo 5f90e525517d...) và có thể nhấn nút Tải chứng thư để lưu tệp JSON Blockcerts V3 về máy cá nhân. Trong kịch bản kiểm thử tự động Functional E2E, tài khoản sinh viên thứ hai cũng được sử dụng để xác nhận cơ chế cô lập đối tượng ngăn chặn việc xem trộm chứng thư của người học khác. Giao diện cổng sinh viên và thao tác tải chứng thư được minh họa trên Hình 3.8.



Hình 3.8. Student xem và tải chứng thư thuộc đúng tài khoản

Tệp JSON tải xuống từ cổng sinh viên là chứng thư Blockcerts V3 hoàn chỉnh dùng để cung cấp cho bên thứ ba kiểm tra trên cổng xác minh công khai. Lưu ý rằng trên giao diện Hình 3.8 có dòng mô tả tóm tắt "Mỗi chứng thư có mã định danh và giao dịch neo riêng" ứng với trường hợp phát hành đơn lẻ trong lượt trình diễn; về mặt kiến trúc tổng quát, mỗi chứng thư luôn có mã định danh (certUid) và nhánh bằng chứng Merkle riêng, nhưng các chứng thư được duyệt trong cùng một lô phát hành sẽ dùng chung một mã giao dịch neo (txid). Ngoài ra, nếu một chứng thư bị thu hồi sau khi cấp, người học vẫn nhìn thấy bản ghi đó trong lịch sử cá nhân kèm nhãn trạng thái đã thu hồi rõ ràng.

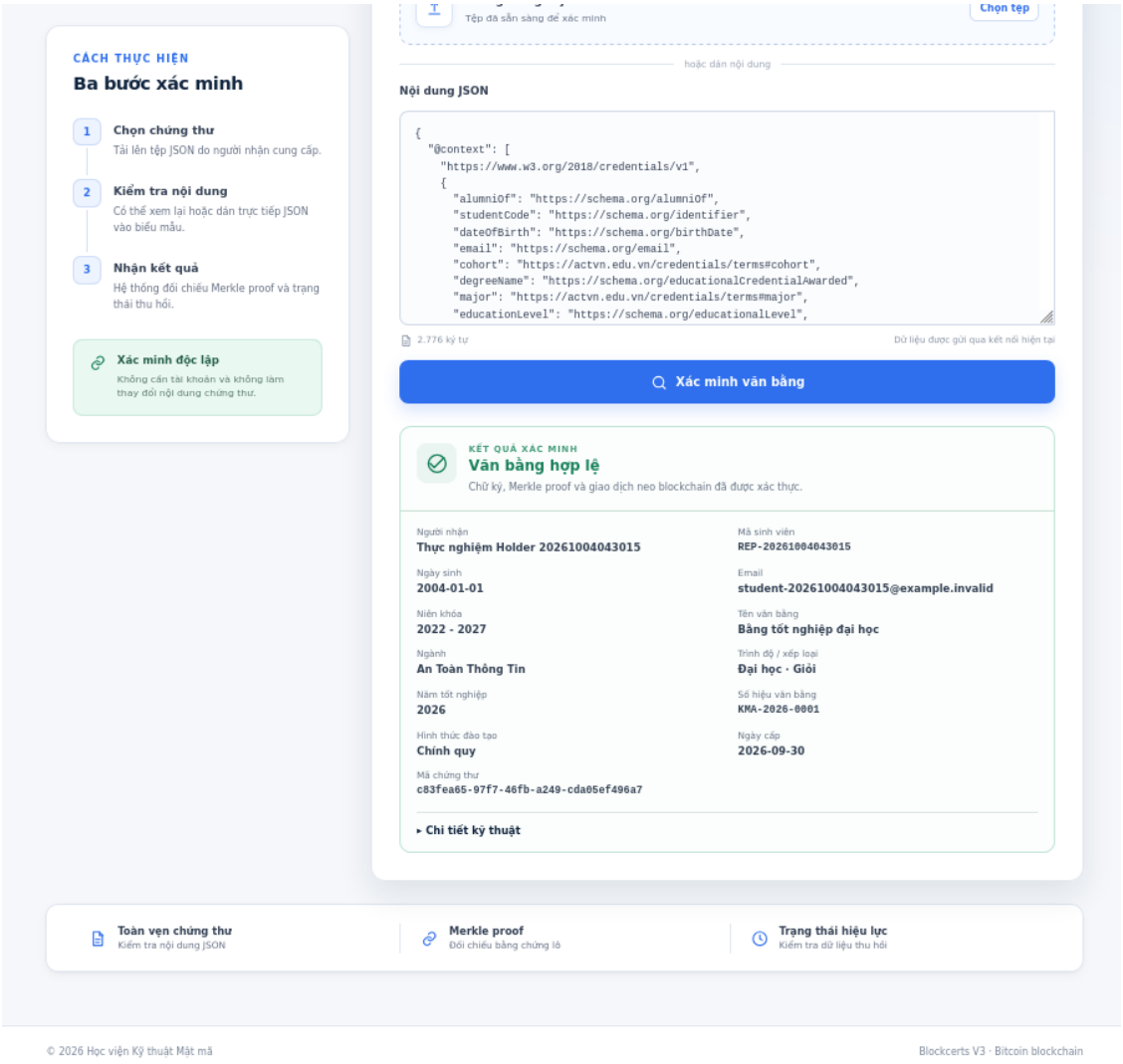
Xác minh văn bằng hợp lệ trên cổng xác minh công khai

Bước 7 – Xác minh tệp gốc. Không cần đăng nhập; mở cổng Verify, chọn tệp JSON vừa tải ở Bước 6 và thực hiện xác minh:

```
http://127.0.0.1:18088/verify/
```

Kết quả ghi nhận: hệ thống trả về trạng thái hợp lệ (VALID, hiển thị thông báo "Văn bằng hợp lệ" trên giao diện). Dịch vụ xác minh đã kiểm tra đầy đủ cấu

trúc ngữ nghĩa JSON-LD, giá trị băm mục tiêu (targetHash), đường dẫn bằng chứng Merkle, giao dịch neo trên mạng Bitcoin regtest, hồ sơ đơn vị cấp (Issuer Profile) và danh sách thu hồi. Giao diện xác nhận chứng thư hợp lệ được minh họa trên Hình 3.9.



Hình 3.9. Cổng Verify xác nhận chứng thư hợp lệ

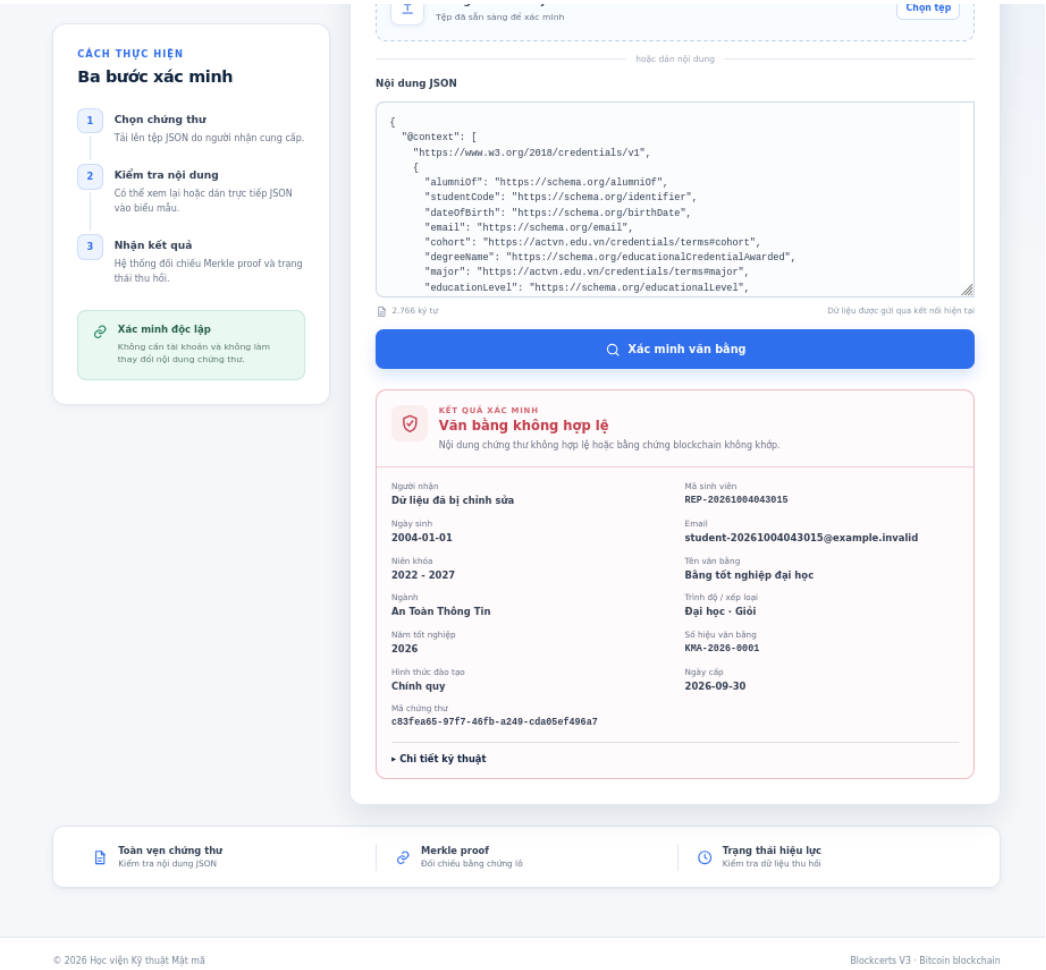
3.5.6. Thực nghiệm phát hiện văn bằng bị can thiệp, sửa đổi

Bước 8 – Tạo bản sao đã sửa để kiểm tra tính toàn vẹn. Không sửa tệp gốc. Giả sử tệp tải về có tên certificate.json trong thư mục Downloads, chạy:

```
$ cd ~/Downloads
$ cp certificate.json certificate-tampered.json
$ jq \
  '.credentialSubject.name = "Dữ liệu đã bị chỉnh sửa"' \
  certificate.json > certificate-tampered.json
```

Tiếp theo, người vận hành tải tệp certificate-tampered.json vừa bị can thiệp lên cổng xác minh công khai. Kết quả ghi nhận đúng như kỳ vọng: hệ thống trả về trạng thái không hợp lệ (INVALID, hiển thị thông báo "Văn bằng không hợp lệ" trên giao diện) do giá trị băm lại của nội dung chứng thư sau khi trường tên người

nhận bị đổi thành “Dữ liệu đã bị chỉnh sửa” không còn khớp với nhánh bằng chứng Merkle và gốc Merkle đã neo trên chuỗi khối. Giao diện phát hiện chứng thư bị sửa đổi được minh họa trên Hình 3.10.

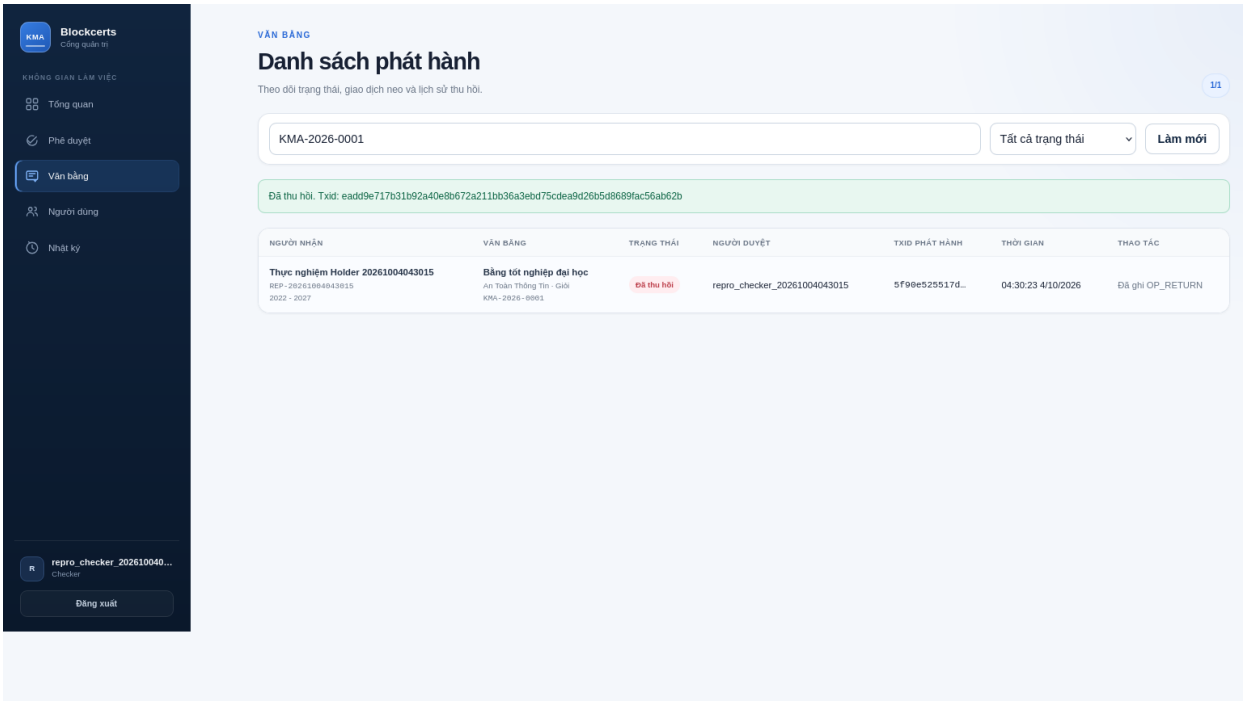


Hình 3.10. Verifier phát hiện chứng thư đã bị thay đổi

3.5.7. Checker thực hiện thu hồi và xác minh lại trạng thái văn bằng

Bước 9 – Checker thực hiện thu hồi chứng thư trên cổng nghiệp vụ. Người vận hành quay lại cổng nghiệp vụ tại địa chỉ dưới đây với tài khoản checker, tìm văn bằng KMA-2026-0001 đang ở trạng thái Đã cấp, nhấn nút Thu hồi, nhập lý do thu hồi và xác nhận thao tác. Dịch vụ backend thực hiện ghi điểm kiểm soát chứa giao dịch thu hồi thô đã ký và mã giao dịch dự kiến (revocationIntentTxid) vào PostgreSQL trước khi phát sóng giao dịch chứa tải trọng OP_RETURN (REVOKE:<certUid>) lên mạng Bitcoin regtest, sau đó xác nhận khối mới và cập nhật trạng thái bản ghi thành Đã thu hồi (kèm mã giao dịch thu hồi eadd9e717b31...) như minh họa trên Hình 3.11.

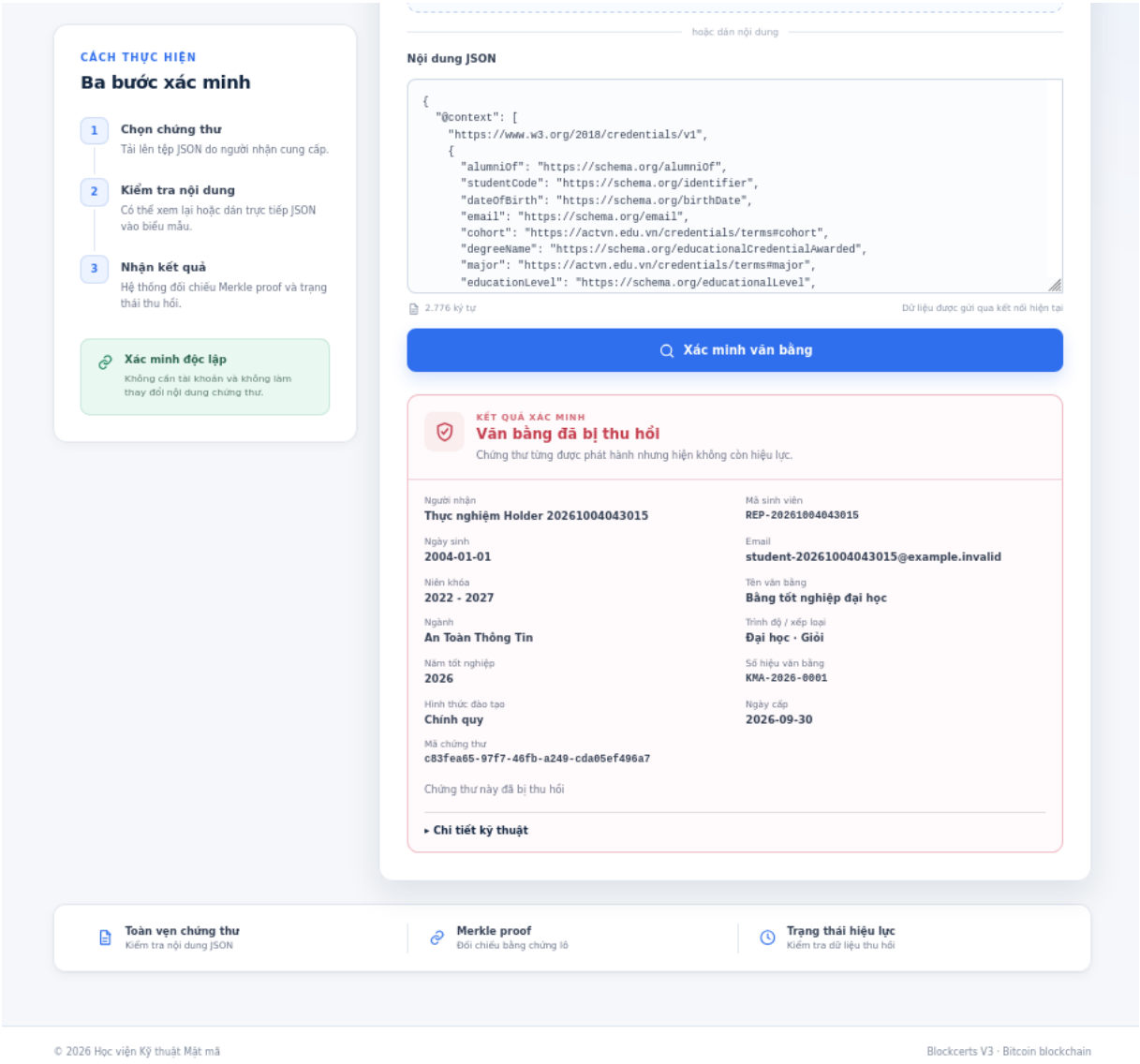
```
http://127.0.0.1:18088/admin/
```



Hình 3.11. Checker thu hồi chứng thư và hệ thống cập nhật trạng thái

Bước 10 – Xác minh lại tệp chứng thư gốc sau khi đã thu hồi theo Hình 3.12. Người vận hành mở lại cổng xác minh công khai tại địa chỉ dưới đây và tải lên đúng tệp certificate.json nguyên bản ban đầu (không dùng tệp đã chỉnh sửa ở Bước 8). Mặc dù chữ ký số, đường dẫn Merkle và giao dịch neo lúc phát hành của tệp gốc vẫn hoàn toàn toàn vẹn về mặt mật mã, quá trình đối soát trạng thái thu hồi phát hiện chứng thư đã bị thu hồi hợp lệ, do đó hệ thống trả về kết quả cuối cùng là REVOKED ("Văn bằng đã bị thu hồi"). Thực nghiệm này làm rõ sự khác biệt bản chất giữa một chứng thư bị giả mạo, sửa đổi nội dung (INVALID) và một chứng thư nguyên vẹn nhưng đã chấm dứt hiệu lực nghiệp vụ (REVOKED).

```
http://127.0.0.1:18088/verify/
```



Hình 3.12. Cổng Verify trả trạng thái REVOKED cho chứng thư gốc

3.6. Kiểm soát an toàn và khả năng phục hồi

Nhằm bảo đảm tính bền vững và an toàn cho toàn bộ quy trình vận hành, các cơ chế kiểm soát an ninh của hệ thống được kiểm chứng trực tiếp ở tầng dịch vụ backend và chuỗi kịch bản tích hợp đầu-cuối (E2E), thay vì chỉ dựa vào các ràng buộc hiển thị trên giao diện người dùng. Danh mục các rủi ro an toàn chính, cơ chế kiểm soát tương ứng và bằng chứng thực nghiệm thu được trong đợt tái kiểm tra được tổng hợp tại Bảng 3.5.

Bảng 3.5. Kiểm soát an toàn và bằng chứng kiểm thử

Rủi ro/thuộc tính	Kiểm soát hiện tại	Bằng chứng
Maker tự duyệt	Checker-only approve + RolesGuard	Maker gọi approve nhận HTTP 403

Rủi ro/thuộc tính	Kiểm soát hiện tại	Bằng chứng
IDOR giữa Maker	List/detail/audit lọc theo requestedBy	Maker B không thấy phiếu của Maker A trong danh sách, không đọc được chi tiết hoặc nhật ký liên quan
Holder đọc nhầm chứng thư	studentId/studentCode lấy từ JWT	Student thứ hai không thấy chứng thư của Student thứ nhất
SSRF qua URL trong credential	safe-fetch, kiểm tra DNS/IP/redirect/timeout	10/10 ca SSRF đạt
Queue mất job sau commit DB	outbox giao dịch + bộ điều phối nền	Outbox pending được dispatch lại
Worker lỗi quanh broadcast	raw tx/txid checkpoint + jobId ổn định	Recovery E2E giữ nguyên txid, chỉ tăng 1 block
Thu hồi lặp hoặc timeout	advisory lock + checkpoint trước broadcast + reconciliation	Retry dùng lại revocationIntentTxid
Audit bị sửa cục bộ	previousHash/entryHash	Kiểm tra chain phát hiện thay đổi giữa chuỗi
Rò PII qua verify log	redaction và assertion E2E	Log không chứa name/email/studentCode của credential thử
Lộ secret	runtime ngoài Git, quyền tệp hạn chế	Preflight + file chứng thư 0600 trong E2E

Bên cạnh các kết quả kiểm thử đạt được, hai giới hạn kỹ thuật của nguyên mẫu cần được xác định rõ. Thứ nhất, chuỗi băm nhật ký (Audit Hash Chain) đóng vai trò là cơ chế phát hiện can thiệp cục bộ trong cơ sở dữ liệu chứ không biến bảng nhật ký thành sổ cái bất biến tuyệt đối, đồng thời thao tác ghi nhật ký chưa được gói nguyên tử trong cùng giao dịch cơ sở dữ liệu với mọi thao tác nghiệp vụ. Thứ hai, các bộ kiểm thử tự động chứng minh tính đúng đắn đối với tập kịch bản đã định nghĩa nhưng không thay thế cho hoạt động kiểm thử xâm nhập độc lập, kiểm thử vận hành dài hạn, kiểm tra sao lưu – phục hồi hoặc mô hình đối phương đã chiếm quyền quản trị máy chủ vật lý.

3.7. Phương pháp kiểm thử và kịch bản thực nghiệm

3.7.1. Phân tầng các cấp độ kiểm thử

Kết quả kiểm thử được phân tầng và báo cáo theo đúng phạm vi chứng minh của từng cấp độ: kiểm thử đơn vị và dịch vụ (unit/service test) xác nhận hành vi của từng mô-đun trong điều kiện cô lập; kiểm thử chức năng đầu-cuối (Functional E2E) kiểm chứng sự phối hợp xuyên suốt giữa tầng API, cơ sở dữ liệu, hàng đợi và nút chuỗi khối; kiểm thử phục hồi (Recovery E2E) đánh giá khả năng xử lý lỗi phát sinh quanh bước phát sóng giao dịch; kịch bản đo hiệu năng (benchmark) ghi nhận thời gian xử lý theo lô trên mạng Bitcoin regtest; và bước kiểm toán toàn diện xác minh độc lập toàn bộ tệp chứng thư đã phát hành. Các cấp độ kiểm thử có bản chất khác nhau được trình bày tách biệt, không cộng gộp cơ học thành một con số tổng chung. Kết quả tổng hợp của từng lớp kiểm thử được trình bày tại Bảng 3.6.

Bảng 3.6. Kết quả kiểm thử tự động và E2E

Lớp kiểm thử	Kết quả	Điểm được chứng minh
Backend Jest	12/12 suite, 49/49 test	RBAC, DTO, state transition, outbox/checkpoint, revocation, audit, issuer/verifier
Verifier	16/16 test	10 ca SSRF; 6 ca phân loại trạng thái
Functional E2E	Đạt	Maker–Checker, partial approval, object isolation, Holder, Verify, revoke
Recovery E2E	Đạt	Phục hồi sau lỗi quanh broadcast giữ nguyên txid và không neo lặp
Build	Backend + 3 frontend đạt	Mã biên dịch và đóng gói được
Health/readiness	Đạt, đủ 10 migration	Stack phụ thuộc và endpoint sẵn sàng

3.7.2. Phương pháp thu thập và kiểm toán dữ liệu thực nghiệm

Mỗi lượt chạy đo hiệu năng và kiểm thử tích hợp đầu-cuối đều xuất ra các tệp dữ liệu kết quả chứa đầy đủ mã lô, mã giao dịch neo, gốc Merkle và giá trị băm kiểm tra. Sau bước kiểm toán độc lập, kịch bản freeze-evidence.py tiến hành đóng băng các tệp kết quả và sinh tệp đặc tả MANIFEST.json ghi nhận kích thước, mã băm SHA-256 của từng tệp cùng giá trị băm tổng hợp. Bộ hồ sơ bằng chứng công bố cuối cùng còn bao gồm tệp tổng hợp benchmark-summary.json và tệp SHA256SUMS

dùng để kiểm tra tính toàn vẹn của toàn bộ 37 tệp dữ liệu. Quy mô thực nghiệm chính thức gồm 9 lượt chạy đo hiệu năng, phát hành tổng cộng 1.830 chứng thư thông qua 9 giao dịch neo trên mạng Bitcoin regtest, sau đó toàn bộ 1.830 tệp chứng thư được đọc lại và xác minh độc lập. Các chỉ số kiểm soát cuối cùng của bộ bằng chứng được tổng hợp tại Bảng 3.7 và trình bày chi tiết tại Phụ lục 4.

Bảng 3.7. Chỉ số kiểm soát cuối của bộ bằng chứng

Kiểm soát	Kết quả
Lần chạy benchmark	9/9 đạt
Chứng thư phát hành	1.830/1.830
Giao dịch neo	9
Xác minh đầy đủ	1.830/1.830 VALID
Payload manifest	35/35 khớp
Tệp SHA-256	37/37 khớp

Kết quả thực nghiệm Kịch bản 1 – Tính toàn vẹn và kiểm soát Maker–Checker

Kịch bản thực nghiệm thứ nhất được thiết kế nhằm kiểm chứng khả năng bảo vệ tính toàn vẹn dữ liệu của chứng thư Blockcerts V3 và hiệu lực của cơ chế kiểm soát chéo Maker–Checker. Quy trình thử nghiệm bắt đầu với việc xác minh một tệp chứng thư gốc hợp lệ, sau đó lần lượt mô phỏng các hành vi sửa đổi trái phép họ tên người nhận, tên văn bằng, chuỗi chữ ký mật mã (proofValue), mã giao dịch neo (txid) và hành vi dùng tài khoản Maker gọi trực tiếp API phê duyệt. Kết quả thực nghiệm ghi nhận cả 6/6 ca kiểm thử đều đạt yêu cầu: mọi trường hợp sửa đổi nội dung hoặc bằng chứng mật mã đều bị dịch vụ xác minh phân loại chính xác về trạng thái INVALID (chỉ khi xảy ra lỗi kết nối hạ tầng hoặc thiếu dữ kiện đối chiếu bên ngoài hệ thống mới trả về trạng thái INDETERMINATE), và yêu cầu tự phê duyệt của Maker bị chặn lại với mã lỗi HTTP 403. Kết quả chi tiết của Kịch bản 1 được trình bày tại Bảng 3.8.

Bảng 3.8. Kết quả Kịch bản 1

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-01	Chứng thư gốc	VALID	VALID	Đạt
S1-02	Sửa tên người nhận	Không VALID	INVALID	Đạt

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-03	Sửa tên văn bằng	Không VALID	INVALID	Đạt
S1-04	Sửa proofValue	Không VALID	INVALID	Đạt
S1-05	Sửa txid trong proof	Không VALID	INVALID	Đạt
S1-06	Maker tự duyệt	HTTP 403	HTTP 403	Đạt

Kết quả thực nghiệm Kịch bản 2 – Hiệu năng phát hành theo lô cây Merkle

3.9.1. Kết quả đo các đợt phát hành theo lô

Mỗi kích thước 10, 100 và 500 chứng thư được chạy ba lần. Đồng hồ phía client bắt đầu trước request phê duyệt và dừng khi truy vấn trạng thái nhận thấy toàn bộ lô ở issued. Runner kiểm tra mỗi 2 giây; do đó thời gian đo gồm chi phí API và độ trễ phát hiện hoàn tất, không phải timestamp hoàn tất nội bộ chính xác của Worker. Mỗi lượt dùng certificate ID và số hiệu riêng trên dữ liệu giả lập, đồng thời tạo đúng một transaction anchor. Tải đo là số chứng thư trong lô, không phải số người dùng đồng thời. Bảng 3.9 trình bày chín lượt chạy.

Với $n = 3$ lượt chạy, gọi t_j là thời gian quan sát của lượt j và N là số chứng thư trong lô. Thời gian trung bình là tổng các t_j chia cho n. SD trong Bảng 3.10 được tính bằng căn bậc hai của trung bình bình phương độ lệch với mẫu số n, dùng để mô tả ba lượt đã công bố chứ không phải ước lượng không chệch cho tổng thể. Giây/chứng thư của mỗi lượt là t_j/N; thông lượng mỗi lượt là N/t_j. Giá trị thông lượng trong bảng là trung bình của ba giá trị N/t_j, không phải N chia thời gian trung bình. Với ba lượt và cơ chế polling, SD nhỏ chưa đủ để kết luận hiệu năng ổn định trong vận hành dài hạn.

Bảng 3.9. Chín lần chạy hiệu năng tái kiểm tra

Lô–lần	Số CT	Thời gian (s)	s/CT	CT/s	Tx
10–1	10	6,106	0,6106	1,6377	1
10–2	10	6,102	0,6102	1,6388	1
10–3	10	6,094	0,6094	1,6410	1
100–1	100	10,379	0,1038	9,6348	1

Lô–lần	Số CT	Thời gian (s)	s/CT	CT/s	Tx
100–2	100	10,368	0,1037	9,6451	1
100–3	100	10,548	0,1055	9,4805	1
500–1	500	35,943	0,0719	13,9109	1
500–2	500	36,112	0,0722	13,8458	1
500–3	500	35,798	0,0716	13,9673	1

Cả chín lần chạy đều phát hành đủ số chứng thư dự kiến và mỗi lần chỉ tạo một transaction neo.

3.9.2. Phân tích giá trị tổng hợp và thông lượng xử lý

Từ dữ liệu đo đạc chi tiết của chín lượt chạy thực nghiệm, các chỉ số tổng hợp về thời gian phát hành trung bình, độ lệch chuẩn và thông lượng xử lý theo từng kích thước lô được trình bày tại Bảng 3.10.

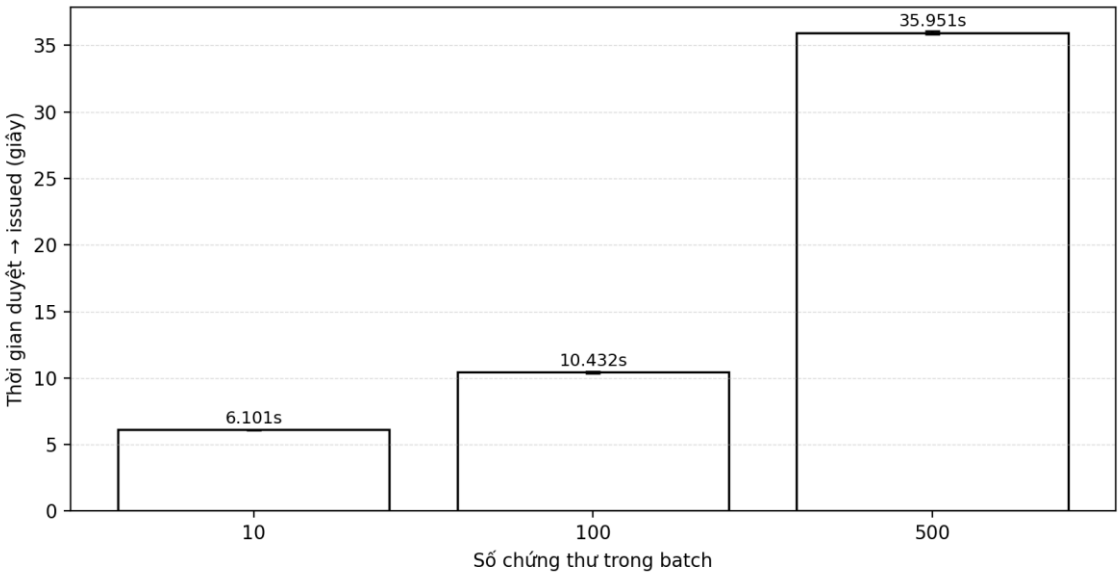
Bảng 3.10. Tổng hợp theo kích thước lô

Kích thước lô	Tổng CT (3 lần)	TB ± SD (s)	s/CT	CT/s	Tx/run
10	30	6,101 ± 0,005	0,6101	1,6392	1,00
100	300	10,432 ± 0,082	0,1043	9,5868	1,00
500	1.500	35,951 ± 0,128	0,0719	13,9080	1,00

Số liệu trong Bảng 3.10 phản ánh mức biến thiên của thời gian xử lý quan sát được và thông lượng phát hành khi quy mô lô tăng dần từ 10 lên 500 chứng thư trên cùng cấu hình máy chủ thử nghiệm; ký hiệu ± biểu diễn độ lệch chuẩn mô tả của ba lượt chạy lặp lại, không phải khoảng tin cậy suy diễn thống kê.

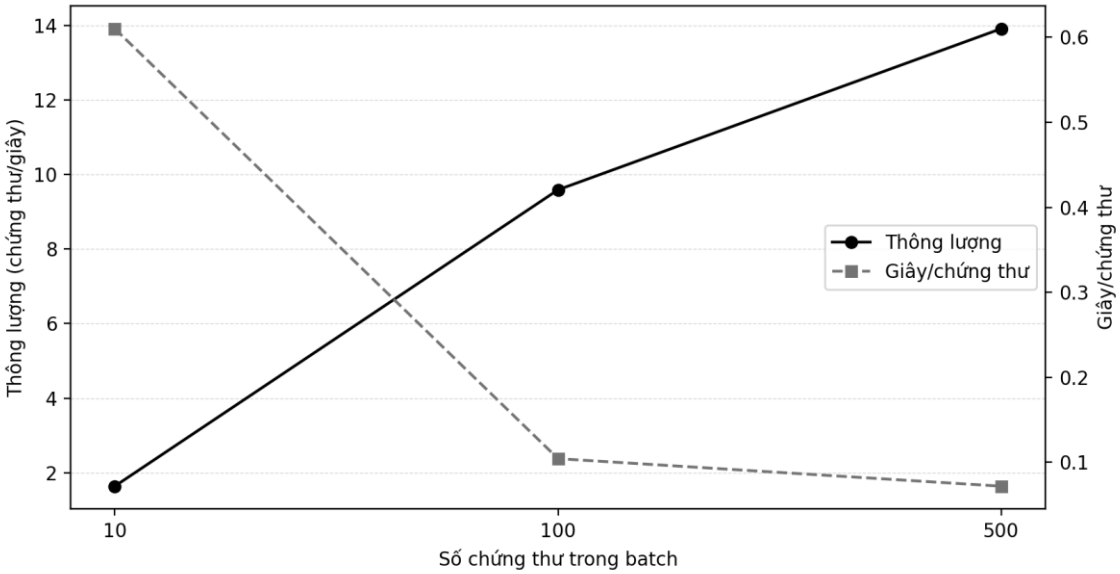
Qua ba quy mô lô thử nghiệm, khi số lượng chứng thư trong lô tăng gấp 50 lần (từ 10 lên 500 chứng thư), tổng thời gian phát hành quan sát trung bình chỉ tăng khoảng 5,89 lần (từ 6,101 giây lên 35,951 giây). Biểu đồ cột so sánh thời gian phát

hành quan sát trung bình giữa ba kích thước lô (10, 100 và 500 chứng thư) được thể hiện trên Hình 3.13.



Hình 3.13. Thời gian phát hành trung bình theo kích thước lô

Nhờ cơ chế gom nhóm vào một cây Merkle và neo bằng một giao dịch Bitcoin duy nhất, thông lượng xử lý trung bình tăng mạnh từ 1,6392 chứng thư/giây ở lô 10 lên 9,5868 chứng thư/giây ở lô 100 và đạt 13,9080 chứng thư/giây ở lô 500; tương ứng, thời gian xử lý trung bình tính trên mỗi chứng thư giảm 88,21% (từ 0,6101 giây/chứng thư xuống còn 0,0719 giây/chứng thư). Cần lưu ý rằng đây là xu hướng hiệu năng đo được giữa các kích thước lô khác nhau trong môi trường Bitcoin regtest, không phải phép đối chứng với hệ thống không áp dụng gom lô và không phải cam kết hiệu năng trên mạng Bitcoin mainnet. Biểu đồ đường trục tung kép biểu diễn tương quan giữa thông lượng (trục tung bên trái, tính bằng chứng thư/giây) và thời gian xử lý trung bình trên mỗi chứng thư (trục tung bên phải, tính bằng giây/chứng thư) được minh họa trên Hình 3.14.



Hình 3.14. Thông lượng và thời gian trung bình trên mỗi chứng thư

3.9.3. Đánh giá mức độ sử dụng tài nguyên hệ thống

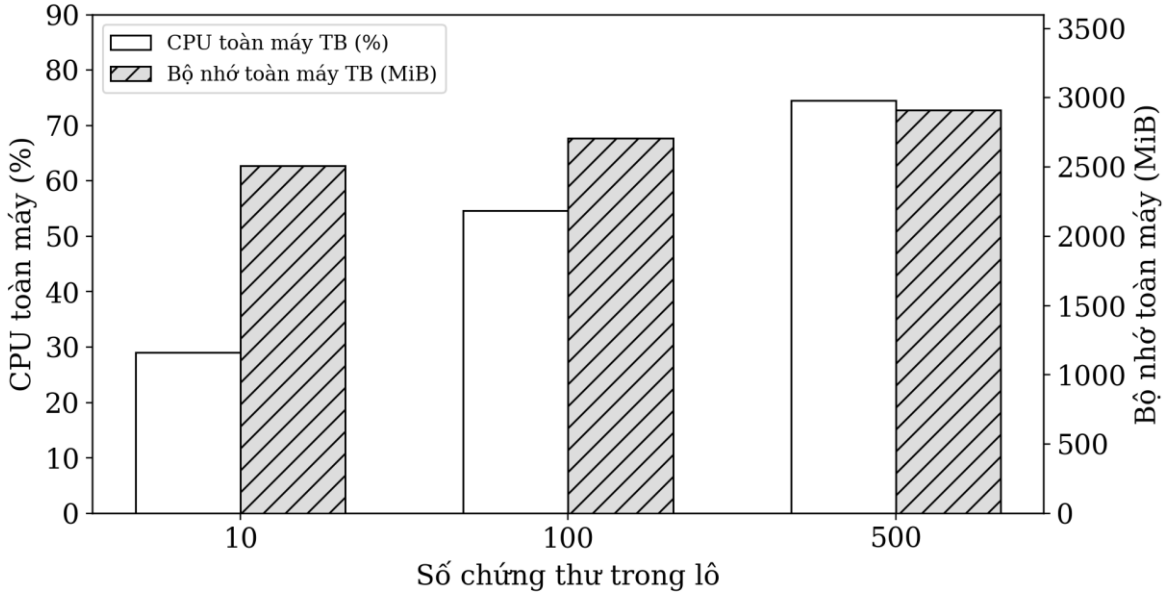
Bên cạnh thời gian xử lý và thông lượng, mức tiêu thụ tài nguyên vi xử lý (CPU) và bộ nhớ (RAM, RSS) của máy chủ trong suốt quá trình phát hành theo lô được ghi nhận và tổng hợp tại Bảng 3.11.

Bảng 3.11. Tài nguyên hệ thống theo kích thước lô

Kích thước lô	CPU TB / đỉnh (%)	RAM TB / đỉnh (MiB)	RSS backend TB / đỉnh (MiB)
10	29,00 / 42,71	2508,9 / 2543,7	166,9 / 169,4
100	54,62 / 100,00	2705,6 / 3042,8	201,5 / 205,3
500	74,45 / 100,00	2912,7 / 3323,4	221,3 / 235,6

Số liệu trong Bảng 3.11 đặt các chỉ số thời gian phát hành của mục trước vào bối cảnh tiêu thụ tài nguyên phần cứng thực tế của máy chủ 4 vCPU, bao gồm tỷ lệ sử dụng CPU toàn máy, dung lượng RAM tiêu thụ toàn máy và bộ nhớ thường trú (RSS) riêng của tiến trình backend.

CPU được lấy từ /proc/stat ở mức toàn máy; bộ nhớ được tính bằng MemTotal − MemAvailable từ /proc/meminfo, cũng ở mức toàn máy. RSS backend lấy riêng từ /proc/<pid>/status. Runner lấy mẫu mỗi giây và tại đầu/cuối lượt đo; TB là trung bình mẫu trong lượt, sau đó tổng hợp ba lượt; cột đỉnh là trung bình các cực đại từng lượt. Bộ nhớ chia cho 2^{20} byte nên đơn vị là MiB. Số liệu toàn máy có thể bao gồm tiến trình khác, không được diễn giải là tài nguyên riêng của backend. CPU cực đại đạt 100% ở nhóm 100 và 500; đỉnh bộ nhớ trung bình của nhóm 500 khoảng 3.323,4 MiB. Hình 3.15 thể hiện cấu hình 4 vCPU với tối đa 16 chunk đồng thời, không phải 16 người dùng đồng thời.



Hình 3.15. Mức sử dụng tài nguyên theo kích thước lô

3.9.4. Kiểm toán tính toàn vẹn của tệp dữ liệu kết quả

Kiểm toán sau benchmark không chỉ đếm số tệp. Script đối chiếu certificate ID, target hash, Merkle proof, issuanceBatchId, transaction đã xác nhận và Merkle root trong OP_RETURN. Sau đó, bước xác minh đầy đủ đọc toàn bộ chứng thư bằng cert-verifier-js. Kết quả cho thấy 1.830/1.830 proof khớp và 1.830/1.830 chứng thư được kết luận VALID. Kết quả được tổng hợp tại Bảng 3.12.

Bảng 3.12. Kết quả kiểm toán tệp kết quả

Chỉ số	Kết quả
Lần chạy	9/9 đạt
Chứng thư phát hành	1.830/1.830
Certificate ID/target hash duy nhất	1.830/1.830
Merkle proof được đối chiếu	1.830/1.830
Xác minh đầy đủ bằng cert-verifier-js	1.830/1.830 VALID
Lô phát hành/giao dịch đã xác nhận	9/9
Payload trong manifest	35/35
Tệp công bố qua SHA256SUMS	37/37

Kết quả thực nghiệm Kịch bản 3 – Quy trình thu hồi, phân quyền RBAC và chuỗi băm nhật ký

Kịch bản 3 kiểm tra quyền thu hồi và tính nhất quán giữa Bitcoin, PostgreSQL, Revocation List, Verify và audit. Chỉ Checker được phép thu hồi. Tệp nguyên vẹn trước thu hồi phải cho kết quả VALID và sau thu hồi phải cho kết quả REVOKED. Tám ca chính được tổng hợp tại Bảng 3.13.

Bảng 3.13. Kết quả Kịch bản 3

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S3-01	Trước thu hồi	VALID	VALID	Đạt
S3-02	Student thu hồi	HTTP 403	HTTP 403	Đạt
S3-03	Maker thu hồi	HTTP 403	HTTP 403	Đạt
S3-04	Checker thu hồi	HTTP 201	HTTP 201	Đạt
S3-05	Sau thu hồi	REVOKED	REVOKED	Đạt
S3-06	Tx thu hồi xác nhận	≥1 confirmation	1	Đạt

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S3-07	Có trong Revocation List	Có	Có	Đạt
S3-08	Audit đủ role/action	4 action	request/approve/issue/revoke	Đạt

Ngoài tám ca trên, functional E2E còn kiểm tra các điều kiện gia cố an toàn không nên gộp vào ba kịch bản cũ: duyệt từng phần, Maker-to-Maker isolation, Student isolation và privacy của verification log. Các kiểm tra bổ sung được tổng hợp tại Bảng 3.14.

Bảng 3.14. Một số kiểm tra E2E bổ sung của phiên bản hiện tại

Kiểm tra	Quan sát cuối	Kết quả
Hai lần duyệt từng phần cùng requestBatchId	Sinh hai issuanceBatchId khác nhau	Đạt
Maker B list phiếu Maker A	Không xuất hiện	Đạt
Maker B đọc trực tiếp phiếu Maker A	HTTP 404	Đạt
Maker B đọc audit mục tiêu của Maker A	Không xuất hiện	Đạt
Student khác đọc chứng thư	Không xuất hiện	Đạt
Verification log chứa name/email/studentCode	Không tìm thấy	Đạt
Quyền tệp chứng thư trong workspace	0600	Đạt

3.11. Đánh giá kết quả, hạn chế và hướng phát triển

3.11.1. Mức độ đáp ứng yêu cầu

Tổng hợp kết quả đối chiếu giữa các yêu cầu kỹ thuật cốt lõi đã đề ra ở Chương 2 với bằng chứng thực nghiệm đạt được tại Chương 3 được trình bày tại Bảng 3.15.

Bảng 3.15. Đối chiếu yêu cầu chính và bằng chứng

Yêu cầu	Bằng chứng	Kết quả
Maker không tự duyệt	S1-06 + functional E2E	Đạt

Yêu cầu	Bằng chứng	Kết quả
Phát hiện sửa chứng thư/bằng chứng	S1-02 đến S1-05	Đạt
Phát hành lô 10/100/500	9 lần chạy, 1.830 chứng thư	Đạt
Một giao dịch mỗi lô	9 lô / 9 giao dịch	Đạt
Duyệt từng phần không xung đột lô	Functional E2E	Đạt
Holder/Maker object isolation	Functional E2E	Đạt
Chỉ Checker thu hồi	S3-02 đến S3-04	Đạt
Verify nhận biết thu hồi	S3-05 đến S3-07	Đạt
Phục hồi sau lỗi quanh broadcast	Recovery E2E	Đạt
Build/test/readiness	49/49 backend; 16/16 verifier; 3 frontend; 10 migration	Đạt

Kết quả tổng hợp tại Bảng 3.15 xác nhận toàn bộ mười nhóm yêu cầu kỹ thuật cốt lõi đều có bằng chứng kiểm thử tự động và thực nghiệm đầu-cuối tương ứng; ma trận truy vết chi tiết từ từng mã yêu cầu sang thành phần hiện thực và tệp bằng chứng được trình bày đầy đủ tại Phụ lục 5.

3.11.2. Những điểm hạn chế của nguyên mẫu

Mặc dù nguyên mẫu đã đáp ứng đầy đủ các yêu cầu thiết kế trong phạm vi đồ án, hệ thống vẫn tồn tại bảy điểm hạn chế kỹ thuật cần được nhìn nhận khách quan. Thứ nhất, môi trường mạng thử nghiệm cục bộ Bitcoin regtest không phản ánh biến động phí giao dịch, độ trễ xác nhận khối, sự cạnh tranh trong vùng nhớ chờ (mempool), hiện tượng tái tổ chức chuỗi (reorg) và mô hình an ninh kinh tế của mạng Bitcoin mainnet. Thứ hai, thực nghiệm hiệu năng mới được tiến hành trên một máy chủ đơn lẻ 4 vCPU với ba lần lặp cho mỗi kích thước lô, do đó chưa đại diện cho điều kiện vận hành dài hạn, mô hình phân tán nhiều nút mạng hoặc tải truy cập đồng thời lớn. Thứ ba, khóa riêng của đơn vị phát hành hiện lưu trên tệp hệ thống chứ chưa được bảo vệ bằng thiết bị phần cứng chuyên dụng HSM hoặc dịch vụ KMS; hệ thống cũng chưa tích hợp xác thực đa nhân tố (MFA), dịch vụ quản lý bí mật tập trung và cấu hình HTTPS/HSTS hoàn chỉnh cho môi trường công cộng.

Thứ tư, danh sách thu hồi (Revocation List) hiện vẫn truy vấn trạng thái từ cơ sở dữ liệu PostgreSQL và chưa có bộ lập chỉ mục (indexer) riêng để tái dựng toàn bộ trạng thái thu hồi thuần túy từ dữ liệu trên chuỗi khối; cấu trúc tải trọng REVOKE:<certUid> trong trường OP_RETURN là quy ước mở rộng của nguyên mẫu nên khả năng tương tác với các bộ xác minh Blockcerts bên ngoài cần được đánh giá bổ sung. Thứ năm, cơ chế chuỗi băm nhật ký (Audit Hash Chain) mới chỉ giúp phát hiện hành vi sửa đổi hoặc chèn xóa cục bộ trong bảng nhật ký, thao tác ghi nhật ký chưa được thực thi nguyên tử trong cùng giao dịch với mọi thay đổi dữ liệu nghiệp vụ và giá trị băm đỉnh chuỗi chưa được neo định kỳ sang một kho lưu trữ chỉ-ghi (append-only) độc lập.

Thứ sáu, kịch bản kiểm thử phục hồi (Recovery E2E) mới tập trung vào điểm lỗi ngay sau bước phát sóng giao dịch chuỗi khối, chưa bao phủ toàn diện các tình huống tiến trình bị dừng đột ngột ở mọi thời điểm, mất kết nối RPC kéo dài, hỏng phân vùng lưu trữ (volume), quy trình sao lưu – khôi phục (backup/restore) hoặc sự cố phần cứng máy chủ. Thứ bảy, mười ca kiểm thử chống SSRF đã kiểm chứng các mẫu tấn công đại diện theo thiết kế nhưng không thay thế cho hoạt động kiểm thử xâm nhập độc lập. Ngoài ra, bộ ảnh chụp giao diện tại Mục 3.5 đóng vai trò minh họa trực quan cho một lượt trình diễn nghiệp vụ chuẩn, trong khi toàn bộ số liệu định lượng và kết quả kiểm thử tự động được trích xuất từ bộ bằng chứng tái kiểm tra đóng băng ngày 30/09/2026.

3.11.3. Hướng phát triển và hoàn thiện

Để nâng cấp nguyên mẫu lên mức độ sẵn sàng triển khai trong thực tế, các hướng nghiên cứu và phát triển tiếp theo tập trung vào việc chuẩn hóa mô hình dữ liệu theo hướng tách biệt độc lập các bảng yêu cầu, lô phát hành, chứng thư và hồ sơ thu hồi; xây dựng bộ lập chỉ mục (indexer) trên chuỗi để tự động đồng bộ và phục hồi trạng thái thu hồi; neo định kỳ giá trị băm đỉnh của chuỗi nhật ký kiểm toán vào kho lưu trữ độc lập; tích hợp HSM/KMS để bảo vệ khóa ký và tách riêng tiến trình Worker có đặc quyền ký khỏi máy chủ API; đồng thời bổ sung xác thực đa nhân tố (MFA), giao thức HTTPS/HSTS, cơ chế sao lưu – phục hồi tự động, hệ thống giám sát tập trung và kiểm thử xâm nhập định kỳ. Trước khi chuyển sang mạng thử nghiệm công cộng (testnet) hoặc mạng chính (mainnet), cơ sở đào tạo cần xây dựng chính sách số khối xác nhận tối thiểu, cơ chế dự toán phí giao dịch, đánh giá khả năng tương thích với các bộ xác minh bên ngoài và ban hành quy chế vận hành phù hợp với quy định pháp luật hiện hành [6]. Đối với dữ liệu cá nhân của người học, hệ thống cần bổ sung chính sách thời hạn lưu trữ, mã hóa các bản sao lưu và quy trình

đáp ứng quyền của chủ thể dữ liệu theo đúng Luật Bảo vệ dữ liệu cá nhân và Nghị định hướng dẫn thi hành [7], [8].

3.12. Kết luận chương 3

Chương 3 đã trình bày chi tiết quy trình cài đặt, triển khai và kiểm chứng hệ thống trong môi trường phòng thí nghiệm cô lập, đi từ bước chuẩn bị hạ tầng đến vận hành thực tế ba cổng giao diện Web. Chương đã làm rõ cách hiện thực hóa cơ chế phát hành Blockcerts V3 theo lô dựa trên cây Merkle, mẫu thiết kế Transactional Outbox, điểm kiểm soát trạng thái (checkpoint) phục vụ đối soát phục hồi lỗi, quy trình xác minh công khai bốn trạng thái tích hợp bộ lọc chống SSRF và cơ chế thu hồi chứng thư. Toàn bộ chu trình nghiệp vụ trên giao diện Web đã được kiểm chứng xuyên suốt qua mười bước thực nghiệm: từ Maker tra cứu hồ sơ và lập phiếu, Checker kiểm tra và phê duyệt, tiến trình Worker phát hành bất đồng bộ, người học nhận và tải chứng thư JSON, xác minh tính hợp lệ trên cổng Verify, phát hiện chứng thư bị sửa đổi nội dung cho đến thu hồi văn bằng.

Trong đợt tái kiểm tra đóng băng bằng chứng cuối cùng, hệ thống đạt 12/12 bộ kiểm thử đơn vị và tích hợp ở tầng backend (49/49 ca kiểm thử), đạt 16/16 ca kiểm thử ở dịch vụ xác minh (gồm 10 ca chống SSRF và 6 ca phân loại trạng thái), đồng thời vượt qua toàn bộ kịch bản kiểm thử chức năng đầu-cuối (Functional E2E) và kiểm thử phục hồi sau sự cố (Recovery E2E). Chín lượt đo hiệu năng trên ba quy mô lô (10, 100 và 500 hồ sơ) đã phát hành thành công 1.830/1.830 chứng thư chỉ với 9 giao dịch neo trên mạng Bitcoin regtest; bước kiểm toán độc lập xác nhận 1.830/1.830 bằng chứng Merkle khớp tuyệt đối và 100% tệp kết quả trong bộ bằng chứng đều toàn vẹn theo mã băm SHA-256.

KẾT LUẬN

Đồ án đã hoàn thành toàn bộ mục tiêu đề ra trong việc nghiên cứu, phân tích, thiết kế và xây dựng nguyên mẫu hệ thống thông tin hỗ trợ quản lý, phát hành, xác minh và thu hồi chứng thư số theo chuẩn Blockcerts V3 trên nền tảng Bitcoin. Hệ thống phân định rõ quyền hạn của các vai trò người lập phiếu (Maker), người phê duyệt (Checker) và người học (Student); tổ chức ba cổng giao diện Web chuyên biệt gồm cổng nghiệp vụ, cổng người học (Holder) và cổng xác minh công khai (Verify); đồng thời kết hợp chặt chẽ quy trình kiểm soát chéo Maker–Checker, phân quyền theo vai trò (RBAC), cô lập dữ liệu theo đối tượng, cơ chế gom lô cây Merkle (Merkle batching), mẫu thiết kế Transactional Outbox, điểm kiểm soát phục hồi (checkpoint/reconciliation) và chuỗi băm nhật ký kiểm toán (Audit Hash Chain). Kiến trúc tuân thủ nghiêm ngặt nguyên tắc tối thiểu hóa dữ liệu trên chuỗi khi chỉ neo giá trị gốc Merkle 32-byte và dấu vết thu hồi qua trường OP_RETURN, trong khi toàn bộ dữ liệu cá nhân và hồ sơ nghiệp vụ được quản lý bảo mật ngoài chuỗi tại cơ sở dữ liệu PostgreSQL.

Kết quả thực nghiệm trên máy chủ phòng thí nghiệm ghi nhận 12/12 bộ kiểm thử backend đạt tuyệt đối (49/49 ca kiểm thử), 16/16 ca kiểm thử dịch vụ xác minh đạt yêu cầu, cùng sự vận hành chính xác của hai kịch bản kiểm thử tích hợp Functional E2E và Recovery E2E. Qua chín lượt đo kiểm hiệu năng ở các quy mô lô 10, 100 và 500 hồ sơ, hệ thống đã phát hành thành công 1.830/1.830 chứng thư điện tử thông qua 9 giao dịch neo trên mạng Bitcoin regtest, nâng thông lượng xử lý trung bình từ 1,6392 chứng thư/giây (lô 10) lên 13,9080 chứng thư/giây (lô 500) và xác minh độc lập 1.830/1.830 chứng thư đạt trạng thái hợp lệ (VALID).

Các kết quả thực nghiệm khẳng định nguyên mẫu đã giải quyết hiệu quả bài toán đặt ra trong phạm vi mạng Bitcoin regtest và bộ dữ liệu giả lập: ngăn chặn rủi ro một cá nhân tự lập và tự duyệt văn bằng, phát hiện chính xác mọi can thiệp sửa đổi nội dung chứng thư hoặc bằng chứng Merkle, tối ưu hóa chi phí neo dữ liệu theo lô, bảo đảm cô lập dữ liệu giữa các người dùng, hỗ trợ xác minh công khai bốn trạng thái và thu hồi văn bằng an toàn. Tuy nhiên, tác giả xác định rõ kết quả trên môi trường regtest không đồng nghĩa với việc hệ thống đã sẵn sàng vận hành thương mại hoặc triển khai trực tiếp trên mạng Bitcoin mainnet. Tệp chứng thư Blockcerts trong đồ án đóng vai trò là bằng chứng kỹ thuật phục vụ nghiên cứu; giá trị pháp lý của văn bằng số trong thực tiễn vẫn phụ thuộc vào thẩm quyền cấp phát của cơ sở giáo dục, quy trình công nhận tốt nghiệp và sự tuân thủ quy chế quản lý văn bằng, chứng chỉ hiện hành [6].

Về mặt kỹ thuật, đồ án mang lại năm đóng góp cụ thể. Thứ nhất, thiết kế và hiện thực hóa mô hình kiểm soát hai bước Maker–Checker kết hợp phân quyền RBAC và cô lập đối tượng ở tầng API, triệt tiêu nguy cơ một tài khoản đơn lẻ tự khép kín quy trình cấp phát. Thứ hai, xây dựng đường ống phát hành Blockcerts V3 theo lô bất đồng bộ, cho phép gom tối đa 500 hồ sơ vào một gốc Merkle và một giao dịch Bitcoin duy nhất. Thứ ba, tách biệt định danh lô yêu cầu (requestBatchId) với định danh lô phát hành (issuanceBatchId), kết hợp bảng hàng đợi trung gian (outbox), mã công việc ổn định và điểm kiểm soát giao dịch thô để hỗ trợ phê duyệt từng phần và phục hồi lũy đẳng sau sự cố. Thứ tư, hoàn thiện quy trình xác minh bốn trạng thái (VALID, INVALID, REVOKED, INDETERMINATE) đi kèm cơ chế phòng chống tấn công SSRF, chuỗi băm nhật ký kiểm toán và thu hồi có đối soát trạng thái. Thứ năm, xây dựng bộ kịch bản tái lập thực nghiệm tự động kèm hồ sơ bằng chứng đóng băng có kiểm tra mã băm SHA-256.

Bên cạnh những kết quả đạt được, nguyên mẫu vẫn tồn tại một số hạn chế kỹ thuật cần tiếp tục giải quyết trước khi áp dụng vào thực tế. Môi trường Bitcoin regtest chưa phản ánh biến động phí giao dịch, độ trễ hàng đợi mempool, hiện tượng tái tổ chức khối (reorg) và mức độ phân tán của mạng mainnet; khóa riêng phát hành hiện lưu trên tệp hệ thống với quyền 0600 chứ chưa được bảo vệ bằng thiết bị phần cứng HSM hoặc dịch vụ KMS; hệ thống cũng chưa tích hợp xác thực đa nhân tố (MFA), cấu hình HTTPS/HSTS công khai, kiến trúc sẵn sàng cao (HA) và cơ chế sao lưu – phục hồi tự động. Ngoài ra, chuỗi băm nhật ký kiểm toán hiện mới phát hiện can thiệp cục bộ trong cơ sở dữ liệu mà chưa neo định kỳ giá trị băm đỉnh chuỗi ra kho lưu trữ độc lập; danh sách thu hồi (Revocation List) vẫn truy vấn từ PostgreSQL và chưa có bộ lập chỉ mục (indexer) để tái dựng hoàn toàn trạng thái thu hồi thuần túy từ chuỗi khối. Để phát triển hệ thống lên mức độ triển khai thực tế, các hướng nghiên cứu tiếp theo sẽ tập trung vào việc tách riêng tiến trình Worker có đặc quyền ký khỏi máy chủ API, tích hợp HSM/KMS để bảo vệ khóa, xây dựng bộ lập chỉ mục thu hồi trên chuỗi, bổ sung MFA cùng giao thức TLS, mở rộng kiểm thử tải đồng thời quy mô lớn, đánh giá khả năng tương tác với các bộ xác minh Blockcerts bên ngoài và hoàn thiện quy trình quản trị vòng đời dữ liệu cá nhân theo đúng quy định của Luật Bảo vệ dữ liệu cá nhân và các văn bản hướng dẫn thi hành [7], [8].

TÀI LIỆU THAM KHẢO

[1] D. Yaga, P. Mell, N. Roby và K. Scarfone, "Blockchain Technology Overview," NISTIR 8202, National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.IR.8202.

[2] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Có tại: https://bitcoin.org/bitcoin.pdf (truy cập ngày 02/10/2026).

[3] P. Schmidt, "Certificates, Reputation, and the Blockchain," MIT Media Lab, 2015. Có tại: https://www.media.mit.edu/posts/certificates-reputation-and-the-blockchain/ (truy cập ngày 02/10/2026).

[4] Blockcerts, "cert-schema: JSON schemas and context for Blockcerts," commit 1e390ac, 31/07/2025. Có tại: https://github.com/blockchain-certificates/cert-schema/tree/1e390ace91fb2b09518e45760168f344ef3a4409 (truy cập ngày 02/10/2026).

[5] Blockcerts, "jsonld-signatures-merkleproof2019: MerkleProof2019 suite," commit 26a9c30, 25/08/2026. Có tại: https://github.com/blockchain-certificates/jsonld-signatures-merkleproof2019/tree/26a9c3071355a9ccd83b0b90fc375a4d8cf1d4b8 (truy cập ngày 02/10/2026).

[6] Bộ Giáo dục và Đào tạo, "Thông tư số 10/2026/TT-BGDĐT ngày 26/02/2026 ban hành Quy chế văn bằng, chứng chỉ của hệ thống giáo dục quốc dân," có hiệu lực từ 15/04/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?classid=1&docid=217140&pageid=27160&typegroupid=6 (truy cập ngày 02/10/2026).

[7] Quốc hội, "Luật số 91/2025/QH15 ngày 26/06/2025: Luật Bảo vệ dữ liệu cá nhân," có hiệu lực từ 01/01/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?docid=214590&pageid=27160 (truy cập ngày 02/10/2026).

[8] Chính phủ, "Nghị định số 356/2025/NĐ-CP ngày 31/12/2025 quy định chi tiết một số điều và biện pháp thi hành Luật Bảo vệ dữ liệu cá nhân," có hiệu lực từ 01/01/2026. Cổng Thông tin điện tử Chính phủ. Có tại: https://vanban.chinhphu.vn/?classid=1&docid=216387&pageid=27160 (truy cập ngày 02/10/2026).

[9] Bitcoin Project, "Bitcoin Developer Guide – Testing Applications," 2024. Có tại: https://developer.bitcoin.org/examples/testing.html (truy cập ngày 02/10/2026).

[10] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi và J. Wang, "Untangling Blockchain: A Data Processing View of Blockchain Systems," IEEE Transactions on Knowledge and Data Engineering, tập 30, số 7, tr. 1366–1385, 2018. DOI: 10.1109/TKDE.2017.2781227.

[11] R. C. Merkle, "Secrecy, Authentication, and Public Key Systems," Luận án tiến sĩ, Stanford University, 1979.

[12] M. Castro và B. Liskov, "Practical Byzantine Fault Tolerance," Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), 1999, tr. 173–186. Có tại: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance (truy cập ngày 02/10/2026).

[13] D. Ongaro và J. Ousterhout, "In Search of an Understandable Consensus Algorithm," 2014 USENIX Annual Technical Conference, 2014, tr. 305–319. Có tại: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro (truy cập ngày 02/10/2026).

[14] E. Androulaki và cộng sự, "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains," Proceedings of the Thirteenth EuroSys Conference, 2018, tr. 1–15. DOI: 10.1145/3190508.3190538.

[15] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.

[16] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5.

[17] W3C, "Verifiable Credentials Data Model v2.0," W3C Recommendation, 15/05/2025. Có tại: https://www.w3.org/TR/2025/REC-vc-data-model-2.0-20250515/ (truy cập ngày 02/10/2026).

[18] W3C, "JSON-LD 1.1," W3C Recommendation, 16/07/2020. Có tại: https://www.w3.org/TR/2020/REC-json-ld11-20200716/ (truy cập ngày 02/10/2026).

[19] W3C, "Verifiable Credentials Data Model v1.1," W3C Recommendation, 09/11/2021. Có tại: https://www.w3.org/TR/2021/REC-vc-data-model-20211109/ (truy cập ngày 02/10/2026).

[20] OWASP Foundation, "Authorization Cheat Sheet," OWASP Cheat Sheet Series, 2024. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (truy cập ngày 02/10/2026).

[21] NestJS, "NestJS Documentation," 2024. Có tại: https://docs.nestjs.com/ (truy cập ngày 02/10/2026).

[22] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation – Transactions," 2024. Có tại: https://www.postgresql.org/docs/15/tutorial-transactions.html (truy cập ngày 02/10/2026).

[23] TypeORM, "TypeORM Documentation – Migrations and Transactions," 2024. Có tại: https://typeorm.io/docs/advanced-topics/migrations/ và https://typeorm.io/docs/advanced-topics/transactions/ (truy cập ngày 02/10/2026).

[24] BullMQ, "BullMQ Documentation," 2024. Có tại: https://docs.bullmq.io/ (truy cập ngày 02/10/2026).

[25] M. Sporny và D. Longley (biên tập), "Verifiable Credential Data Integrity 1.0," W3C Recommendation, 15/05/2025. Có tại: https://www.w3.org/TR/vc-data-integrity/ (truy cập ngày 02/10/2026).

[26] M. Jones, J. Bradley và N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, 2015. Có tại: https://www.rfc-editor.org/rfc/rfc7519 (truy cập ngày 02/10/2026).

[27] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series, 2024. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (truy cập ngày 02/10/2026).

[28] OWASP Foundation, "Server-Side Request Forgery Prevention Cheat Sheet," OWASP Cheat Sheet Series, 2024. Có tại: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html (truy cập ngày 02/10/2026).

PHỤ LỤC

Phụ lục 1. Danh mục API chính và quyền truy cập

Bảng PL1.1 đối chiếu các API với controller trong bản phát hành ổn định Release v1.0. Giao diện chỉ hỗ trợ trải nghiệm; backend quyết định cho phép hoặc từ chối bằng JWT, RolesGuard và kiểm tra phạm vi đối tượng. Maker chỉ đọc dữ liệu trong phạm vi của mình; Student chỉ nhận chứng thư gắn với subject hiện tại.

Bảng PL1.1. API chính của nguyên mẫu

Phương thức	Đường dẫn	Quyền	Mục đích
POST	/api/auth/login	Công khai	Đăng nhập và nhận JWT
POST	/api/auth/register	Checker	Tạo tài khoản
GET	/api/admin/users	Checker	Liệt kê tài khoản
POST	/api/admin/users	Checker	Tạo tài khoản từ cổng nghiệp vụ
PATCH	/api/admin/users/:id/student-profile	Checker	Cập nhật hồ sơ Student
GET	/api/audit	Maker, Checker	Đọc audit theo phạm vi
GET	/api/audit/integrity	Maker, Checker	Kiểm tra chuỗi hash audit
POST	/api/issue/request	Maker	Lập một phiếu
POST	/api/issue/batch/request	Maker	Lập lô 1–500 phiếu
GET	/api/issue/students/:studentCode	Maker	Tra cứu Student chỉ đọc
POST	/api/issue/:id/approve	Checker	Duyệt một phiếu
POST	/api/issue/batch/approve	Checker	Duyệt nhiều phiếu
POST	/api/issue/batch/:batchId/retry	Checker	Retry lô thất bại
POST	/api/issue/:id/reject	Checker	Từ chối phiếu
GET	/api/issue	Maker, Checker	Danh sách phiếu/văn bằng theo phạm vi

Phương thức	Đường dẫn	Quyền	Mục đích
GET	/api/issue/:id	Maker, Checker	Chi tiết theo phạm vi
GET	/api/issue/holder/certificates	Student	Chứng thư của Student hiện tại
POST	/api/revoke/check	Công khai	Kiểm tra trạng thái thu hồi
POST	/api/revoke/:id	Checker	Thu hồi chứng thư
POST	/api/verify	Công khai	Xác minh chứng thư JSON
GET	/api/blockcerts/issuers/kma/profile.json	Công khai	Issuer Profile
GET	/api/blockcerts/issuers/kma/revocation-list.json	Công khai	Revocation List
GET	/health	Công khai	Liveness tối thiểu

Phụ lục 2. Khung cấu hình tái lập và bảo vệ bí mật

Các nhóm cấu hình trong Bảng PL2.1 được phân thành dữ liệu công khai, dữ liệu nội bộ và bí mật. Tệp mẫu chỉ nêu tên biến; giá trị thật phải được sinh riêng cho từng môi trường. Mật khẩu, JWT secret, Bitcoin RPC credential và WIF không được đưa vào Git, ảnh chụp hoặc bộ bằng chứng công khai. Tệp runtime và khóa riêng phải giới hạn quyền đọc; dữ liệu người học dùng trong thử nghiệm phải là dữ liệu giả lập.

Bảng PL2.1. Nhóm cấu hình và mức nhạy cảm

Nhóm	Ví dụ	Nguyên tắc
PostgreSQL	POSTGRES_PASSWORD, DATABASE_URL	Sinh riêng; chỉ mở trong phạm vi cần thiết
Redis	REDIS_PASSWORD, REDIS_URL	Không công bố; chỉ dùng cho queue/runtime
JWT	JWT_SECRET, JWT_EXPIRES_IN	JWT_SECRET ngẫu nhiên đủ mạnh; thời hạn token cấu hình riêng; không ghi log hoặc công bố secret

Nhóm	Ví dụ	Nguyên tắc
Bitcoin RPC	BITCOIN_RPC_USER, BITCOIN_RPC_PASSWORD	Chỉ backend/worker cần truy cập
Khóa phát hành	WIF trong storage/credentials	Không commit; quyền tệp hạn chế; production cần HSM/KMS
Issuer public	Issuer Profile, revocation URL	Được công bố; không chứa khóa riêng
Frontend	API base URL	Không chứa credential
Bằng chứng	Manifest, checksum, kết quả test	Loại secret và dữ liệu cá nhân không cần thiết trước khi công bố

Phụ lục 3. Quy trình tái lập và lệnh đại diện

Phụ lục 3 là danh sách lệnh tra cứu nhanh; Mục 3.2 của Chương 3 mới là hướng dẫn đầy đủ gồm trình tự, lệnh và kết quả đối chiếu. Không gọi trực tiếp các kịch bản nội bộ benchmark/E2E ngoài chuỗi chính nếu không có mục đích gỡ lỗi.

C1. Cố định mã nguồn

```
$ git clone https://github.com/ebkk24/DATN-AT190226.git
$ cd DATN-AT190226
$ git checkout main  # hoặc git checkout tags/v1.0.0
$ git status --short
```

C2. Chuẩn bị stack tái lập. prepare.sh tự sinh runtime/.env và secret, build backend/frontend, dựng PostgreSQL/Redis/Bitcoin, sinh cấu hình Blockcerts và chạy migration; không có tệp .env.repro.example cần sao chép.

```
$ bash experiments/repro/scripts/prepare.sh --reset
$ bash experiments/repro/scripts/start.sh
$ bash experiments/repro/scripts/health.sh
```

Kết quả health đạt:

```
HEALTH_PASS containers=3 migrations=10 api=200 admin=200 student=200 verify=200
```

C3. Chạy kiểm thử mặc định. run-tests.sh tự tạo source manifest với --require-clean, tự start/health, bootstrap tài khoản, chạy backend/verifier test, Functional E2E và Recovery E2E.

```
$ bash experiments/repro/scripts/run-tests.sh
TEST_PASS default=PASS benchmark=SKIPPED
```

C4. Chạy benchmark đầy đủ. Runner tự chạy 10/100/500 × 3 với đúng --size/--repeat, sau đó kiểm toán tệp kết quả và bước xác minh đầy đủ; không gọi benchmark.mjs trống.

```
$ bash experiments/repro/scripts/run-tests.sh --benchmark
```

```
TEST_PASS default=PASS benchmark=PASS
```

C5. Chuẩn bị demo web sau khi test/benchmark. Lệnh quick giữ image, migration và blockchain regtest nhưng làm sạch dữ liệu nghiệp vụ, queue và tệp runtime, rồi tạo lại tài khoản.

```
$ bash experiments/repro/scripts/demo-reset.sh --dry-run
$ bash experiments/repro/scripts/demo-reset.sh --quick
```

```
DEMO_READY mode=quick health=PASS accounts=READY data=CLEAN
```

C6. Dừng hoặc xóa stack tái lập

```
$ bash experiments/repro/scripts/stop.sh --app-only
$ bash experiments/repro/scripts/stop.sh --down
# Chỉ khi cần xóa toàn bộ stack repro:
$ bash experiments/repro/scripts/stop.sh --purge
```

Khi một bước thất bại, không xóa checkpoint hoặc broadcast thủ công để "làm sạch" trạng thái. Với phát hành, kiểm tra issuance_batches, issuance_outbox, jobId và checkpoint của worker; với thu hồi, kiểm tra revocationIntentTxid, revocationRawTransaction và trạng thái revocation_pending/revocation_reconciliation_required. Chỉ retry sau khi xác định nguyên nhân và phải sử dụng lại định danh/giao dịch đã checkpoint.

Phụ lục 4. Bộ bằng chứng thực nghiệm chính thức

Bảng PL4.1 tổng hợp bộ kết quả thực nghiệm chính thức được lưu có cấu trúc tại experiments/results/final-revalidation-20260930/. Kiểm thử và đo sử dụng mã nguồn sạch của Release v1.0, với runId 20260930T103044Z-3582ddaa. Cách tổ chức này giúp truy vết và kiểm tra độc lập các số liệu hiệu năng và nhật ký audit.

Bảng PL4.1. Thành phần bộ bằng chứng và mục đích

Tệp/nhóm tệp	Nội dung kiểm chứng
source-manifest.json	Commit, trạng thái cây nguồn và fingerprint của tập tệp được kiểm soát
backend-jest.json	Kết quả 12/12 suite, 49/49 backend test
functional-e2e.json	Maker–Checker, duyệt từng phần, Holder, object isolation, verify và revoke

Tệp/nhóm tệp	Nội dung kiểm chứng
recovery.json	Phục hồi quanh broadcast, giữ nguyên txid và không tạo anchor thứ hai
benchmark.json	Dữ liệu thô của 9 lần chạy cho lô 10, 100 và 500
benchmark-summary.json	Giá trị tổng hợp thời gian, thông lượng và tài nguyên
benchmark-artifact-audit.json	Đối chiếu 1.830 chứng thư, bằng chứng Merkle, lô, giao dịch và manifest
manifests/*.json	Manifest theo từng lô/lần chạy, gồm ID, txid, gốc Merkle và tệp kết quả
SHA256SUMS	Checksum của bộ tệp công bố

Các chỉ số chốt gồm: 9/9 lượt benchmark đạt; 1.830/1.830 chứng thư được phát hành; 9/9 lô tương ứng với 9 giao dịch đã xác nhận; 1.830/1.830 Merkle proof được đối chiếu; xác minh đầy đủ kết luận 1.830/1.830 chứng thư VALID; manifest và SHA-256SUMS khớp với bộ tệp công bố. Các chỉ số chỉ áp dụng cho regtest và cấu hình máy thử nghiệm trong Chương 3.

Phụ lục 5. Ma trận truy vết yêu cầu – hiện thực – bằng chứng

Bảng PL5.1 liên kết các yêu cầu quan trọng với thành phần hiện thực và bằng chứng thực thi. Trong phạm vi đồ án, một yêu cầu chỉ được coi là đã kiểm chứng khi có cả cơ chế hiện thực và kết quả thử nghiệm tương ứng; kết quả regtest không được suy rộng thành kết luận về production/mainnet.

Bảng PL5.1. Ma trận truy vết yêu cầu và bằng chứng

Yêu cầu	Thành phần hiện thực	Bằng chứng chính
Maker không tự duyệt	JWT, RolesGuard, Checker-only approve	S1-06; functional E2E trả HTTP 403
Cô lập Maker theo đối tượng	Filter requestedBy ở list/detail/audit	Functional E2E: Maker B không thấy dữ liệu Maker A
Cô lập Student	studentId từ subject JWT	Functional E2E với hai Student
Duyệt từng phần	requestBatchId/issuanceBatchId; migration 10	Hai lượt duyệt tạo hai issuanceBatchId khác nhau

Yêu cầu	Thành phần hiện thực	Bằng chứng chính
Phát hành bền vững	issuance_batches, issuance_outbox, jobId ổn định	Backend test + functional E2E
Phục hồi sau broadcast	Điểm kiểm soát giao dịch thô/txid, đối soát trạng thái	Recovery E2E giữ nguyên txid
Phát hiện sửa chứng thư	Merkle proof + anchor verification	S1-02 đến S1-05 trả INVALID
Thu hồi đúng quyền	Checker-only revoke + checkpoint thu hồi	S3-02 đến S3-07; Verify trả REVOKED
Phân loại lỗi xác minh	VALID/INVALID/REVOKED/INDETERMINATE	16/16 verifier test
Chống SSRF	Safe-fetch, DNS/IP/redirect checks	10/10 regression test SSRF
Kiểm toán	Audit hash chain + integrity endpoint	Backend test; /api/audit/integrity
Triển khai thực nghiệm và kiểm toán số liệu	Source manifest, manifest theo run, checksum	9/9 lần chạy; 1.830/1.830 xác minh đầy đủ; SHA256SUMS